

TechTribers

Java Interview Series · Part 14 · Q580–Q679

Strings
The Complete Guide

The string pool, memory and immutability, every method, StringBuilder and concatenation, Unicode and regex, best practices, and 31 string coding problems, all run on a real JDK.

100	99	15	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `ST-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

100	9	99	15
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🏊	01	The String Pool & Interning	Q580–Q589	6
🧬	02	Immutability & Memory Internals	Q590–Q598	15
🔎	03	Methods I: Searching, Comparing & Inspecting	Q599–Q608	24
🛠️	04	Methods II: Transforming, Splitting & Formatting	Q609–Q617	34
🏗️	05	StringBuilder, Concatenation & Joiners	Q618–Q625	44
🌐	06	Unicode, Charsets & Locale	Q626–Q632	53
🧩	07	Regular Expressions in Java	Q633–Q640	60
✅	08	Best Practices & Pitfalls	Q641–Q648	69
💻	09	String Coding Questions	Q649–Q679	78

© TechTribers. For personal use only. Do not redistribute.

All questions

🏊 The String Pool & Interning

Q580 What is the String pool (string table), and what ends up in it? 6

Q581 How many objects does `new String("hello")` create? 7

Q582 Predict the output: == vs equals() with literals, new, concatenation and intern(). 8

Q583 Which expressions are compile-time constants, and why does `final` change `==`? 9

Q584 How big is the string table, and what is -XX:StringTableSize? 10

Q585 Where did the string pool live before Java 7, and why was it moved? 11

Q586 Can string literals and interned strings be garbage-collected? 11

Q587 Which String operations return pooled (or the same) instances, and which create new objects? 12

Q588 When should you intern strings yourself, and what are the alternatives? 13

Q589 How does a switch on a String work, and what happens with null? 14

🧬 Immutability & Memory Internals

Q590 Can String immutability be broken with reflection, and what would happen? 15

Q591 What does a String object look like in memory? 16

Q592 What are compact strings, and how much memory do they save? 17

Q593 How is String.hashCode() calculated, and why is it cached? 18

Q594 Why should passwords be stored in a char[] instead of a String? 19

Q595 How has substring() changed, and what was the old memory leak? 20

Q596 How do Strings cause memory problems, and how do you find them? 21

Q597 Why is String declared final, and what can you do instead of subclassing it? 22

Q598 Why is String such a good HashMap key? 23

🔎 Methods I: Searching, Comparing & Inspecting

Q599 What are the edge cases of indexOf() and lastIndexOf()? 24

Q600 contains vs indexOf vs matches vs regionMatches: which should you use? 25

Q601 equals vs contentEquals vs equalsIgnoreCase vs compareTo: which is right when? 26

Q602 How should you sort strings: compareTo, CASE_INSENSITIVE_ORDER or Collator? 27

Q603 charAt vs codePointAt vs chars() vs codePoints(): how do you iterate characters correctly? 28

Q604 How do you check prefixes and suffixes safely (offsets, case, URLs and extensions)? 29

Q605 What exceptions do charAt and substring throw, and what do the messages say? 30

Q606 What is CharSequence, and why do so many APIs accept it instead of String? 31

Q607 What are the ways to count occurrences of a character or substring? 32

Q608 What are all the ways to construct a String? 33

🛠️ Methods II: Transforming, Splitting & Formatting

Q609 What are the exact boundary rules of substring()? 34

Q610 What are the pitfalls of the replacement string in replaceAll() ($ and backslash)? 35

Q611 Why does split() drop trailing empty strings, and how does the limit argument work? 36

Q612 What did Java 11–21 add to String (repeat, lines, strip, indent, transform, formatted...)? 37

Q613 Why must you pass Locale.ROOT to toLowerCase/toUpperCase? (the Turkish i problem) 38

Q614 What are the most useful String.format specifiers and flags, and what goes wrong? 39

Q615 How do you convert between String, char[], byte[], int[] code points, List<Character> and streams? 40

Q616 How do you parse numbers from strings safely? (parseInt quirks) 41

Q617 How are double and float converted to strings, and why does 0.1 + 0.2 print 0.30000000000000004? 42

© TechTribers. For personal use only. Do not redistribute.

🏗️ StringBuilder, Concatenation & Joiners

Q618 How does the + operator compile? (StringBuilder in Java 8, invokedynamic in Java 9+) 44

Q619 Why is s += x inside a loop slow, and what are the fixes? 45

Q620 How does StringBuilder work internally, and how does its capacity grow? 46

Q621 What can StringBuilder do? (insert, delete, replace, reverse, setLength...) 48

Q622 Why doesn't StringBuilder.equals() compare content, and how do you compare builders? 49

Q623 How do StringJoiner, String.join and Collectors.joining differ? 50

Q624 What are the efficient ways to build large or multi-line text? 51

Q625 Why doesn't StringBuffer's synchronisation make shared text building safe? 52

🌐 Unicode, Charsets & Locale

Q626 What is the difference between char, code point and grapheme cluster, and why does length() surprise people? 53

Q627 How do you encode and decode strings safely (charsets, mojibake, malformed input)? 54

Q628 What is Unicode normalisation, and how do you do accent-insensitive comparison? 55

Q629 Which String methods depend on the default Locale or charset? 56

Q630 How does case-insensitive comparison behave across Unicode (equalsIgnoreCase, toLowerCase, β, ı, Σ)? 57

Q631 What do the Character methods and Unicode regex classes check, and why isn't isLetter the same as [A-Za-z]? 58

Q632 How do you reverse or truncate text safely when it contains surrogate pairs and combining marks? 59

🧩 Regular Expressions in Java

Q633 What are the main parts of the Pattern and Matcher API (matches, find, lookingAt, groups, replace, results)? 60

Q634 How do capturing, non-capturing and named groups and backreferences work? 61

Q635 What is the difference between greedy, reluctant and possessive quantifiers? 62

Q636 How do lookahead, lookbehind and word boundaries work (and what are their traps)? 63

Q637 What do the Pattern flags do, and what are Pattern.quote and LITERAL for? 64

Q638 What is catastrophic backtracking (ReDoS), and how do you defend against it? 65

Q639 What are reliable regex recipes (email, IPv4, date, phone), and what are their limits? 66

Q640 What does regex cost, and why do String.matches, replaceAll and split compile a Pattern on every call? 68

✅ Best Practices & Pitfalls

Q641 What are the key best practices when working with Strings in Java? 69

Q642 What are the most common String pitfalls, and how do you spot them in code review? 70

Q643 How should you handle null and empty strings? 72

Q644 Why avoid "stringly-typed" code, and what should replace it? 73

Q645 What are the string-related security risks (SQL, log, format-string injection, path traversal)? 74

Q646 How do you externalise and format messages for different languages (ResourceBundle, MessageFormat, ChoiceFormat)? 75

Q647 Why use parameterised logging instead of string concatenation? 76

Q648 What are the practical String performance tips? 77

© TechTribers. For personal use only. Do not redistribute.

💻 String Coding Questions

Q649 Reverse a string (several ways, including surrogate-safe). 78

Q650 Check whether a string is a palindrome (exact, and ignoring case and punctuation). 79

Q651 Check whether two strings are anagrams (sorting, counting, Unicode-safe). 80

Q652 Count vowels, consonants, digits and spaces in a string. 81

Q653 Remove duplicate characters from a string, keeping the first occurrence. 82

Q654 Check whether a string contains only digits, only letters or only alphanumerics (ASCII vs Unicode). 82

Q655 Convert a sentence to Title Case (capitalise each word). 83

Q656 Convert between camelCase and snake_case. 84

Q657 Remove all whitespace, or collapse multiple spaces into one. 85

Q658 Implement a Caesar cipher (encrypt and decrypt). 86

Q659 Decode a run-length encoded string ("a3b2c1" becomes "aaabbc"). 87

Q660 Check whether two strings are isomorphic. 88

Q661 Add two non-negative numbers given as strings (without BigInteger). 89

Q662 Multiply two non-negative numbers given as strings. 90

Q663 Validate whether a string is a valid integer or decimal number without using parse methods. 91

Q664 Find the longest substring with at most K distinct characters (sliding window). 92

Q665 Check whether all characters in a string are unique. 93

Q666 Find the longest common substring of two strings. 94

Q667 Check whether one string is a subsequence of another. 94

Q668 Count the words in a sentence (spaces, punctuation and tabs). 95

Q669 Count palindromic substrings in a string. 96

Q670 Reverse only the vowels in a string. 97

Q671 Remove adjacent duplicates from a string using a stack (and the k-duplicates variant). 98

Q672 Generate all combinations of well-formed parentheses for n pairs. 99

Q673 Convert a number to an Excel column title (1 → A, 27 → AA) and back. 100

Q674 Simplify a Unix file path ("/a/./b/../../c/" becomes "/c"). 100

Q675 Check whether a sentence is a pangram (contains every letter a-z). 101

Q676 Generate the count-and-say (look-and-say) sequence. 102

Q677 Parse a CSV line with quoted fields and escaped quotes. 103

Q678 Implement your own split() for a single-character delimiter (keeping empty fields). 104

Q679 Check whether two strings are at most one edit apart (insert, delete or replace one character). 105

© TechTribers. For personal use only. Do not redistribute.

Section 01

The String Pool & Interning

Where literals live, how == and equals() really behave, which operations are pooled, and how the JVM implements string switch.

Questions 580–589 · 10 questions



Q580 ST-001 • Basic

What is the String pool (string table), and what ends up in it?

⚡ Quick answer

The string pool (the JVM's **string table**) is a hash table of unique `String` objects on the heap. Every **string literal** and every **compile-time constant expression** is resolved to the one pooled instance, so identical literals share a single object. Strings created at run time with `new`, concatenation of variables, `substring` and similar are **not** pooled unless you call `intern()`.

Why it exists: strings are immutable, so sharing is safe, and programs repeat the same literals constantly (class names, keys, messages). Sharing saves memory and makes `==` on literals cheap. Since Java 7 the pool lives on the **regular heap** (it was in PermGen before), so pooled strings are ordinary garbage-collectable objects.

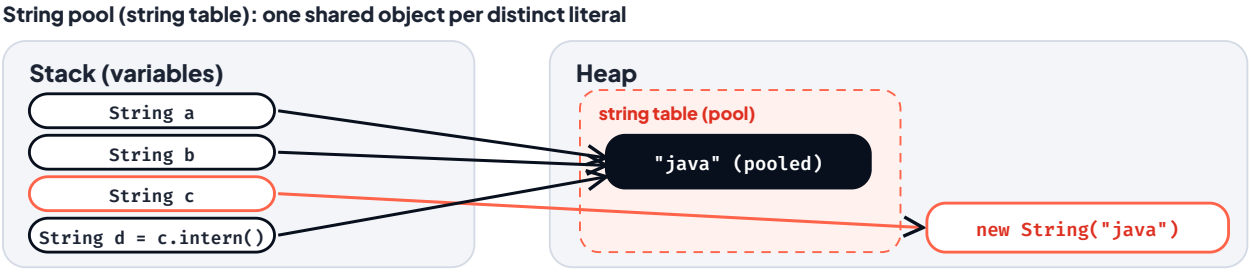
Java

```
String a = "java";                      // literal: resolved from the string pool
String b = "java";                      // the same literal: the SAME pooled object
String c = new String("java");          // explicit new: a separate heap object
String d = c.intern();                  // intern() returns the pooled instance
System.out.println((a == b) + " " + (a == c) + " " + (a == d) + " " + a.equals(c));
String e = "ja" + "va";                 // constant expression: folded at compile time, then pooled
System.out.println(a == e);
String part = "ja";
String f = part + "va";                 // runtime concatenation: a NEW object, not pooled
System.out.println((a == f) + " " + (a == f.intern()));
System.out.println(System.identityHashCode(a) == System.identityHashCode(b));
```

▶ Output

```
true false true true
true
false true
true
```


© TechTribers. For personal use only. Do not redistribute.



a == b → true (same pooled object) · a == c → false · a == d → true · all four equals() each other

⚠️ **Interview trap**

Pool membership is an optimisation detail. Never depend on `==` for string comparison in application code: it works for literals by accident and fails for strings that came from user input, files, or `StringBuilder`.

Q581 ST-002 • Basic

How many objects does `new String("hello")` create?

⚡ **Quick answer**

At most **two String objects**: the pooled literal `"hello"` (created when the literal is first resolved, if it wasn't already) and the new heap `String` produced by `new`. The characters are **not** copied twice: the new `String` shares the original's internal `byte[]`. If the literal was already in the pool, `new String("hello")` allocates exactly one new 24-byte object.

The reflection lines show what is really shared. `new String(String)` is rarely useful: its only legitimate use was trimming a large backing array before Java 7u6 changed `substring`. Today it only wastes 24 bytes and defeats pooling, so prefer the literal itself.

Java

```
String lit = "hello";                                  // pooled
String s1 = new String("hello");                       // one more String object (the literal
already exists)
String s2 = new String("hello");                       // and another
System.out.println((s1 == s2) + " " + (s1 == lit) + " " + s1.equals(lit));
System.out.println(s1.intern() == lit);                // interning gives back the pooled one

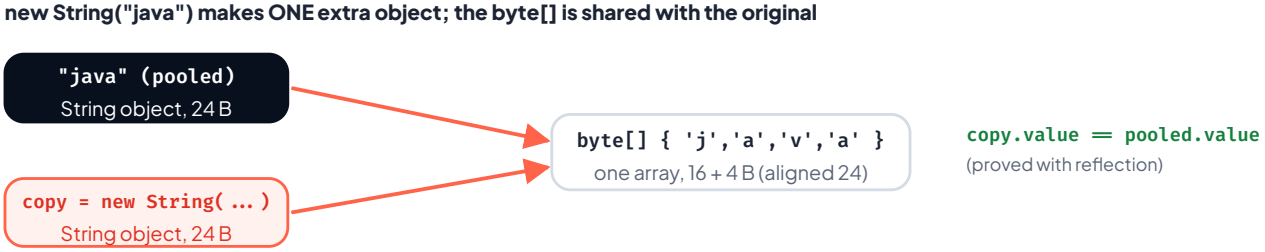
Field value = String.class.getDeclaredField("value"); value.setAccessible(true);
System.out.println(value.get(s1) == value.get(lit));    // the new String SHARES the original's
byte[]
System.out.println(value.get(new String(lit.toCharArray())) == value.get(lit));   // the char[]
constructor copies the characters
System.out.println(new String(new char[]{'h', 'i'}).intern() == "hi");
```

▶ **Output**

```
false false true
true
```


© TechTribers. For personal use only. Do not redistribute.

```
true
false
true
```



Cost of the explicit new: 24 bytes. The characters are not copied (the constructor reuses the array reference).

> 💡 **Pro tip**
> Interview answer: "One in the pool if it isn't there yet, one on the heap: so 1 or 2 String objects, sharing one byte array."

🎯 Q582 ST-003 • Intermediate

Predict the output: == vs equals() with literals, new, concatenation and intern().

> ⚡ **Quick answer**
> `==` compares references. Literals and constant expressions are pooled (so `==` is true), while `new String(...)`, runtime concatenation and most method results are fresh objects (so `==` is false) until you `intern()` them. Operands that are `final` variables initialised with constants are still constants.

Line 1, in order: `s1 == s2` literals → true; `s1 == s3` `new` → false; `s1 == s4` constant folding → true; `s1 == s5` runtime concat of a non-final variable → false; `s1 == s6` concat with a **final** constant variable → true; `s1 == s7` interned → true.

Last line: `+` is left-associative. `"a" + 1 + 2` is string concatenation from the start (`a12`); `1 + 2 + "a"` adds the numbers first (`3a`); `'a' + 'b'` is integer addition (97 + 98 = 195) before the string appears (`195c`); `"c" + 'a' + 'b'` concatenates (`cab`).

Java

```
String s1 = "hello", s2 = "hello", s3 = new String("hello"), s4 = "hel" + "lo";
String h = "hel"; String s5 = h + "lo";
final String fh = "hel"; String s6 = fh + "lo";
String s7 = s3.intern();
System.out.println((s1 == s2) + " " + (s1 == s3) + " " + (s1 == s4) + " " + (s1 == s5) + " " + (s1
== s6) + " " + (s1 == s7));
System.out.println(("hel" + "lo" == s1) + " " + (h + "lo" == s1) + " " + (s1 == s5.intern()));
System.out.println(s1.equals(s3) + " " + s1.equals(s5) + " " + "HELLO".equalsIgnoreCase(s5));
System.out.println(("a" + 1 + 2) + " " + (1 + 2 + "a") + " " + ('a' + 'b' + "c") + " " + ("c" + 'a'
+ 'b'));
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
true false true false true true
true false true
true true true
a12 3a 195c cab
```

⚠️ Interview trap

This exact question (with variations) is asked in almost every Java screening. The two rules to state out loud: constants are folded and pooled; anything computed at run time is a new object.

Q583 ST-004 • Advanced

Which expressions are compile-time constants, and why does `final` change `==`?

⚡ Quick answer

A **constant expression** consists of literals, operators and **constant variables** (a `final` variable of primitive or `String` type initialised with a constant expression). The compiler folds it into one literal, which is pooled. A non-`final` variable, a method call, or a `null` is not constant, so the result is computed at run time and is a new object.

Why `c` is not constant: it is `final`, but its initialiser is a method call, so the compiler can't know its value. Why `"x" + null` is not constant: `null` is not a constant expression. The same rules decide whether a `switch` case label, an annotation value or a `static final` field can use an expression (it must be constant).

Java

```
static final String PREFIX = "api/";
static String suffix() { return "v1"; }
final String a = "x";  String b = "x";  final String c = suffix();
System.out.println(("x" + "y" == "xy"));
System.out.println((a + "y" == "xy") + " " + (b + "y" == "xy") + " " + (c + "y" == "xy"));      //
only constant variables fold
final String t = true ? "x" : "z";                                                              //
a constant conditional
System.out.println(t + "y" == "xy");
System.out.println(("x" + 1 == "x1") + " " + ("x" + 1.5 == "x1.5") + " " + ("x" + 'c' == "xc") + " "
+ ("x" + null == "xnull"));
System.out.println(PREFIX + "v1" == "api/v1");
```

▶ Output

```
true
true false false
true
true true true false
true
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If a puzzle changes only `final` or only the initialiser, ask: "is every operand a constant variable?" That decides the answer in two seconds.

📏 **Q584** ST-005 • Advanced

How big is the string table, and what is -XX:StringTableSize?

> ⚡ **Quick answer**
> The string table is a hash table whose **bucket count** is set by `-XX:StringTableSize` (65,536 by default in current JDKs, as printed by `-XX:+PrintFlagsFinal` below). Interning a very large number of distinct strings with a small table makes the buckets long and every `intern()` slower. Modern JDKs resize the table, but if you intern millions of strings deliberately, set a larger value and measure.

Guidelines: choose a **prime** near the number of strings you expect to keep; the statistics output shows the bucket count, entry count, and average and maximum bucket length (long maximum chains mean the table is too small). Interning is not free: each call hashes the string and probes the table, and every pooled string stays reachable as long as it is referenced elsewhere.

Most applications never need this flag. If you are interning to save memory, compare with G1's **string deduplication** (`-XX:+UseStringDeduplication`, covered in the JVM part), which shares character arrays in the background without changing identity.

Terminal

```
# default size (measured on JDK 25)
java -XX:+PrintFlagsFinal -version | grep -E "StringTableSize|CompactStrings"
#   uintx StringTableSize        = 65536   {product} {default}
#   bool  CompactStrings         = true    {pd product} {default}

# bigger table for an application that interns millions of distinct strings
java -XX:StringTableSize=1000003 -XX:+PrintStringTableStatistics -jar app.jar

# inspect a running JVM
jcmd <pid> VM.stringtable
```

> ⚠️ **Interview trap**
> Interning user-controlled input (request parameters, log lines) lets an attacker grow the table without bound. Intern only values drawn from a small, known set.

© TechTribers. For personal use only. Do not redistribute.

Q585 ST-006

• Advanced

Where did the string pool live before Java 7, and why was it moved?

⚡ Quick answer

Up to Java 6 the pool lived in **PermGen**, a fixed-size region outside the normal heap. Interned strings could never be collected and the region's size was fixed (`-XX:MaxPermSize`), so heavy interning caused `OutOfMemoryError: PermGen space` . **Java 7** moved the pool onto the regular heap, where pooled strings are garbage-collected like other objects. **Java 8** removed PermGen altogether (class metadata moved to Metaspace).

Java	Pool location	Consequence
≤ 6	PermGen (fixed size)	Interned strings never freed; PermGen OOM; advice to avoid `intern()`
7	Main heap	Pooled strings collectable; `intern()` safe to use; `-XX:StringTableSize` tunes the hash table
8+	Main heap (no PermGen)	Same; Metaspace holds class metadata
9+	Main heap, compact strings	Latin-1 strings store one byte per character

Related Java 7 changes in the same release: `String.substring` stopped sharing the parent's array (7u6), which removed a classic memory leak, and `String.offset` / `count` fields were dropped.

💡 Pro tip

Old advice such as "never call intern() because of PermGen" is out of date. Today the question is only whether interning pays for itself (hash cost vs. memory saved).

Q586 ST-007

• Advanced

Can string literals and interned strings be garbage-collected?

⚡ Quick answer

Yes, since Java 7. The string table holds its entries **weakly**: an interned string that nothing else references becomes garbage like any other object. A **literal** stays alive while the class that contains it is loaded (its constant pool references the string), and is freed after that class loader is unloaded and a GC runs.

So a long-running service that interns a bounded vocabulary is fine, while one that interns unbounded input pays in table size and GC work (see the string-table question). A literal in a class you keep loaded (the normal case) lives for the whole process.

```java
String unique = new String("temp-" + System.nanoTime()).intern();      // a pooled string nothing else will reference
```


© TechTribers. For personal use only. Do not redistribute.

```
WeakReference<String> ref = new WeakReference<>(unique);
System.out.println(ref.get() != null);
unique = null;                                                    // drop the only strong
reference
for (int i = 0; i < 40 && ref.get() != null; i++) { System.gc(); Thread.sleep(50); }     // GC
timing is not guaranteed, so retry
System.out.println("collected: " + (ref.get() == null));
String literal = "kept-literal";
WeakReference<String> litRef = new WeakReference<>(literal);
System.gc();
System.out.println("literal still alive: " + (litRef.get() != null));
```

▶ Output

```
true
collected: true
literal still alive: true
```

⚠️ **Interview trap**

Don't assume `System.gc()` collects anything deterministically. The loop above retries because a collection is only a request; the result is reliable here but not guaranteed by the language.

🔍 **Q587** ST-008 • Advanced

Which String operations return pooled (or the same) instances, and which create new objects?

⚡ **Quick answer**

Only literals, constant expressions and `intern()` results are pooled. Many methods return `this` **when they have nothing to change (** `substring(0)`, `trim()` **with no whitespace,** `toLowerCase()` **on already-lowercase text,** `replace` **with no match,** `concat("")`, `repeat(1)` **), which is an optimisation,** not a guarantee** you may rely on.

Use this table as folklore, not as an API contract: the Javadoc for `trim`, `toLowerCase` and `replace` doesn't promise to return the same instance, and implementations may change. The reliable rules are only these: literals and constant expressions are pooled, `intern()` returns the pooled instance, everything else is unspecified.

© TechTribers. For personal use only. Do not redistribute.

```java
String pooled = "hello";
System.out.println(("hello world".substring(0, 5) == pooled) + " " + (new String("hello") == pooled)
+ " " + (String.valueOf(new char[]{'h','e','l','l','o'}) == pooled));   // new objects
System.out.println((pooled.concat("") == pooled) + " " + (pooled.substring(0) == pooled) + " " +
(pooled.trim() == pooled) + " " + (pooled.strip() == pooled));       // nothing to change: returns
this
System.out.println((pooled.toLowerCase() == pooled) + " " + (pooled.toUpperCase() == pooled) + " " +
(pooled.replace('z', 'y') == pooled) + " " + (pooled.repeat(1) == pooled));
System.out.println((String.valueOf(pooled) == pooled) + " " + (pooled.toString() == pooled) + " " +
(pooled.intern() == pooled) + " " + ("hel".concat("lo") == pooled));
```

▶ Output

```
false false false
true true true true
true false true true
true true true false
```

⚠️ **Interview trap**

Code such as `if (s.trim() == s)` to detect "no whitespace" works today by accident. Compare with `equals` or check `s.isBlank()` / `s.strip().length()` explicitly.

🔧 **Q588** ST-009 • Advanced

When should you intern strings yourself, and what are the alternatives?

⚡ **Quick answer**

Intern (or canonicalise) strings when a **bounded** set of values is repeated millions of times (country codes, status names, tags parsed from big files) and memory matters. Alternatives: a **custom interner** backed by a `ConcurrentHashMap` (no global table, you control the lifetime), G1 **string deduplication** (JVM-level, no code change), or storing small vocabularies as **enums** or ids.

Compare the options: `String.intern()` is simple and shared JVM-wide but uses the native table (and its size flag); a custom `ConcurrentHashMap` interner is plain Java, can be cleared when the data set is replaced, and can use weak values (Guava `Interners.newWeakInterner()`); G1 deduplication needs no code but only shares the byte arrays of strings that survived a few GCs. In every case measure first: Latin-1 compact strings already halve the cost of ASCII text.

```java
class Interner {
    private final ConcurrentHashMap<String, String> canon = new ConcurrentHashMap<>();
    String intern(String s) { String prev = canon.putIfAbsent(s, s); return prev == null ? s : prev;
}
    int size() { return canon.size(); }
}
List<String> raw = new ArrayList<>();
```


© TechTribers. For personal use only. Do not redistribute.

```
for (int i = 0; i < 1000; i++) raw.add(new String(i % 2 == 0 ? "ACTIVE" : "INACTIVE"));     // 1000 distinct String objects, 2 distinct values
System.out.println(Collections.newSetFromMap(new IdentityHashMap<String, Boolean>()).addAll(raw) + " " + countObjects(raw));
Interner in = new Interner();
List<String> canon = new ArrayList<>();
for (String s : raw) canon.add(in.intern(s));
System.out.println(countObjects(canon) + " " + in.size() + " " + raw.equals(canon));

static int countObjects(List<String> list) { Set<String> ids = Collections.newSetFromMap(new IdentityHashMap<>()); ids.addAll(list); return ids.size(); }
```

▶ Output

```
true 1000
2 2 true
```

💡 **Pro tip**

The `IdentityHashMap`-backed set is the standard trick for counting **distinct objects** (as opposed to distinct values), handy when investigating duplicates.

Q589 ST-010 • Intermediate

How does a switch on a String work, and what happens with null?

⚡ **Quick answer**

The compiler translates `switch (s)` into two steps: a `lookupswitch` on `s.hashCode()` to pick a candidate, then `s.equals(label)` to confirm (hash collisions such as `"Aa"` and `"BB"`, both 2112, are handled by the `equals` chain), and finally a second switch on an int index to run the case. `switch (null)` throws `NullPointerException` because `hashCode()` is invoked on the selector.

Consequences: string switch is O(1) on average (hash), case labels must be **compile-time constants** and unique, and the match is **case-sensitive**. With `case null ->` (Java 21 pattern switch) you can handle null explicitly. For large domains prefer an `enum` (compile-time checked, exhaustive) over string labels.

Java

```
static int code(String s) { switch (s) { case "Aa": return 1; case "BB": return 2; case "hello": return 3; default: return 0; } }
static String kind(String s) { return switch (s) { case "Aa", "BB" -> "collision pair"; case "hello" -> "greeting"; default -> "other"; }; }
System.out.println(code("Aa") + " " + code("BB") + " " + code("hello") + " " + code("HELLO") + " " + kind("BB") + " " + kind("x"));
try { code(null); } catch (NullPointerException e) { System.out.println("switch(null) throws NPE"); }
System.out.println("Aa".hashCode() + " " + "BB".hashCode() + " " + "hello".hashCode());

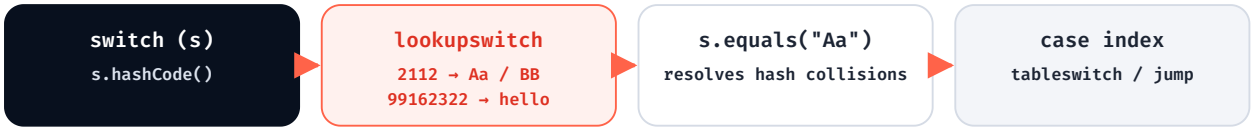
// javap -c on the compiled switch (JDK 25) -- the key instructions:
//    5: invokevirtual String.hashCode:()I
```


© TechTribers. For personal use only. Do not redistribute.

```
//    8: lookupswitch { 2112: 36, 99162322: 64 }      <- "Aa" and "BB" share the entry 2112
//   39: invokevirtual String.equals:(Ljava/lang/Object;)Z    <- confirms the candidate
```

▶ Output

```
1 2 3 0 collision pair other
switch(null) throws NPE
2112 2112 99162322
```



A switch on a String is compiled to: hashCode() → lookupswitch on the hash → equals() to confirm → case jump (real javap output).
null → NullPointerException (hashCode on null). "Aa" and "BB" share hash 2112, so both reach the same equals() chain.

💡 **Pro tip**

A common follow-up: "is a long if/else on strings slower?" Each `equals` is O(length), so a switch does one hash plus one or two `equals`, while a ladder does up to n of them.

Section 02

Immutability & Memory Internals

What a String looks like in memory, compact strings, hash caching, and whether immutability can be broken.

Questions 590–598 · 9 questions



🔓 **Q590** ST-011 • Advanced

Can String immutability be broken with reflection, and what would happen?

⚡ **Quick answer**

Technically yes, if the module system allows it: with `--add-opens java.base/java.lang=ALL-UNNAMED` you can reflectively get the private `byte[] value` and write into it. Since Java 16–17, strong encapsulation **blocks this by default** (`InaccessibleObjectException`). Doing it corrupts everything sharing that string: literals, the pool, cached hash codes and map keys.

The output shows the damage: the **literal** `"hello"` elsewhere in the program now reads `Jello`, because every `"hello"` in the JVM is the same pooled object. A mutated string that was already used as a `HashMap` key would

© TechTribers. For personal use only. Do not redistribute.

also sit in the wrong bucket (its `hash` field is cached). This is why immutability is a **safety invariant**, not just a convention.

Without the `--add-opens` flag this program fails on JDK 17+ with `InaccessibleObjectException: Unable to make field private final byte[] java.lang.String.value accessible`. That is the intended outcome of JEP 396/403.

Java

```
String s = "hello";
Field value = String.class.getDeclaredField("value");
value.setAccessible(true);
byte[] bytes = (byte[]) value.get(s);
bytes[0] = (byte) 'J';                                  // mutate the literal's backing array
in place
System.out.println(s + " " + "hello" + " " + ("hello" == s) + " " + "hello".equals("Jello"));
```

▶ Output

```
Jello hello true true
```

⚠️ **Interview trap**

If an interviewer asks "is String *really* immutable?", the strong answer is: immutable by API and by design; reflection can bypass it only by deliberately opening the module, and the JDK now blocks that by default.

📐 **Q591** ST-012 • Advanced

What does a String object look like in memory?

⚡ **Quick answer**

A `String` is **two objects**: the `String` itself (24 bytes: 12-byte header, a 4-byte reference to the array, a 4-byte cached `hash`, a 1-byte `coder`, a 1-byte `hashIsZero`, padding) and a separate `byte[]` (16-byte header plus the characters, rounded up to 8). The characters take 1 byte each for Latin-1 text and 2 bytes each otherwise.

Measured heap cost per string on JDK 25 (compressed oops, including the 4-byte slot in the holding array):

String	Measured	Arithmetic
1 character	52 B	24 + (16+1 → 24) + 4
10 chars, Latin-1	60.6 B	24 + (16+10 → 32) + 4
10 chars, UTF-16	68.2 B	24 + (16+20 → 40) + 4
100 chars, Latin-1	149.3 B	24 + (16+100 → 120) + 4

So a short string costs far more than its characters: a 10-character ASCII id uses ~60 bytes, six times its content. A million such ids are ~60 MB, which is why id-heavy code sometimes stores `long`s or `byte[]`s, interns repeated

© TechTribers. For personal use only. Do not redistribute.

values, or lets the GC deduplicate.

```java
for (Field f : String.class.getDeclaredFields())
    if (!Modifier.isStatic(f.getModifiers())) System.out.println(f.getType().getSimpleName() + " " + f.getName());
```

▶ Output

```
byte[] value
byte coder
int hash
boolean hashIsZero
```

A String is two objects: the String header + the byte[] that holds the characters

java.lang.String = 24 bytes

header	12 B
value (byte[])	4 B ref
hash (int)	4 B
coder (byte)	1 B
hashIsZero	1 B
padding	2 B

byte[] value

array header 16 B + 1 byte per char (Latin-1)
or 2 bytes per char (UTF-16), rounded up to 8 B

10-char Latin-1 string: 24 + 32 = 56 B

measured with a 4 B array slot: 60.6 B each

1-char string: 24 + 24 = 48 B (measured 52 B)

💡 **Pro tip**

With compact object headers (default in JDK 27) every object header shrinks from 12 to 8 bytes, so both objects of a String get smaller.

Q592 ST-013 • Intermediate

What are compact strings, and how much memory do they save?

⚡ **Quick answer**

Since Java 9 a `String` stores its characters as a `byte[]` plus a `coder` flag: **LATIN1** (one byte per char) when every character is ≤ U+00FF, otherwise **UTF16** (two bytes per char). Most text in business applications is Latin-1, so heap usage for strings drops substantially. One wide character (like `\u4e2d` or `\u20ac`) switches the **whole string** to UTF-16.

Measured on JDK 25: a 100-character Latin-1 string used **149 B** with compact strings (the default) and **247 B** with `-XX:-CompactStrings`, about 40% less. A 10-character UTF-16 string cost the same 68 B either way, because it needs 2 bytes per char regardless.

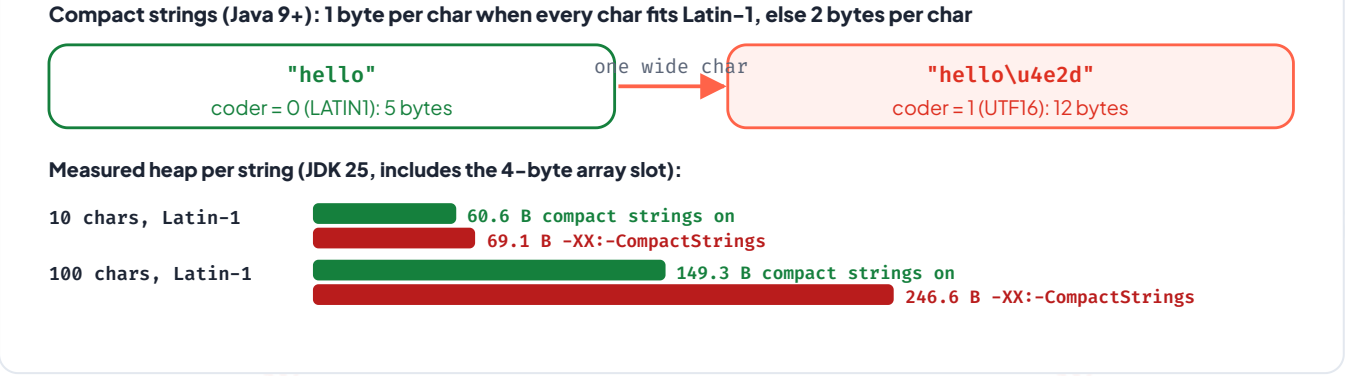
Facts to remember: `length()` and `charAt()` are unaffected (the API is still in `char` s); it is on by default (`-XX:+CompactStrings`); `StringBuilder` / `StringBuffer` use the same scheme; and mixing even one non-Latin-1 character into a long string doubles its character storage.

© TechTribers. For personal use only. Do not redistribute.

```java
Field coder = String.class.getDeclaredField("coder");
coder.setAccessible(true);
System.out.println(coder.getByte("hello") + " " + coder.getByte("caf\u00e9") + " " +
coder.getByte("\u20ac") + " " + coder.getByte("hello\u4e2d"));    // 0 = LATIN1, 1 = UTF16
System.out.println("hello".length() + " " + "hello\u4e2d".length() + " " +
"hello".getBytes(StandardCharsets.UTF_16LE).length);
```

▶ Output

```
0 0 1 1
5 6 10
```



💡 **Pro tip**

For data you store a lot of (ids, names, codes), keeping it pure Latin-1 keeps it compact. A stray emoji or smart quote in a CSV column silently doubles that string's memory.

Q593 ST-014

• Intermediate

How is String.hashCode() calculated, and why is it cached?

⚡ **Quick answer**

`hashCode()` is `s[0]·31^(n-1) + s[1]·31^(n-2) + … + s[n-1]` computed with `int` overflow, i.e. `h = 31·h + c` over the characters. The result is stored in the `hash` field on the first call (with `hashIsZero` for strings whose hash really is 0), so later calls are O(1). That makes `String` an excellent `HashMap` key, because the cost of hashing is paid once.

The formula is part of the `String.hashCode` **specification**, so it can't change between JDKs. Collisions are unavoidable: `"Aa"` and `"BB"` both hash to 2112 (the classic hash-flooding example), and `"polygenelubricants"` famously hashes to `Integer.MIN_VALUE`. `equals` settles collisions, and HashMap's tree bins limit the damage of a flood.

© TechTribers. For personal use only. Do not redistribute.

```java
String s = "hello";
Field hash = String.class.getDeclaredField("hash");
hash.setAccessible(true);
System.out.println("before: " + hash.getInt(s));
int h = 0; for (char c : s.toCharArray()) h = 31 * h + c;       // the specified formula
System.out.println(h == s.hashCode());
System.out.println("after: " + hash.getInt(s) + " " + "".hashCode());
System.out.println("Aa".hashCode() + " " + "BB".hashCode() + " " + "polygenelubricants".hashCode());
```

▶ Output

```
before: 0
true
after: 99162322 0
2112 2112 -2147483648
```

String.hashCode(): computed once from all characters, then cached in the hash field



`h = 31·h + c for every char: s[0]·31^(n-1) + s[1]·31^(n-2) + … + s[n-1]`

"Aa".hashCode() == "BB".hashCode() == 2112: collisions exist, equals() settles them.

A string whose hash really is 0 (such as "") sets hashIsZero, so it is not recomputed each time either.

⚠️ **Interview trap**

Because the hash is cached in the object, a `String` key can never change bucket. That safety is lost if the key is a mutable object, which is the reason `StringBuilder` has no value-based `hashCode`.

🔑 **Q594** ST-015 • Intermediate

Why should passwords be stored in a char[] instead of a String?

⚡ **Quick answer**

A `String` can't be erased: it is immutable, may be pooled or interned, and any copy (or the original) stays in memory until the GC reclaims it, so it can appear in heap dumps, swap files or logs long after use. A `char[]` can be **overwritten immediately** after authentication (`Arrays.fill(pw, '\0')`), shrinking the window in which the secret exists. `Console.readPassword()` and Swing's `JPasswordField` return `char[]` for this reason.

Honest limits: the guidance reduces exposure but doesn't make memory safe. `new String(password)` makes a permanent copy, GC may have **moved** the array (leaving a copy behind), the JIT or OS may copy it, and logging the password defeats everything. Real protection comes from not holding secrets longer than needed, never

© TechTribers. For personal use only. Do not redistribute.

logging them, using a **hash** (bcrypt/Argon2) for stored passwords, and a secrets manager for application secrets.

Frameworks hand you the credential as a `String` anyway (HTTP basic auth, JSON bodies). Treat `char[]` as defence in depth, not a complete answer.

Java

```
char[] password = {'s', '3', 'c', 'r', 'e', 't'};
boolean ok = Arrays.equals(password, new char[]{'s', '3', 'c', 'r', 'e', 't'});
Arrays.fill(password, '\0');                          // overwrite as soon as the check is done
int zeros = 0;
for (char c : password) if (c == 0) zeros++;
System.out.println(ok + " " + zeros);
String leaked = "s3cret";                             // a literal lives in the pool for the life
of its class; nothing can wipe it
```

▶ Output

```
true 6
```

⚠️ **Interview trap**

Passing a secret to `String.valueOf(char[])`, `String.format`, or any logging call creates a String copy and undoes the point of using an array.

✂️ **Q595** ST-016 • Intermediate

How has substring() changed, and what was the old memory leak?

⚡ **Quick answer**

Before Java 7u6, `substring` returned a new `String` that **shared the parent's `char[]` with an offset and count, which made it O(1) but meant a tiny substring kept a huge array alive (a memory leak when you extracted a few characters from a big file). From Java 7u6 on,** `substring` copies** just the needed characters, so it is O(length of the result) and substrings can't pin the parent.

Pre-7u6 workaround: `new String(s.substring(a, b))`, which copied the characters into a right-sized array. That idiom is obsolete and now just wastes an object. The trade-off today is that extracting many small substrings from a large text allocates each separately (still the right default, since it avoids leaks).

Java

```
String big = "x".repeat(1_000_000) + "KEY=42";
String small = big.substring(big.length() - 2);               // since 7u6: a copy of just 2
characters
System.out.println(small + " " + small.length());
String all = big.substring(0);
System.out.println((all == big) + " " + (big.substring(1) == big) + " " + big.substring(0,
1).equals("x"));
String cut = big.substring(big.indexOf("KEY=") + 4);
big = null;                                                    // the 1 MB string is now
```


© TechTribers. For personal use only. Do not redistribute.

```
collectable: 'cut' holds only its own 2 bytes
System.out.println(cut);
```

▶ Output

```
42 2
true false true
42
```

💡 Pro tip

The same change explains why `String` no longer has `offset` and `count` fields, as the layout diagram shows (`value` , `coder` , `hash` , `hashIsZero`).

🗑️ Q596 ST-017 • Advanced

How do Strings cause memory problems, and how do you find them?

⚡ Quick answer

Typical causes: **duplicates** (the same value stored in millions of separate objects), **unbounded** interning or static caches keyed by strings, very large strings built by concatenation, holding on to big text only to use a small part (pre-7u6 `substring` , or keeping the parent string), and **UTF-16 inflation** from one wide character. Find them with a heap dump (Eclipse MAT's *Duplicate Strings* / *Group By Value*), JFR allocation events, or `jcmd GC.class_histogram` .

Fixes in order of effort: remove the duplication at the source (parse into enums or ids); canonicalise with an interner; turn on **G1 string deduplication** (`-XX:+UseStringDeduplication` , JVM part); avoid building huge strings (stream to a `Writer` instead); and keep text Latin-1 where you can (compact strings).

In MAT, open the dominator tree and the **Duplicate Strings** report, sort by *wasted bytes*; for a live system, `jcmd <pid> GC.class_histogram` shows `java.lang.String` and `byte[]` counts at a glance.

Java

```
List<String> names = new ArrayList<>();
for (int i = 0; i < 1000; i++) names.add(new String(i % 4 == 0 ? "ENG" : i % 4 == 1 ? "OPS" : i % 4
== 2 ? "HR" : "FIN"));
Map<String, Long> duplicates = names.stream().collect(Collectors.groupingBy(s -> s, TreeMap::new,
Collectors.counting()));
System.out.println(duplicates);                                                    // 4 distinct
values stored 1000 times
Set<String> identity = Collections.newSetFromMap(new IdentityHashMap<>());
identity.addAll(names);
System.out.println(identity.size() + " distinct objects for " + new HashSet<>(names).size() + "
distinct values");
```

▶ Output

```
{ENG=250, FIN=250, HR=250, OPS=250}
1000 distinct objects for 4 distinct values
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Report waste as "distinct values vs objects" (as the output above): it turns a vague "too many strings" into a number a team can act on.

🔒 **Q597** `ST-018` • Intermediate

Why is String declared final, and what can you do instead of subclassing it?

> ⚡ **Quick answer**
> `String` is `final` so nobody can create a mutable subclass that breaks the invariants the JDK relies on: security checks that validate a string and then use it, the string pool, cached hash codes, class loading by name, and thread safety. If you need extra behaviour, use **composition** (a wrapper class), static helper methods, or the *`CharSequence`* interface, which `String`, `StringBuilder` and your own types can implement.

APIs worth knowing that accept `CharSequence`: `String.contains`, `replace`, `join`, `contentEquals`, `StringBuilder.append`, `Pattern.matcher`, `Character` methods via `chars()`. Writing your own methods against `CharSequence` (not `String`) lets callers pass a `StringBuilder`, a `CharBuffer` or a view without copying.

Java

```
final class Slice implements CharSequence {                    // a zero-copy view of part of a
String
    private final String base; private final int from, to;
    Slice(String base, int from, int to) { this.base = base; this.from = from; this.to = to; }
    @Override public int length() { return to - from; }
    @Override public char charAt(int i) { return base.charAt(from + i); }
    @Override public CharSequence subSequence(int a, int b) { return new Slice(base, from + a, from
+ b); }
    @Override public String toString() { return base.substring(from, to); }
}
CharSequence slice = new Slice("key=value;other=1", 4, 9);
System.out.println(slice + " " + slice.length() + " " + "value".contentEquals(slice) + " " +
Pattern.compile("v.l").matcher(slice).find() + " " + String.join("-", slice, "x"));
System.out.println(new StringBuilder(slice).append('!') + " " + CharSequence.compare(slice, "value")
+ " " + slice.chars().sum());
```

▶ **Output**

```
value 5 true true value-x
value! 0 541
```

> 💡 **Pro tip**
> Other final classes with the same reasoning: `Integer` and the other wrappers, `Class`, `System`, `Math`, `LocalDate` and every other value-based class.

© TechTribers. For personal use only. Do not redistribute.

Q598 ST-019 • Intermediate

Why is String such a good HashMap key?

⚡ Quick answer

Four properties: it is **immutable** (the key can't change bucket after insertion), its `hashCode` **is cached** (hashing is O(n) once, then O(1)), it has well-defined `equals` / `hashCode` with a decent distribution, and it implements *`Comparable`*, which lets HashMap use tree bins when many keys collide (limiting hash-flooding attacks). A mutable key can't offer any of these guarantees.

The `StringBuilder` and `char[]` examples show the alternative: no value equality (identity hash and `equals`), so lookups with an equal-looking key fail. If you want mutable text as a key, convert to a `String` first. Case-insensitive keys need normalisation (`toLowerCase(Locale.ROOT)`) or a `TreeMap(String.CASE_INSENSITIVE_ORDER)`.

Java

```
Map<String, Integer> map = new HashMap<>();
map.put("Aa", 1); map.put("BB", 2);                                    // same hashCode (2112), different keys
System.out.println(map.get("Aa") + " " + map.get("BB") + " " + map.get(new String("Aa")) + " " + map.size());
StringBuilder mutable = new StringBuilder("k");
Map<StringBuilder, Integer> bad = new HashMap<>();
bad.put(mutable, 1);
System.out.println(bad.get(new StringBuilder("k")) + " " + bad.get(mutable));          // identity-based: an equal builder is NOT found
mutable.append("2");
System.out.println(bad.get(mutable) + " " + bad.size() + " " + ("k".compareTo("k2") < 0));
Map<char[], Integer> arrays = new HashMap<>(); arrays.put(new char[]{'a'}, 1);
System.out.println(arrays.get(new char[]{'a'}));                                        // arrays use identity hash/equals
```

▶ Output

```
1 2 1 2
null 1
1 1 true
null
```

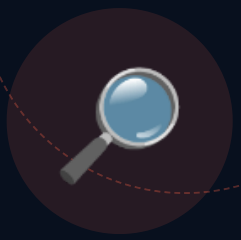
💡 Pro tip

Interview shortcut: "immutable + cached hash + proper equals/hashCode + Comparable" is the whole answer.

© TechTribers. For personal use only. Do not redistribute.

Section 03

Methods I: Searching, Comparing & Inspecting



Edge cases and corner behaviour of the inspection methods: indexOf, contains, equals family, compare, charAt and code points, regionMatches, exceptions and constructors.

Questions 599–608 · 10 questions

Q599 ST-020

• Intermediate

What are the edge cases of indexOf() and lastIndexOf()?

⚡ Quick answer

Both return the index or **-1**. `indexOf("")` returns 0 and `lastIndexOf("")` returns `length()`. A negative `fromIndex` behaves like 0, a too-large one returns -1 (and `lastIndexOf(x, fromIndex)` searches **backwards** starting at `fromIndex`). The `char` overloads actually take an `int` **code point**. Java 21 adds `indexOf(ch/str, beginIndex, endIndex)` to search a bounded range.

Overlapping matches: searching `"ana"` in `"banana"` with `i + 1` finds indexes 1 and 3; stepping by the match length (`i + pattern.length()`) would find only 1. Pick the step that matches your definition. For repeated searches over a large text, a compiled `Pattern` or a purpose-built algorithm (KMP, Boyer-Moore) is faster than repeated `indexOf` calls with different needles.

Java

```
String s = "banana";
System.out.println(s.indexOf("") + " " + s.lastIndexOf("") + " " + s.indexOf("na") + " " +
s.lastIndexOf("na") + " " + s.indexOf("x"));
System.out.println(s.indexOf('a', 2) + " " + s.indexOf("a", -5) + " " + s.indexOf("a", 99) + " " +
s.lastIndexOf("a", 3) + " " + s.lastIndexOf('a', -1));
System.out.println(s.indexOf(97) + " " + s.indexOf('a', 1, 4) + " " + s.indexOf("an", 1, 3));
// Java 21: search only inside [from, to)
List<Integer> all = new ArrayList<>();
for (int i = s.indexOf("an"); i >= 0; i = s.indexOf("an", i + 1)) all.add(i);
// find every occurrence
System.out.println(all);
```

▶ Output

```
0 6 2 4 -1
3 1 -1 3 -1
1 1 1
[1, 3]
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`indexOf` returning 0 means *found at the start*, not *found*: write `indexOf(x) >= 0` (or use `contains`), never `if (s.indexOf(x))`-style truthiness from other languages.

🔦 **Q600** ST-021 • Intermediate

contains vs indexOf vs matches vs regionMatches: which should you use?

⚡ **Quick answer**

`contains(CharSequence)` answers *"is it somewhere inside?"* (plain text, case-sensitive). `indexOf` also gives the **position**. `matches(regex)` must match the **whole** string (use `find()` for a partial match). `regionMatches(offset, other, otherOffset, len)` compares a specific **region** and has a case-insensitive overload, without creating substrings.

Cost notes: `contains` and `indexOf` are intrinsics on modern JVMs (SIMD-accelerated), so they are very fast; `matches` compiles a `Pattern` on every call. `regionMatches(true, ...)` is the right tool for a case-insensitive prefix check because `startsWith` has no case-insensitive form and `toLowerCase().startsWith` allocates.

Java

```
String s = "Hello, World";
System.out.println(s.contains("World") + " " + s.contains("") + " " + s.contains("world") + " " +
s.toLowerCase().contains("world"));
System.out.println(s.contains(new StringBuilder("lo, W")));                    // accepts
any CharSequence
System.out.println(s.matches("Hello") + " " + s.matches("Hello.*") + " " +
Pattern.compile("World").matcher(s).find());
System.out.println(s.regionMatches(7, "World", 0, 5) + " " + s.regionMatches(true, 7, "WORLD", 0, 5)
+ " " + s.regionMatches(7, "World!", 0, 6));
try { s.contains(null); } catch (NullPointerException e) { System.out.println("contains(null) throws
NPE"); }
```

▶ **Output**

```
true true false true
true
false true true
true true false
contains(null) throws NPE
```

💡 **Pro tip**

Case-insensitive `contains` has no built-in. Options: `regionMatches(true, ...)` in a loop, `Pattern.compile(Pattern.quote(x), CASE_INSENSITIVE | UNICODE_CASE)`, or lower-casing both sides with `Locale.ROOT`.

© TechTribers. For personal use only. Do not redistribute.

Q601 ST-022

• Intermediate

equals vs contentEquals vs equalsIgnoreCase vs compareTo: which is right when?

⚡ **Quick answer**

`equals` is true only for another `String` with the same characters (a `StringBuilder` with the same text is **not equal**). `contentEquals` compares with **any** `CharSequence` . `equalsIgnoreCase` **ignores case per character.** `compareTo` **gives ordering (negative, zero, positive). Put the** constant first** (`"x".equals(s)`) or use `Objects.equals` to be null-safe.

`compareTo` returns the difference of the first differing **UTF-16 code units** (`'v' - 'b'` = 20, `'a' - 'A'` = 32), or the **length difference** when one string is a prefix of the other (`4 - 10` = -6). Only the **sign** is meaningful; never rely on the magnitude.

Contract reminders: `equals` is reflexive, symmetric, transitive and null-safe (`equals(null)` is false); `compareTo` is consistent with `equals` for `String` ; and `equalsIgnoreCase` is **not** transitive-safe for all Unicode (see the Unicode section).

Java

```
String a = "Java", b = new String("Java"); StringBuilder sb = new StringBuilder("Java"); String nul
= null;
System.out.println(a.equals(b) + " " + a.equals(sb) + " " + a.contentEquals(sb) + " " +
a.equalsIgnoreCase("JAVA") + " " + a.equals(null));
System.out.println("Java".equals(nul) + " " + Objects.equals(nul, nul) + " " + Objects.equals(a,
nul));
try { nul.equals("Java"); } catch (NullPointerException e) { System.out.println("null receiver:
NPE"); }
System.out.println(a.hashCode() == b.hashCode());
System.out.println(a.compareTo("Jabc") + " " + a.compareTo("Java") + " " + a.compareTo("JavaScript")
+ " " + "apple".compareTo("Apple"));
```

▶ **Output**

```
true false true true false
false true false
null receiver: NPE
true
20 0 -6 32
```

⚠️ **Interview trap**

`s.equals("x")` throws NPE when `s` is null; `"x".equals(s)` doesn't. Many codebases enforce the constant-first style (the "Yoda" form) with a lint rule for exactly this reason.

© TechTribers. For personal use only. Do not redistribute.

Q602 ST-023 • Intermediate

How should you sort strings: compareTo, CASE_INSENSITIVE_ORDER or Collator?

⚡ Quick answer

`compareTo` sorts by **UTF-16 code unit**, so all uppercase letters come before lowercase and accented letters land after `z`. `String.CASE_INSENSITIVE_ORDER` ignores case but not accents. For text shown to users, use a *`Collator`* for the right `Locale`, which knows the language's alphabetical rules (accents, ligatures, language-specific ordering) and offers strength levels.

`Collator` strengths: `PRIMARY` ignores accents and case (base letters only), `SECONDARY` distinguishes accents, `TERTIARY` (default) also distinguishes case. Collators are **not thread-safe** unless cloned, and are slower than `compareTo`, so for machine keys (ids, map keys) keep plain `compareTo`, and use a `Collator` only when sorting text for humans.

Java

```
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
List<String> words = new ArrayList<>(List.of("banana", "Apple", "cherry", "apple", "\u00c9clair",
"eclair", "Zebra"));
List<String> natural = new ArrayList<>(words); Collections.sort(natural);
System.out.println(natural.stream().map(x -> esc(x)).toList());
List<String> ci = new ArrayList<>(words); ci.sort(String.CASE_INSENSITIVE_ORDER);
System.out.println(ci.stream().map(x -> esc(x)).toList());
List<String> loc = new ArrayList<>(words); loc.sort(Collator.getInstance(Locale.FRENCH));
System.out.println(loc.stream().map(x -> esc(x)).toList());
Collator primary = Collator.getInstance(Locale.ENGLISH); primary.setStrength(Collator.PRIMARY);
System.out.println(primary.compare("eclair", "\u00c9CLAIR") + " " + primary.equals("a", "A"));
```

▶ Output

```
[Apple, Zebra, apple, banana, cherry, eclair, \u00c9clair]
[Apple, apple, banana, cherry, eclair, Zebra, \u00c9clair]
[apple, Apple, banana, cherry, eclair, \u00c9clair, Zebra]
0 true
```

⚠️ Interview trap

Sorting user-visible names with `Collections.sort` gives "Zebra" before "apple" and "\u00c9clair" after "Zebra": correct for ASCII-only code points, wrong for people.

© TechTribers. For personal use only. Do not redistribute.

Q603 ST-024 • Intermediate

charAt vs codePointAt vs chars() vs codePoints(): how do you iterate characters correctly?

⚡ **Quick answer**

`charAt(i)` and `chars()` work on UTF-16 **code units** (`char` s). Characters outside the Basic Multilingual Plane (emoji, rare CJK) take **two** units (a surrogate pair), so `charAt` sees half a character. `codePointAt(i)` and `codePoints()` give whole **code points**. Advance with `Character.charCount(cp)`, or just use `codePoints()`.

Use `char` iteration only when you know the text is BMP-only (ASCII identifiers, digits, hex). For user text, iterate code points; for "user-perceived characters" (accents built from combining marks, emoji sequences) iterate **grapheme clusters** with `BreakIterator` or `\X` (see the Unicode section).

Java

```
String s = "a\uD83D\uDE00b";                                    // 'a', U+1F600 (a surrogate pair),
'b'
System.out.println(s.length() + " " + s.codePointCount(0, s.length()));
System.out.println((int) s.charAt(1) + " " + s.codePointAt(1) + " " +
Character.charCount(s.codePointAt(1)));
System.out.println(s.chars().count() + " " + s.codePoints().count());
StringBuilder sb = new StringBuilder();
for (int i = 0; i < s.length(); ) { int cp = s.codePointAt(i);
sb.append(Integer.toHexString(cp)).append(' '); i += Character.charCount(cp); }
System.out.println(sb);
System.out.println(s.codePoints().mapToObj(cp -> new
String(Character.toChars(cp))).map(String::length).toList());
```

▶ **Output**

```
4 3
55357 128512 2
4 3
61 1f600 62
[1, 2, 1]
```

💡 **Pro tip**

`s.length()` is the number of UTF-16 units. A database column of "10 characters" and a Java `length() <= 10` check can disagree on emoji.

© TechTribers. For personal use only. Do not redistribute.

Q604 ST-025 • Basic

How do you check prefixes and suffixes safely (offsets, case, URLs and extensions)?

⚡ **Quick answer**

`startsWith(prefix)` and `startsWith(prefix, offset)` check the start (or a position); `endsWith(suffix)` checks the end. All are **case-sensitive**: for case-insensitive checks use `regionMatches(true, ...)` (no allocation) or normalise with `toLowerCase(Locale.ROOT)`. The empty string is a prefix and suffix of every string. For paths and extensions, prefer `Path` / `lastIndexOf('.')` logic over ad-hoc `endsWith` chains.

The `lastIndexOf('.') == -1` case (no extension) and leading-dot files (`.gitignore`) are the usual bugs in hand-written extension code. `url.toLowerCase().startsWith("https://")` allocates a lower-cased copy of the whole string; `regionMatches(true, 0, "https://", 0, 8)` doesn't.

Java

```
String url = "HTTPS://example.com/a.PNG";
System.out.println(url.startsWith("https") + " " + url.regionMatches(true, 0, "https", 0, 5) + " " +
url.toLowerCase(Locale.ROOT).startsWith("https://"));
System.out.println(url.startsWith("example", 8) + " " + url.endsWith(".png") + " " +
url.toLowerCase(Locale.ROOT).endsWith(".png"));
System.out.println("abc".startsWith("") + " " + "abc".endsWith("") + " " + "".startsWith("") + " " +
"abc".startsWith("abcd"));
String name = "file.tar.gz";
System.out.println(name.substring(name.lastIndexOf('.') + 1) + " " + name.substring(0,
name.indexOf('.')) + " " + "noext".lastIndexOf('.') + " " +
Path.of("/tmp/report.pdf").getFileName());
System.out.println("a/b/".endsWith("/") + " " + "a/b/".replaceAll("/+$", "") + " " + "prefix-
id".substring("prefix-".length()));
```

▶ **Output**

```
false true true
true false true
true true true false
gz file -1 report.pdf
true a/b id
```

⚠️ **Interview trap**

`toLowerCase()` with no argument uses the **default locale**. In a Turkish locale `"FILE".toLowerCase().endsWith("file")` is false (`I` becomes dotless `\u0131`). Always pass `Locale.ROOT` for protocol, extension and identifier checks.

© TechTribers. For personal use only. Do not redistribute.

Q605 ST-026

• Intermediate

What exceptions do charAt and substring throw, and what do the messages say?

⚡ Quick answer

Out-of-range indexes throw `StringIndexOutOfBoundsException` (a subclass of `IndexOutOfBoundsException`). `charAt(i)` requires `0 <= i < length`; `substring(begin, end)` requires `0 <= begin <= end <= length` (note `end` may equal `length`, and `begin == end` gives the empty string). Since Java 17 the messages say exactly what was wrong.

Frequent real-world causes: `charAt(0)` on an empty string (guard with `isEmpty()`), `substring(indexOf(x) + 1)` when `indexOf` returned -1 (it then returns the whole string, silently wrong), and `substring(0, n)` with `n > length` for a "truncate" helper (use `Math.min(n, s.length())`). Any of them is a candidate for `Objects.checkIndex` or `String.indexOf` validation in new code.

Java

```
String s = "abc";
for (Runnable r : List.<Runnable>of(() -> s.charAt(3), () -> s.charAt(-1), () -> s.substring(4), ()
-> s.substring(2, 1), () -> s.substring(-1, 2), () -> s.substring(0, 4), () -> "".charAt(0))) {
    try { r.run(); } catch (StringIndexOutOfBoundsException e) { System.out.println(e.getMessage());
}
}
System.out.println("[" + s.substring(3) + "] [" + s.substring(1, 1) + "] " + s.charAt(2));
try { Objects.checkIndex(5, s.length()); } catch (IndexOutOfBoundsException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
Index 3 out of bounds for length 3
Index -1 out of bounds for length 3
Range [4, 3) out of bounds for length 3
Range [2, 1) out of bounds for length 3
Range [-1, 2) out of bounds for length 3
Range [0, 4) out of bounds for length 3
Index 0 out of bounds for length 0
[] [] c
Index 5 out of bounds for length 3
```

💡 Pro tip

A safe truncate: `s.length() <= n ? s : s.substring(0, n)`. Truncating by UTF-16 units can cut a surrogate pair in half, so for user text cut by code points (`offsetByCodePoints`).

© TechTribers. For personal use only. Do not redistribute.

Q606 ST-027 • Intermediate

What is CharSequence, and why do so many APIs accept it instead of String?

⚡ Quick answer

`CharSequence` is the read-only interface for "a sequence of `char`s": `length()`, `charAt`, `subSequence`, `toString`, plus default `isEmpty()` (Java 15) and `chars()` / `codePoints()`. `String`, `StringBuilder`, `StringBuffer` and `CharBuffer` implement it. APIs take `CharSequence` so callers can pass a builder or buffer **without converting to `String` first** (no copy).

Design guidance: **accept** `CharSequence` in utility methods that only read characters, **return** `String` (immutable, safe to share). Do not use a `CharSequence` as a map key or compare with `equals` (only `String` has value equality; `StringBuilder.equals` is identity). `CharSequence.compare` (Java 11) gives a lexicographic comparison across types.

Java

```
CharSequence[] seqs = {"text", new StringBuilder("text"), new StringBuffer("text"),
java.nio.CharBuffer.wrap("text")};
for (CharSequence cs : seqs) System.out.print(cs.getClass().getSimpleName() + ":" + cs.length() +
":" + cs.isEmpty() + " ");
System.out.println();
System.out.println(CharSequence.compare("abc", new StringBuilder("abd")) + " " +
"text".contentEquals(new StringBuilder("text")) + " " + String.join("+", new StringBuilder("a"),
"b"));
System.out.println(seqs[1].subSequence(1, 3) + " " + seqs[1].subSequence(1,
3).getClass().getSimpleName() + " " + seqs[0].subSequence(1, 3).getClass().getSimpleName());
System.out.println(Pattern.compile("e.t").matcher(seqs[1]).find() + " " + seqs[1].chars().sum() + "
" + seqs[2].toString().equals("text"));
```

▶ Output

```
String:4:false StringBuilder:4:false StringBuffer:4:false StringCharBuffer:4:false
-1 true a+b
ex String String
true 453 true
```

⚠️ Interview trap

`subSequence` on a `String` returns a *`String`* (a copy), while on a `StringBuilder` it also returns a `String` snapshot: modifying the builder later doesn't change the sequence you got.

© TechTribers. For personal use only. Do not redistribute.

Q607 ST-028

• Basic

What are the ways to count occurrences of a character or substring?

⚡ Quick answer

Count a **character** with `chars().filter(c -> c == x).count()`, a plain loop, or `length() - replace(x, "").length()`. Count a **substring** with an `indexOf` loop (decide whether matches may overlap), `split(regex, -1).length - 1` (careful with regex metacharacters), or a `Matcher.results().count()`. A plain loop or `indexOf` loop is the fastest and allocation-free.

`"banana"` contains `"ana"` twice if overlaps are allowed (index 1 and 3) and once if not. State the rule you implement. Zero-width lookahead `(?=ana)` is the regex way to count overlapping matches. Frequency tables (`groupingBy(..., counting())`) answer all single-character counts in one pass.

Java

```
String s = "banana bandana";
long a1 = s.chars().filter(c -> c == 'a').count();
int a2 = 0; for (int i = 0; i < s.length(); i++) if (s.charAt(i) == 'a') a2++;
int a3 = s.length() - s.replace("a", "").length();
System.out.println(a1 + " " + a2 + " " + a3);
int sub = 0; for (int i = s.indexOf("ana"); i >= 0; i = s.indexOf("ana", i + 1)) sub++;
// overlapping matches counted
int sub2 = 0; for (int i = s.indexOf("ana"); i >= 0; i = s.indexOf("ana", i + 3)) sub2++;
// non-overlapping
System.out.println(sub + " " + sub2 + " " + (s.split("ana", -1).length - 1) + " " +
Pattern.compile("ana").matcher(s).results().count() + " " + Pattern.compile("(?
=ana)").matcher(s).results().count());
Map<Character, Long> freq = s.chars().mapToObj(c -> (char) c).collect(Collectors.groupingBy(c -> c,
TreeMap::new, Collectors.counting()));
System.out.println(freq);
```

▶ Output

```
6 6 6
3 2 2 2 3
{ =1, a=6, b=2, d=1, n=4}
```

⚠️ Interview trap

`s.split(sep).length - 1` is wrong when the separator is at the very end (trailing empties are dropped) and when `sep` contains regex metacharacters. Use `split(sep, -1)` plus `Pattern.quote`, or the `indexOf` loop.

© TechTribers. For personal use only. Do not redistribute.

Q608 ST-029

• Intermediate

What are all the ways to construct a String?

⚡ **Quick answer**

Literals (pooled), `new String()` (empty), `new String(String)`, `new String(char[])` and `(char[], offset, count)`, `new String(byte[], Charset)` and `(byte[], offset, length, Charset)`, `new String(int[] codePoints, offset, count)`, `new String(StringBuilder/StringBuffer)`, and factories `String.valueOf(...)`, `String.copyValueOf(char[])`, `String.join`, `String.format` / `formatted`, `repeat`, `StringBuilder.toString()` and text blocks.

Choose by source type: bytes need an explicit **charset** (never the platform default); `char[]` /code point constructors copy the data; `String.valueOf(char[])` is the same as `new String(char[])`, while `String.valueOf((Object) null)` yields the text `"null"` and `String.valueOf((char[]) null)` throws NPE (overload resolution trap). Avoid `new String("literal")`, it only creates a redundant object.

Java

```
char[] chars = {'h', 'e', 'l', 'l', 'o'};
byte[] bytes = {104, 105};
int[] cps = {0x48, 0x1F600, 0x49};
System.out.println(new String(chars) + " " + new String(chars, 1, 3) + " " + new String(bytes,
StandardCharsets.UTF_8) + " " + new String(bytes, 0, 1, StandardCharsets.US_ASCII));
String fromCps = new String(cps, 0, 3);
System.out.println(fromCps.length() + " " + fromCps.codePointCount(0, fromCps.length()) + " " + new
String(new StringBuilder("sb")) + " [" + new String() + "]");
System.out.println(String.valueOf(chars) + String.valueOf(chars, 0, 2) + " " + String.valueOf(3.5) +
String.valueOf(true) + String.valueOf('c') + " " + String.copyValueOf(chars, 2, 2) + " " +
String.valueOf((Object) null));
System.out.println("ab".repeat(2) + " " + String.join("|", "x", "y") + " " + "%d-%s".formatted(7,
"z") + " " + new StringBuilder("q").append(1).toString() + " " + Character.toString(65) + " " +
Character.toString(0x1F600).length());
```

▶ **Output**

```
hello ell hi h
4 3 sb []
hellohe 3.5truec ll null
abab x|y 7-z q1 A 2
```

💡 **Pro tip**

`String.valueOf(x)` is the null-safe replacement for `x.toString()` and for `"" + x` when you want the intent to be obvious.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Methods II: Transforming, Splitting & Formatting



Boundary rules of substring, replacement pitfalls, split behaviour, the Java 11–21 additions, locale-safe case conversion, format specifiers and number conversions.

Questions 609–617 · 9 questions

Q609 ST-030 • Intermediate

What are the exact boundary rules of substring()?

⚡ Quick answer

`substring(begin)` returns from `begin` to the end; `substring(begin, end)` is **end-exclusive**. Valid ranges: `0 <= begin <= end <= length()`. `substring(length())` and `substring(k, k)` are legal and return the empty string; `substring(0)` returns the same string. Anything outside the range throws `StringIndexOutOfBoundsException`.

The length of the result is always `end - begin`, which makes it easy to reason about. The idiom `s.substring(s.indexOf(x) + 1)` is dangerous when `x` is missing: `indexOf` returns -1, `-1 + 1 = 0`, and you silently get the whole string. Check for `-1` first.

Java

```
String s = "hello";
System.out.println("[" + s.substring(5) + "] [" + s.substring(0) + "] " + (s.substring(0) == s) + "
[" + s.substring(2, 2) + "] " + s.substring(1, 4));
for (int[] r : new int[][]{{6, 6}, {-1, 2}, {3, 2}, {0, 9}}) {
    try { s.substring(r[0], r[1]); } catch (StringIndexOutOfBoundsException e) {
System.out.println(Arrays.toString(r) + " -> " + e.getMessage()); }
}
System.out.println(s.subSequence(1, 3) + " " + s.subSequence(1, 3).getClass().getSimpleName());
String path = "/usr/local/bin/java";
System.out.println(path.substring(path.lastIndexOf('/') + 1) + " " + path.substring(0,
path.lastIndexOf('/')));
System.out.println("no-slash".substring("no-slash".lastIndexOf('/') + 1));              // lastIndexOf
= -1, so +1 = 0: the whole string
```

▶ Output

```
[] [hello] true [] ell
[6, 6] -> Range [6, 6) out of bounds for length 5
[-1, 2] -> Range [-1, 2) out of bounds for length 5
[3, 2] -> Range [3, 2) out of bounds for length 5
[0, 9] -> Range [0, 9) out of bounds for length 5
el String
```


© TechTribers. For personal use only. Do not redistribute.

```
java /usr/local/bin
no-slash
```

> 💡 **Pro tip**
> Prefer `strip`, `split`, `Path` or a `Pattern` group over hand-computed substring indexes when the format has any structure.

$ Q610 ST-031 • Intermediate

What are the pitfalls of the replacement string in replaceAll() ($ and backslash)?

> ⚡ **Quick answer**
> In `replaceAll` / `replaceFirst` the **pattern is a regex** and the **replacement string is also special**: `$1` or `${name}` refers to a capture group and `\` escapes the next character. A literal `$` or `\` in the replacement must be escaped, or passed through *`Matcher.quoteReplacement`*. `replace(CharSequence, CharSequence)` is fully literal and has none of these problems.

Rule of thumb: if you are replacing **fixed text**, use `replace`. If you need a regex, escape the pattern with `Pattern.quote(text)` and the replacement with `Matcher.quoteReplacement(text)` when either comes from data. This is also a classic **injection** bug: a user-supplied replacement string such as `$9` crashes with `IndexOutOfBoundsException: No group 9`.

Java

```
String price = "cost: PRICE";
System.out.println(price.replace("PRICE", "$5"));                    // replace(): literal text, fine
try { price.replaceAll("PRICE", "$5"); }
catch (RuntimeException e) { System.out.println(e.getClass().getSimpleName() + ": " + e.getMessage()); }      // $5 is a group reference
System.out.println(price.replaceAll("PRICE", Matcher.quoteReplacement("$5")));
System.out.println("a.b.c".replaceAll(".", "-") + " " + "a.b.c".replace(".", "-") + " " + "a.b.c".replaceAll("\\.", "-"));
System.out.println("a1b22".replaceAll("(\\d+)", "<$1>") + " " + "john smith".replaceAll("(\\w+) (\\w+)", "$2, $1") + " " + "x".replaceAll("x", "\\\\"));
```

▶ **Output**

```
cost: $5
IndexOutOfBoundsException: No group 5
cost: $5
----- a-b-c a-b-c
a<1>b<22> smith, john \
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`"a.b.c".replaceAll(".", "-")` returns `-----`, because `.` matches every character. Mixing up `replace` and `replaceAll` is one of the most common Java string bugs.

Q611 ST-032 • Intermediate

Why does split() drop trailing empty strings, and how does the limit argument work?

⚡ **Quick answer**

`split(regex)` equals `split(regex, 0)`: it **removes trailing empty strings**. A **positive** limit `n` returns at most `n` parts (the last holds the rest); a **negative** limit keeps **all** trailing empties. A leading empty string is kept (except for a zero-width match at the start). The argument is a **regex**, so metacharacters (`. | + * ? ( ) [ ] \ ^ $`) must be escaped or wrapped in `Pattern.quote`.

Why trailing empties are dropped: historical Perl-compatible behaviour, convenient for `"a b c "` but surprising for CSV. For CSV always pass `-1` (or use a real CSV parser). `"".split(",")` returns an array with **one** empty string (the input itself), not an empty array. For tokenising by whitespace, trim first (`" a b ".split("\\s+")` starts with an empty string).

Java

```
System.out.println(Arrays.toString("a,b,,c,,".split(",")) + " " + "a,b,,c,,".split(",").length);
System.out.println(Arrays.toString("a,b,,c,,".split(",", -1)) + " " +
Arrays.toString("a,b,c".split(",", 2)));
System.out.println(Arrays.toString(",a,b".split(",")) + " " + Arrays.toString("".split(",")) + " " +
"".split(",").length + " " + Arrays.toString(",".split(",")) + " " + ",".split(",").length);
System.out.println(Arrays.toString("a.b.c".split(".")) + " " + Arrays.toString("a.b.c".split("\\."))
+ " " + Arrays.toString("a|b".split("|")) + " " + Arrays.toString("a|b".split("\\|")) + " " +
Arrays.toString("a|b".split(Pattern.quote("|"))));
System.out.println(Arrays.toString("abc".split("")) + " " + Arrays.toString("  a  b
".trim().split("\\s+")) + " " + Arrays.toString("a1b2c3".split("(?<=\\d)")));
System.out.println(Arrays.toString("a,b;c".splitWithDelimiters("[,;]", 0)));                //
Java 21: keep the delimiters as elements
```

▶ **Output**

```
[a, b, , c] 4
[a, b, , c, , ] [a, b,c]
[, a, b] [] 1 [] 0
[] [a, b, c] [a, |, b] [a, b] [a, b]
[a, b, c] [a, b] [a1, b2, c3]
[a, ,, b, ;, c]
```


© TechTribers. For personal use only. Do not redistribute.

`"a,b,,c,,".split(",")`	→ `[a, b, , c]`	trailing empty strings are removed
`"a,b,,c,,".split(",", -1)`	→ `[a, b, , c, , ]`	negative limit keeps them all
`"a,b,c".split(",", 2)`	→ `[a, b,c]`	limit n = at most n parts
`",a".split(",")`	→ `[, a]`	a LEADING empty string is kept
`"".split(",")`	→ `[""] (length 1)`	no delimiter found: the original string
`"a.b".split(".")`	→ `[]`	. is a regex that matches everything
`"a|b".split("|")`	→ `[a, |, b]`	| is regex alternation of empty patterns
`"a.b".split("\\.")`	→ `[a, b]`	escape it, or use Pattern.quote(".")

⚠️ **Interview trap**
`line.split(",")` on `"x,y,,"` yields 2 elements, not 4: code that indexes `parts[3]` throws `ArrayIndexOutOfBoundsException`. Use `split(",", -1)`.

🆕 **Q612** ST-033 • Intermediate

What did Java 11–21 add to String (repeat, lines, strip, indent, transform, formatted…)?

⚡ **Quick answer**
Java 11: `isBlank`, `lines`, `repeat`, `strip` / `stripLeading` / `stripTrailing`. Java 12: `indent`, `transform`. Java 13–15: `stripIndent`, `translateEscapes`, `formatted` (with text blocks), `describeConstable`. Java 21: `indexOf(ch/str, from, to)`, `splitWithDelimiters`, `StringBuilder.repeat`, and `Character.isEmoji` helpers. Each one replaces a common utility or library call.

Method	Replaces
`isBlank()`	`trim().isEmpty()` (and handles Unicode whitespace)
`lines()`	`split("\\R")` / `BufferedReader.lines()` for in-memory text
`repeat(n)`	`String.join("", Collections.nCopies(n, s))`, Apache `StringUtils.repeat`
`strip*()`	`trim()` (which only removes chars ≤ U+0020)
`indent(n)`	manual per-line padding
`transform(fn)`	method chaining with an arbitrary function
`formatted(args)`	`String.format(fmt, args)` (reads left to right, works on text blocks)
`stripIndent()`, `translateEscapes()`	the processing text blocks apply to their content

`lines()` splits on `\n`, `\r` and `\r\n` and **does not** produce a final empty line for a trailing newline (`"a\nb\n".lines().count()` is 2).

© TechTribers. For personal use only. Do not redistribute.

```java
System.out.println("ab".repeat(3) + " [" + "  hi  ".strip() + "] [" + "  hi  ".stripLeading() + "] [" + "  hi  ".stripTrailing() + "] " + "  ".isBlank());
System.out.println("a\nb\r\nc".lines().toList() + " " + "a\nb\n".lines().count());
System.out.print("x\ny".indent(2));                                  // adds 2 spaces to every line and normalises line endings
System.out.print("    x\n  y".indent(-2));                          // negative: removes up to 2 leading spaces per line
String shout = "hello".transform(s -> s + "!").transform(String::toUpperCase);
System.out.println(shout);
System.out.println("%s has %d items".formatted("cart", 3));
System.out.println("tab\\there".translateEscapes().length() + " " + "tab\\there".length());
System.out.println(String.valueOf((Object) null) + " " + "x".describeConstable().isPresent());
StringBuilder sb = new StringBuilder("ab"); sb.repeat("-", 3).repeat('x', 2);              // Java 21
System.out.println(sb);
```

▶ Output

```
ababab [hi] [hi  ] [  hi] true
[a, b, c] 2
  x
  y
  x
y
HELLO!
cart has 3 items
8 9
null true
ab---xx
```

💡 Pro tip

`String.lines()` is the cleanest way to process multi-line text in memory: `text.lines().map(String::strip).filter(Predicate.not(String::isBlank))`.

🌍 Q613 ST-034 • Advanced

Why must you pass Locale.ROOT to toLowerCase/toUpperCase? (the Turkish i problem)

⚡ Quick answer

`toLowerCase()` and `toUpperCase()` use the **default locale**. In Turkish (and Azerbaijani) `I` lower-cases to dotless `\u0131` and `i` **upper-cases to dotted** `\u0130`, so `"TITLE".toLowerCase()` is not `"title"` and identifier or protocol checks silently fail. Some mappings also change the **length** (`\u00df` becomes `SS`). For machine-readable text (identifiers, headers, keys, file extensions) use `Locale.ROOT`.

The last line shows another context-sensitive rule: Greek capital sigma lower-cases to `\u03c3` normally but to the *final sigma* `\u03c2` * at the end of a word. Correct case conversion is language-specific, which is exactly

© TechTribers. For personal use only. Do not redistribute.

why the locale argument exists.

Practical rule: **machine text** → `Locale.ROOT`, user-visible text → the user's locale**. Static analysers (Error Prone, SonarQube, IntelliJ) flag the no-argument overloads for this reason. The same applies to `String.format` (decimal separators) and `Scanner`.

Java

```
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
Locale tr = Locale.forLanguageTag("tr");
System.out.println(esc("TITLE".toLowerCase(tr)) + " " + "TITLE".toLowerCase(Locale.ROOT) + " " +
esc("title".toUpperCase(tr)) + " " + "title".toUpperCase(Locale.ROOT));
System.out.println("\u00df".toUpperCase(Locale.ROOT) + " " + "\u00df".length() + " " +
"\u00df".toUpperCase(Locale.ROOT).length());
System.out.println("i".equalsIgnoreCase("I") + " " + "I".toLowerCase(tr).equals("i") + " " +
"MAIL".toLowerCase(tr).equals("mail"));
System.out.println(esc("\u03a3".toLowerCase(Locale.ROOT)) + " " +
esc("\u039f\u0394\u039f\u03a3".toLowerCase(Locale.ROOT)));
```

▶ Output

```
t\u0131tle title T\u0130TLE TITLE
SS 1 2
true false false
\u03c3 \u03bf\u03b4\u03bf\u03c2
```

⚠️ **Interview trap**

The Turkish bug famously breaks `if (header.toLowerCase().equals("title"))` only on devices set to Turkish, so it survives tests and appears in production.

Q614 ST-035 • Intermediate

What are the most useful String.format specifiers and flags, and what goes wrong?

⚡ **Quick answer**

Specifier: `%[argument_index$][flags][width][.precision]conversion`. Conversions: `d` integer, `f` / `e` floating point, `s` string, `c` char, `b` boolean, `x` / `X` hex, `o` octal, `n` newline, `%%` percent. Flags: `-` left-justify, `0` zero-pad, `,` grouping, `+` sign, `(` negatives in parentheses, `#` alternate form. Wrong type or count throws `IllegalFormatException` at run time (e.g. `%d` with a double).

Use `%n` (not `\n`) for the platform line separator and `%,d` / `%,.2f` for human-readable numbers. **Money** should use `BigDecimal` with `NumberFormat.getCurrencyInstance(locale)`, not `double` with `%f`. `String.format` is relatively slow (it parses the format on every call): in hot paths prefer `StringBuilder` or a cached `Formatter`. A user-controlled **format string** is an injection vector: never pass untrusted text as the first argument.

© TechTribers. For personal use only. Do not redistribute.

```java
System.out.println(String.format("%5d|%-5d|%05d|%,d|%+d|%x|%X|%o|%b", 42, 42, 42, 1234567, 42, 255, 255, 8, true));
System.out.println(String.format("%.2f|%8.3f|%-8.1f|%e|%,.2f|%10s|%-10s|%.3s|%c|%%", 3.14159, 3.14159, 3.14159, 12345.678, 1234567.891, "right", "left", "truncate", 'x'));
System.out.println(String.format("%2$s %1$s %1$s", "a", "b") + " " + String.format("%s %<s", "again") + " " + String.format("%08.2f", -3.5));
try { String.format("%d", 3.5); } catch (IllegalFormatException e) {
System.out.println(e.getClass().getSimpleName()); }
try { String.format("%s %s", "only one"); } catch (IllegalFormatException e) {
System.out.println(e.getClass().getSimpleName()); }
System.out.println(String.format(Locale.GERMANY, "%,.2f", 1234567.891) + " " +
String.format(Locale.US, "%,.2f", 1234567.891) + " " + String.format("%s %S %10.4s|", null, "up", "abcdefgh"));
```

▶ Output

```
   42|42   |00042|1,234,567|+42|ff|FF|10|true
3.14|   3.142|3.1     |1.234568e+04|1,234,567.89|     right|left      |tru|x|%
b a a again again -0003.50
IllegalFormatConversionException
MissingFormatArgumentException
1.234.567,89 1,234,567.89 null UP       abcd|
```

💡 **Pro tip**

`%s` calls `toString()` (or `Formattable.formatTo`) and accepts `null` (prints `null`), so it's the safest conversion; use `%d` only for integral types.

Q615 ST-036 • Basic

How do you convert between String, char[], byte[], int[] code points, List<Character> and streams?

⚡ **Quick answer**

`toCharArray()` and `new String(char[])` (or `String.valueOf`); `getBytes(charset)` and `new String(bytes, charset)`; `codePoints().toArray()` and `new String(int[], 0, n)`; `chars().mapToObj(c -> (char) c)` for a `List<Character>` and `collect(Collectors.joining())` or a `StringBuilder` back. Always pass a **charset** when converting bytes.

Cost notes: `toCharArray` and `getBytes` copy the data (strings are immutable); `chars()` is lazy and doesn't. `UTF_16` adds a 2-byte **BOM** (the first two printed bytes, -2 and -1) while `UTF_16LE` / `BE` don't. `List<Character>` boxes every char (about 16 bytes each); for processing prefer `chars()` and primitive streams.

```java
String s = "hello";
char[] chars = s.toCharArray(); byte[] bytes = s.getBytes(StandardCharsets.UTF_8); int[] cps = s.codePoints().toArray();
```


© TechTribers. For personal use only. Do not redistribute.

```
List<Character> list = s.chars().mapToObj(c -> (char) c).toList();
String fromChars = new String(chars), fromBytes = new String(bytes, StandardCharsets.UTF_8),
fromList = list.stream().map(String::valueOf).collect(Collectors.joining());
String fromCps = new String(cps, 0, cps.length), viaBuilder = new
StringBuilder().appendCodePoint(104).append('i').toString();
System.out.println(chars.length + " " + bytes.length + " " + cps.length + " " + list + " " +
fromChars + fromBytes + fromList + fromCps + " " + viaBuilder);
System.out.println(String.valueOf(chars, 1, 3) + " " + String.valueOf(42) + " " +
Integer.toString(255, 16) + " " + Integer.toBinaryString(5) + " " + Integer.parseInt("ff", 16) + " "
+ Arrays.toString("a,b".split(",")) + " " + String.join("-", List.of("x", "y")));
System.out.println(Arrays.toString(s.getBytes(StandardCharsets.UTF_16)) + " " +
s.getBytes(StandardCharsets.UTF_16LE).length);
System.out.println(s.chars().mapToObj(Character::toString).collect(Collectors.toList()) + " " +
Stream.of("a", "b").flatMapToInt(String::chars).sum());
```

▶ Output

```
5 5 5 [h, e, l, l, o] hellohellohellohello hi
ell 42 ff 101 255 [a, b] x-y
[-2, -1, 0, 104, 0, 101, 0, 108, 0, 108, 0, 111] 10
[h, e, l, l, o] 195
```

⚠️ **Interview trap**

`new String(bytes)` and `str.getBytes()` without a charset used the **platform default** before Java 18. Always pass `StandardCharsets.UTF_8` explicitly: it documents the intent and keeps older runtimes consistent.

🔢 **Q616** ST-037 • Intermediate

How do you parse numbers from strings safely? (parseInt quirks)

⚡ **Quick answer**

`Integer.parseInt` accepts an optional sign and **Unicode digits** (Arabic-Indic `\u0664\u0662` parses as 42), but rejects whitespace, underscores, hex prefixes and exponents, and throws `NumberFormatException` on overflow. `Double.parseDouble` **trims whitespace** and accepts `NaN`, `Infinity`, exponents and a trailing `d` / `f`. Use `valueOf` for boxing, `decode` for `0x` /octal prefixes, and `parseInt(s, radix)` for other bases.

Parsing user input: **trim first** (`parseInt(s.strip())`), catch `NumberFormatException`, and bound-check the value. To avoid exceptions for control flow, test with a regex or a small helper. For decimals and money use `new BigDecimal(String)` (never `new BigDecimal(double)`) and `NumberFormat.parse(text)` for locale-specific input like `1.234,56`.

Java

```
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
for (String in : new String[]{"42", " 42", "42 ", "+42", "-0", "4_2", "0x2A", "1e3", "",
"2147483648", "\u0664\u0662"}) {
    try { System.out.println("[" + esc(in) + "] -> " + Integer.parseInt(in)); }
    catch (NumberFormatException e) { System.out.println("[" + esc(in) + "] -> " + e.getMessage());
```


© TechTribers. For personal use only. Do not redistribute.

```
}
}
System.out.println(Integer.decode("0x2A") + " " + Integer.parseInt("2A", 16) + " " +
Integer.valueOf("7") + " " + Double.parseDouble(" 1e3 ") + " " + Double.parseDouble("NaN") + " " +
Long.parseLong("-9223372036854775808"));
System.out.println(Integer.parseUnsignedInt("4294967295") + " " + Character.digit('7', 10) + " " +
Character.getNumericValue('\u0664') + " " + Boolean.parseBoolean("TRUE") + " " +
Boolean.parseBoolean("yes"));
```

▶ Output

```
[42] -> 42
[ 42] -> For input string: " 42"
[42 ] -> For input string: "42 "
[+42] -> 42
[-0] -> 0
[4_2] -> For input string: "4_2"
[0x2A] -> For input string: "0x2A"
[1e3] -> For input string: "1e3"
[] -> For input string: ""
[2147483648] -> For input string: "2147483648"
[\u0664\u0662] -> 42
42 42 7 1000.0 NaN -9223372036854775808
-1 7 4 true false
```

⚠ Interview trap

`Boolean.parseBoolean("yes")` is `false`: only the text `true` (any case) is true. Use an explicit mapping if you accept `yes`, `1`, `on`.

Q617 ST-038 • Intermediate

How are double and float converted to strings, and why does 0.1 + 0.2 print 0.30000000000000004?

⚡ Quick answer

`Double.toString` prints the **shortest decimal that uniquely identifies the double** (fixed in Java 19; earlier versions sometimes printed extra digits). Binary floating point cannot represent 0.1 or 0.2 exactly, so the sum is the double nearest to 0.30000000000000004, which differs from the double for 0.3. Compare with a tolerance, or use `BigDecimal` for exact decimal arithmetic.

Presentation: `%.2f` or `DecimalFormat` / `NumberFormat` for display; **never** use `double` to store money. The literal `2.675` is stored as 2.67499999999999982236431605997495353221893310546875 (the last output line). `new BigDecimal(2.675).setScale(2, HALF_UP)` sees that exact value and gives `2.67`, while `new BigDecimal("2.675")` and Java's own `%.2f` (which rounds the **shortest decimal representation**, `2.675`, half-up) give `2.68`; C's `printf` would print `2.67`. The lesson is the same either way: money must not be stored in a `double`. `BigDecimal.valueOf(double)` uses `Double.toString` (clean), while `new BigDecimal(double)` exposes the exact binary value.

© TechTribers. For personal use only. Do not redistribute.

Java

```
System.out.println(0.1 + 0.2);
System.out.println((0.1 + 0.2 == 0.3) + " " + (Math.abs(0.1 + 0.2 - 0.3) < 1e-9));
System.out.println(new BigDecimal(0.1));                                    // the exact
value of the double 0.1
System.out.println(new BigDecimal("0.1") + " " + BigDecimal.valueOf(0.1) + " " + new
BigDecimal("0.1").add(new BigDecimal("0.2")));
System.out.println(1.0 / 3 + " " + 100.0 + " " + 1e7 + " " + 1.0E-4 + " " + 123456789.0 + " " +
(float) 0.1 + " " + (double) 0.1f);
System.out.println(String.valueOf(1e21) + " " + Double.toString(5) + " " + Double.MIN_VALUE + " " +
(1.0 / 0) + " " + (0.0 / 0));
System.out.println(String.format("%.20f", 0.1) + " " + String.format("%.2f", 2.675) + " " + new
BigDecimal(2.675).setScale(2, RoundingMode.HALF_UP) + " " + new BigDecimal("2.675").setScale(2,
RoundingMode.HALF_UP));
System.out.println(new BigDecimal(2.675));
```

▶ Output

```
0.30000000000000004
false true
0.1000000000000000055511151231257827021181583404541015625
0.1 0.1 0.3
0.3333333333333333 100.0 1.0E7 1.0E-4 1.23456789E8 0.1 0.10000000149011612
1.0E21 5.0 4.9E-324 Infinity NaN
0.10000000000000000000 2.68 2.67 2.68
2.67499999999999982236431605997495353221893310546875
```

💡 **Pro tip**

Say it like this: "binary fractions can't represent most decimal fractions, so equality on doubles is wrong; compare with epsilon or use BigDecimal for decimal arithmetic."

© TechTribers. For personal use only. Do not redistribute.

Section 05

StringBuilder, Concatenation & Joiners



How + compiles, why += in a loop is quadratic, StringBuilder internals and API, equality traps, joiners and efficient text building.

Questions 618–625 · 8 questions

Q618 ST-039

• Advanced

How does the + operator compile? (StringBuilder in Java 8, invokedynamic in Java 9+)

⚡ **Quick answer**

`a + b + "!"` is not a runtime operator: `javac` rewrites it. Up to Java 8 it generates `new StringBuilder().append(a).append(b).append("!").toString()`. From **Java 9** it emits a single *`invokedynamic makeConcatWithConstants`*, and `StringConcatFactory` picks the best strategy at run time, computing the exact length first and filling one array. The compiler flag `-XDstringConcat=inline` restores the old shape.

Benefits of the Java 9+ scheme: no intermediate `StringBuilder` growth, correct final size up front, no extra copy, and **future JDKs can improve it without recompiling** your code (the strategy is selected when the call site is linked). It still can't fix a concatenation **inside a loop**: each iteration is a separate expression producing a separate string (next question).

Conversions: each operand is converted with `String.valueOf` semantics (`null` becomes the text `"null"`, a `char[]` argument is **not** expanded). The result of a constant expression is folded at compile time and pooled (see the pool section).

Java

```
// Source
String f(String a, int b) { return a + b + "!"; }

// javap -c (JDK 25, default): ONE invokedynamic
//    0: aload_1
//    1: iload_2
//    2: invokedynamic #7,  0   // InvokeDynamic #0:makeConcatWithConstants:
(Ljava/lang/String;I)Ljava/lang/String;
//    7: areturn
// constant pool recipe:  "\u0001\u0001!"   (\u0001 = a dynamic argument, the rest = literal text)

// javac -XDstringConcat=inline (Java 8 style)
//    0: new           StringBuilder
//    4: invokespecial StringBuilder.<init>:()V
//    8: invokevirtual StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
```


© TechTribers. For personal use only. Do not redistribute.

```
//   12: invokevirtual StringBuilder.append:(I)Ljava/lang/StringBuilder;
//   17: invokevirtual StringBuilder.append:(Ljava/lang/String;)Ljava/lang/StringBuilder;
//   20: invokevirtual StringBuilder.toString:()Ljava/lang/String;
//   23: areturn
```

return a + b + "!"; compiles differently by Java version

Java 8 (javac)

```
new StringBuilder()
.append(a).append(b).append("!")
.toString()
```

Java 9+ (javac)

```
invokedynamic makeConcatWithConstants
recipe "\u0001\u0001!" + the two arguments
```

StringConcatFactory picks the best strategy at run time: exact-size byte[], no resizing

Java 9+ computes the final length first and fills ONE array: fewer copies than the StringBuilder chain, and JDK upgrades can improve it.

💡 **Pro tip**

Single-expression concatenation (even with 10 operands) is fine and fast. Reach for `StringBuilder` only when you build the string across statements or loop iterations.

Q619 ST-040 • Intermediate

Why is s += x inside a loop slow, and what are the fixes?

⚡ **Quick answer**

Each `s += x` creates a **new string** and copies the whole old content plus the addition, so iteration k copies about k characters and n iterations copy **n(n+1)/2**: quadratic time and heavy garbage. Measured here: 20,000 appends took 65 ms with `+=` and 3 ms with `StringBuilder`; at 60,000 it was 323 ms vs 4 ms. Fix: one `StringBuilder` (or `String.join`, `Collectors.joining`, `repeat`) outside the loop.

Measured (JDK 25, single run each):

n	`s += "x"`	`StringBuilder`
20,000	65 ms	3 ms
60,000	323 ms	4 ms

The JIT can optimise some simple patterns, but not reliably, so don't count on it. For fewer than ~10 iterations the difference is irrelevant; for large or unbounded loops it decides whether the code scales. Also reduces garbage: the `+=` loop above allocates n strings averaging n/2 characters.

Java

```
int n = 10_000;
long copied = 0, len = 0;
String s = "";
for (int i = 0; i < n; i++) { copied += len + 1; len++; s += "x"; }          // each += copies the
```

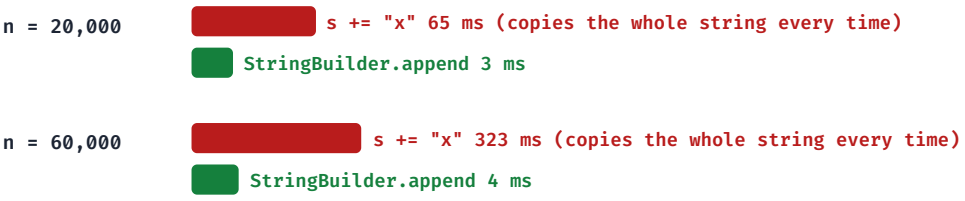

© TechTribers. For personal use only. Do not redistribute.

```
old string plus 1 char
System.out.println(copied + " " + (long) n * (n + 1) / 2 + " " + s.length());
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++) sb.append('x');                    // amortised O(1) per
append
System.out.println(sb.length() + " " + sb.toString().equals(s));
System.out.println(String.join("", Collections.nCopies(5, "ab")).equals("ab".repeat(5)) + " " +
IntStream.range(0, 5).mapToObj(Integer::toString).collect(Collectors.joining(",")));
```

▶ Output

```
50005000 50005000 10000
10000 true
true 0,1,2,3,4
```

Building a string of n characters, one append per iteration (JDK 25, measured)



s += x is O(n²): iteration k copies k characters, so n iterations copy n(n+1)/2. Bar length is logarithmic.

Tripling n multiplied the += time by 5 (65 → 323 ms) while StringBuilder barely changed.

⚠️ **Interview trap**

The same trap hides in recursion (`return f(n - 1) + x`) and in `stream.reduce("", String::concat)` . Use `Collectors.joining` for streams.

⚙️ **Q620** ST-041 • Advanced

How does StringBuilder work internally, and how does its capacity grow?

⚡ **Quick answer**

A `StringBuilder` wraps a mutable `byte[] value` (with the same Latin-1/UTF-16 `coder` as `String`), a `count` of used characters and a capacity equal to the array size. The default capacity is **16**; `new StringBuilder("hello")` starts at **length + 16**. When full it grows to *`2 × old + 2`* (16 → 34 → 70 → 142 → ...) and copies. `toString()` copies the used part into a new `String` .

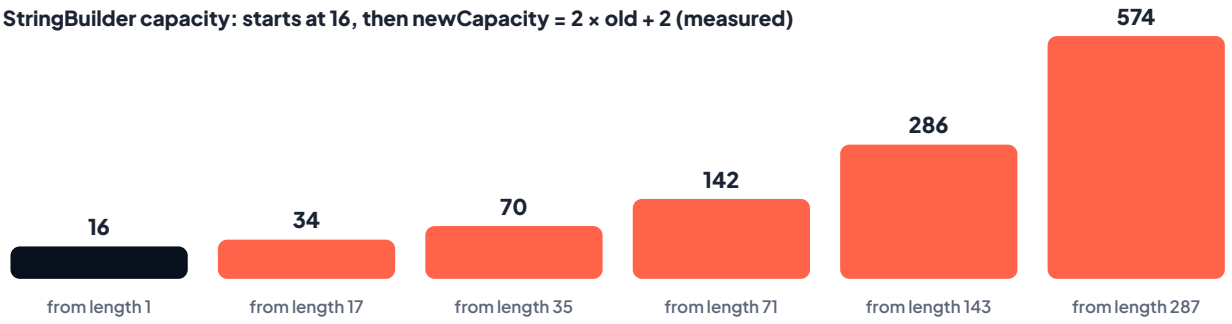
Growth is geometric, so n appends cost O(n) amortised, but each growth copies the whole array and leaves up to ~50% unused. If you know the final size, `new StringBuilder(n)` avoids every resize. `setLength(k)` shortens (or pads with `\0`) without shrinking capacity; `trimToSize()` releases the slack. `StringBuffer` has the same internals plus `synchronized` methods.

© TechTribers. For personal use only. Do not redistribute.

```java
StringBuilder sb = new StringBuilder();
System.out.println(sb.capacity() + " " + new StringBuilder("hello").capacity() + " " + new
StringBuilder(100).capacity());
int last = sb.capacity();
StringBuilder growth = new StringBuilder();
for (int i = 1; i <= 300; i++) { sb.append('x'); if (sb.capacity() != last) { last = sb.capacity();
growth.append(" ").append(i).append("->").append(last); } }
System.out.println(growth.toString().trim());
sb.setLength(10);
System.out.println(sb.length() + " " + sb.capacity());
sb.trimToSize();
System.out.println(sb.capacity());
StringBuilder big = new StringBuilder(); big.ensureCapacity(1000);
System.out.println(big.capacity());
```

▶ Output

```
16 21 100
17->34 35->70 71->142 143->286 287->574
10 574
10
1000
```



Each growth allocates a new array and copies. new StringBuilder(n) or ensureCapacity(n) avoids all of them.

💡 **Pro tip**

Build strings in the exact size you expect: `new StringBuilder(items.size() * 16)` is a decent estimate for typical rows.

© TechTribers. For personal use only. Do not redistribute.

Q621 ST-042

• Basic

What can StringBuilder do? (insert, delete, replace, reverse, setLength...)

⚡ **Quick answer**

`append` (any type, including `char[]`, `CharSequence` ranges and code points), `insert(offset, x)`, `delete(start, end)`, `deleteCharAt(i)`, `replace(start, end, str)`, `reverse()` (surrogate-pair aware), `setCharAt`, `setLength`, `indexOf` / `lastIndexOf`, `charAt`, `substring`, `compareTo` (Java 11), `isEmpty` (Java 15) and `repeat` (Java 21). Most mutators return `this`, so calls chain; `setCharAt` and `setLength` return `void`.

Cost: `insert` and `delete` at the front are O(n) (they shift the array), so building text front-to-back with `append` and reversing once is cheaper than repeated `insert(0, ...)`. `reverse()` treats a surrogate pair as one unit (the emoji's code point is preserved), but it still reverses **code points**, not grapheme clusters (combining accents attach to the wrong letter).

Java

```
StringBuilder sb = new StringBuilder("hello world");
sb.insert(0, ">> ").append('!').append(42).append(true).append(1.5).append((Object) null).append(new
char[]{'x', 'y'});
System.out.println(sb);
sb.setLength(sb.indexOf("!"));                                    // truncate at the first '!'
System.out.println(sb);
sb.delete(0, 3).deleteCharAt(sb.length() - 1).replace(0, 5, "HELLO");
sb.setCharAt(0, 'J');                                             // returns void, so it ends a
chain
System.out.println(sb + " " + sb.length() + " " + sb.charAt(1) + " " + sb.indexOf("o") + " " +
sb.lastIndexOf("o"));
System.out.println(new StringBuilder("abc").reverse() + " " + new
StringBuilder("ab\uD83D\uDE00").reverse().codePointAt(0) + " " + new
StringBuilder("racecar").reverse().toString().equals("racecar"));
System.out.println(new StringBuilder("ab").compareTo(new StringBuilder("ac")) + " " + new
StringBuilder("x").repeat("ab", 3) + " " + new StringBuilder("abc").substring(1) + " " + new
StringBuilder().isEmpty());
```

▶ **Output**

```
>> hello world!42true1.5nullxy
>> hello world
JELLO worl 10 E 7 7
cba 128512 true
-1 xababab bc true
```

⚠️ **Interview trap**

`sb.append(1).append('a')` appends the text `1a`, but `sb.append('a' + 1)` appends the **number** 98: `'a' + 1` is an `int`. Mixing char arithmetic into `append` is a classic bug.

© TechTribers. For personal use only. Do not redistribute.

⚠️ Q622 ST-043

• Intermediate

Why doesn't StringBuilder.equals() compare content, and how do you compare builders?

⚡ **Quick answer**

`StringBuilder` and `StringBuffer` do **not** override `equals` or `hashCode`; both are inherited from `Object` (identity). So `new StringBuilder("x").equals(new StringBuilder("x"))` is `false`, and a builder used as a `HashMap` key is found only by the same instance. Compare content with `toString().equals(...)`, `contentEquals`, `compareTo` (Java 11, returns 0 for equal content) or `CharSequence.compare`.

Why: a mutable object with content-based `equals`/`hashCode` would silently break any `HashMap` or `HashSet` that held it, so the JDK designers kept identity semantics. If you need a text key, call `toString()` when you put it in (a `String` key is immutable). `toString()` copies the contents each call, so cache it if you need it repeatedly.

Java

```
StringBuilder a = new StringBuilder("x"), b = new StringBuilder("x");
System.out.println(a.equals(b) + " " + a.toString().equals(b.toString()) + " " + a.compareTo(b) + "
" + "x".contentEquals(a) + " " + a.isEmpty() + " " + (a.hashCode() == b.hashCode()));
Set<StringBuilder> set = new HashSet<>(List.of(a));
System.out.println(set.contains(b) + " " + set.contains(a));
StringBuilder key = new StringBuilder("k");
Map<StringBuilder, Integer> map = new HashMap<>();
map.put(key, 1);
key.append("2");                                                    // identity hash: mutation does
not move the key
System.out.println(map.get(key) + " " + map.size());
System.out.println((a.toString() == a.toString()) + " " + CharSequence.compare(a, b));
```

▶ **Output**

```
false true 0 true false false
false true
1 1
false 0
```

⚠️ **Interview trap**

`if (sb == "abc")` and `if (sb.equals("abc"))` are both always false, and `"abc".equals(sb)` is false too (different class). Use `"abc".contentEquals(sb)`.

© TechTribers. For personal use only. Do not redistribute.

Q623 ST-044

• Basic

How do StringJoiner, String.join and Collectors.joining differ?

⚡ **Quick answer**

`String.join(delimiter, elements...)` and `String.join(delimiter, Iterable)` is the simplest. `Collectors.joining(delimiter, prefix, suffix)` works on a stream. `StringJoiner(delimiter, prefix, suffix)` lets you add pieces incrementally and set an **empty value** for when nothing was added. All three share the same engine and put the delimiter only **between** elements.

Behaviour to know: elements must be `CharSequence` (use `map(String::valueOf)` for numbers); a `null` element prints as `null`; the empty value is used only when **no** element was added (even empty strings count as elements, as the last line shows); and `merge` appends the other joiner's content as one element. For CSV-like output you also need quoting/escaping, which no joiner does.

Java

```
StringJoiner sj = new StringJoiner(", ", "[", "]");
sj.setEmptyValue("EMPTY");
System.out.println(sj + " " + sj.length());
sj.add("a").add("b");
System.out.println(sj + " " + sj.length());
System.out.println(String.join("/", "a", "b", "c") + " " + String.join("-", List.of("x", "y")) + "
[" + String.join(",", new ArrayList<String>()) + "] " + String.join("-", "a", null, "b"));
System.out.println(Stream.of("a", "b").collect(Collectors.joining(", ", "{", "}")) + " " +
Stream.of(1, 2, 3).map(String::valueOf).collect(Collectors.joining("+")) + " " + Stream.
<String>empty().collect(Collectors.joining(",", "<", ">")));
StringJoiner merged = new StringJoiner("-").add("p").add("q");
StringJoiner other = new StringJoiner("+").add("r").add("s");
System.out.println(merged.merge(other) + " " + new
StringJoiner(",").add("").add("").toString().length());
```

▶ **Output**

```
EMPTY 5
[a, b] 6
a/b/c x-y [] a-null-b
{a, b} 1+2+3 <>
p-q-r+s 1
```

💡 **Pro tip**

The pattern "append a comma after each, then delete the last one" is a smell. Use a joiner, or `StringBuilder` with a `sep` variable (`sb.append(sep).append(x); sep = ", ";`).

© TechTribers. For personal use only. Do not redistribute.

Q624 ST-045 • Intermediate

What are the efficient ways to build large or multi-line text?

⚡ Quick answer

For one expression, `+` (or `formatted`). For loops, a **pre-sized** `StringBuilder`. **For tabular or formatted output,** `Formatter` / `PrintWriter` **over a** `StringWriter`. **For joined data,** `String.join` / `Collectors.joining`. **For very large text,** stream it** to a `Writer` / `OutputStream` instead of building one huge `String`. Text blocks are best for fixed multi-line literals.

Why streaming matters: building a 500 MB report as one `String` needs the characters twice (builder array + final copy) and a contiguous 1–2 GB heap region. Writing rows straight to a `BufferedWriter` / `OutputStream` keeps memory flat. `Appendable` lets one method serve builders, writers and consoles. Avoid `String.format` per row in hot loops: reuse a `Formatter` or build with `append`.

Java

```
StringWriter sw = new StringWriter();
try (PrintWriter pw = new PrintWriter(sw)) {
    pw.printf("%-6s|%5s%n", "name", "qty");
    for (String[] row : new String[][]{{"pen", "5"}, {"ink", "12"}}) pw.printf("%-6s|%5s%n", row[0],
row[1]);
}
System.out.print(sw);
Appendable out = new StringBuilder();
out.append("a").append('b').append("xyz", 1, 2);                    // Appendable: StringBuilder,
Writer, PrintStream...
System.out.println(out);
String report = Stream.of("x", "y", "z").map(String::toUpperCase).collect(Collectors.joining("\n",
"BEGIN\n", "\nEND"));
System.out.println(report.lines().count() + " " + report.replace("\n", "/"));
StringBuilder sb = new StringBuilder(64);
sb.append("id=").append(7).append(", name=").append("asha");
System.out.println(sb);
String block = """
    {
      "k": "v"
    }""";
System.out.println(block.lines().count() + " " + block.length());
```

▶ Output

```
name   |  qty
pen    |    5
ink    |   12
aby
5 BEGIN/X/Y/Z/END
id=7, name=asha
3 14
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Estimate the size to pre-allocate: `new StringBuilder(rows * avgRowLength)`. Over-allocating costs a little memory; under-allocating costs several copies.

🧵 **Q625** ST-046 • Intermediate

Why doesn't StringBuffer's synchronisation make shared text building safe?

> ⚡ **Quick answer**
> `StringBuilder` is **not** thread-safe: concurrent `append`s can lose characters, corrupt the length or throw `ArrayIndexOutOfBoundsException`, because the count update isn't atomic. `StringBuffer` synchronises every method, so each call is atomic, but **sequences** of calls are still not (an `append` followed by `length()` can interleave with another thread). In practice builders are confined to one thread (a method's local variable), which needs no safety at all.

The `StringBuffer` total is always exact. `StringBuilder` may also come out exact on a lightly contended run (in my single-core sandbox it did every time), which is the danger: the bug appears on multi-core production hardware under real parallelism. I could not demonstrate a loss here, so treat the documentation as the authority: not thread-safe.

Guidance: use `StringBuilder` for local building, a thread-safe structure (`ConcurrentLinkedQueue<String>`, a lock around the builder, or per-thread builders merged at the end) for shared output, and `StringBuffer` only for legacy code. `StringBuffer`'s per-call locking is rarely what you want: it doesn't protect compound operations.

Java

```
StringBuffer shared = new StringBuffer();
StringBuilder unsafe = new StringBuilder();
int threads = 4, per = 100_000;
ExecutorService pool = Executors.newFixedThreadPool(threads);
CyclicBarrier gate = new CyclicBarrier(threads);
List<Callable<Void>> jobs = new ArrayList<>();
for (int t = 0; t < threads; t++) jobs.add(() -> { gate.await(); for (int i = 0; i < per; i++) {
shared.append('x'); try { unsafe.append('x'); } catch (RuntimeException e) { } } return null; });
pool.invokeAll(jobs); pool.shutdown();
System.out.println(shared.length() + " " + (unsafe.length() <= threads * per));       //
StringBuffer: exactly 400000; StringBuilder: never more, possibly fewer
```

▶ **Output**

```
400000 true
```

> ⚠️ **Interview trap**
> Sharing a `StringBuilder` through a `static` field or a singleton (a "reusable" builder) is the typical thread-safety bug in web applications. Keep builders local.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Unicode, Charsets & Locale

Code units vs code points vs graphemes, encoding and decoding, normalisation, locale and charset dependence, and safe handling of international text.

Questions 626–632 · 7 questions



Q626 ST-047

• Advanced

What is the difference between char, code point and grapheme cluster, and why does length() surprise people?

⚡ Quick answer

A Java `char` is a 16-bit **UTF-16 code unit**. A Unicode **code point** is a character number; code points above U+FFFF (emoji, rare CJK) need **two** `char` s (a surrogate pair), so `length()` counts units, not characters. What a user perceives as one character is a **grapheme cluster**, which can be several code points (a letter plus combining accent, or an emoji sequence joined by ZWJ). Use `codePointCount` for code points and `BreakIterator` (or regex `\X`) for graphemes.

Where it bites: input limits ("max 10 characters"), truncation (cutting a pair or a ZWJ sequence in half), reversal, cursor movement, and database column sizes. Choose the unit on purpose: **storage** limits → UTF-8/UTF-16 bytes or units, **text processing** → code points, **what the user sees** → graphemes. `BreakIterator` supports extended grapheme clusters (emoji sequences) from JDK 20.

Java

```
String s = "A\u00e9\uD83D\uDE00\uD83D\uDC68\u200D\uD83D\uDC69\u200D\uD83D\uDC67";      // A, e-acute, grinning face, family emoji (man ZWJ woman ZWJ girl)
System.out.println(s.length() + " " + s.codePointCount(0, s.length()));
BreakIterator it = BreakIterator.getCharacterInstance(Locale.ROOT); it.setText(s);
int graphemes = 0; while (it.next() != BreakIterator.DONE) graphemes++;
System.out.println(graphemes);
System.out.println("e\u0301".length() + " " + "e\u0301".codePointCount(0, 2) + " " +
Pattern.compile("\\X").matcher("e\u0301x").results().count());
System.out.println(Character.isSurrogate(s.charAt(2)) + " " + Character.isHighSurrogate(s.charAt(2))
+ " " + Character.isLowSurrogate(s.charAt(3)) + " " +
Character.isSupplementaryCodePoint(s.codePointAt(2)));
```

▶ Output

```
12 8
4
2 2 2
true true true true
```


© TechTribers. For personal use only. Do not redistribute.

One visible symbol, three different 'lengths'

family emoji = man + ZWJ + woman + ZWJ + girl (one symbol on screen)

length()	codePointCount()	grapheme clusters
8	5	1
UTF-16 code units (each emoji = 2 units)	Unicode code points (3 emoji + 2 joiners)	what a user sees (BreakIterator / regex \X)

Use length() only for storage limits. For 'how many characters does the user see', count grapheme clusters.

⚠️ **Interview trap**

`s.substring(0, 5)` can end between the two halves of a surrogate pair and produce an invalid string (a lone high surrogate). Cut with `offsetByCodePoints` or `BreakIterator`.

🔐 **Q627** ST-048 • Advanced

How do you encode and decode strings safely (charsets, mojibake, malformed input)?

⚡ **Quick answer**

`getBytes(charset)` encodes and `new String(bytes, charset)` decodes. Always pass the charset explicitly. Characters the charset can't represent are replaced by `?` when encoding, and malformed bytes become the replacement character U+FFFD when decoding with the lenient constructors. For strict behaviour use `CharsetEncoder` / `CharsetDecoder` with `CodingErrorAction.REPORT`. Decoding with the wrong charset produces **mojibake**.

Rules: use **UTF-8** for files, network and APIs; always specify it in `Files.readString(path, UTF_8)`, `InputStreamReader`, `getBytes`, `URLEncoder` and HTTP headers; and validate untrusted bytes with a strict decoder when corrupt data must be rejected. Since Java 18 UTF-8 is the default charset for the *file APIs and `Charset.defaultCharset()`* (JEP 400), but `System.out` still uses the console/locale encoding (`stdout.encoding`).

Sizes: UTF-8 uses 1–4 bytes per code point (ASCII 1, `é` 2, `€` 3, emoji 4), UTF-16 uses 2 or 4, ISO-8859-1 uses 1 (and can't encode `€`).

Java

```
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
String s = "caf\u00e9 \u20ac";
for (Charset cs : List.of(StandardCharsets.UTF_8, StandardCharsets.ISO_8859_1,
StandardCharsets.UTF_16BE, StandardCharsets.US_ASCII)) {
    byte[] b = s.getBytes(cs);
    System.out.println(cs.name() + ": " + b.length + " bytes, round-trip " + new String(b,
cs).equals(s));
```


© TechTribers. For personal use only. Do not redistribute.

```
}
byte[] utf8 = "\u00e9".getBytes(StandardCharsets.UTF_8);
System.out.println(utf8.length + " " + esc(new String(utf8, StandardCharsets.ISO_8859_1)));
// mojibake: UTF-8 bytes read as Latin-1
CharsetDecoder strict =
StandardCharsets.UTF_8.newDecoder().onMalformedInput(CodingErrorAction.REPORT).onUnmappableCharacter
(CodingErrorAction.REPORT);
try { strict.decode(ByteBuffer.wrap(new byte[]{(byte) 0xC3, (byte) 0x28})); } catch
(CharacterCodingException e) { System.out.println(e.getClass().getSimpleName()); }
System.out.println(esc(new String(new byte[]{(byte) 0xC3, (byte) 0x28}, StandardCharsets.UTF_8)));
// lenient constructor: replacement character U+FFFD
```

▶ Output

```
UTF-8: 9 bytes, round-trip true
ISO-8859-1: 6 bytes, round-trip false
UTF-16BE: 12 bytes, round-trip true
US-ASCII: 6 bytes, round-trip false
2 \u00c3\u00a9
MalformedInputException
\ufffd(
```

⚠️ **Interview trap**

`new String(bytes)` with a wrong default charset is the source of "?" and "Ã©" in output. If you see `Ã©` for `é`, UTF-8 bytes were decoded as Latin-1.

✨ **Q628** ST-049 • Advanced

What is Unicode normalisation, and how do you do accent-insensitive comparison?

⚡ **Quick answer**

The same visible character can have several encodings: `\u00e9` (one precomposed code point, **NFC**) or `e` + `\u0301` (base + combining accent, **NFD**). They look identical but `equals` is false and the lengths differ. `java.text.Normalizer.normalize(text, Form)` converts to a canonical form (`NFC`, `NFD`, and the compatibility forms `NFKC` / `NFKD`). Normalise both sides before comparing, hashing or storing.

Choosing a form: **NFC** is the recommended storage/exchange form (and what most keyboards produce); **NFD** is the starting point for removing accents; **NFKC/NFKD** also fold "compatibility" characters (ligatures such as `\uFB01` into `fi`, full-width digits, superscripts) and are used for search keys and identifier comparison, but lose information. File names from macOS are often NFD, a common source of "identical but not equal" names.

Java

```
String nfc = "\u00e9", nfd = "e\u0301";
System.out.println(nfc.length() + " " + nfd.length() + " " + nfc.equals(nfd) + " " +
Normalizer.normalize(nfd, Normalizer.Form.NFC).equals(nfc) + " " + Normalizer.isNormalized(nfd,
Normalizer.Form.NFD));
String text = "Cr\u00e8me Br\u00fbl\u00e9e \u00c0 la carte";
String plain = Normalizer.normalize(text, Normalizer.Form.NFD).replaceAll("\\p{M}+", "");        //
```

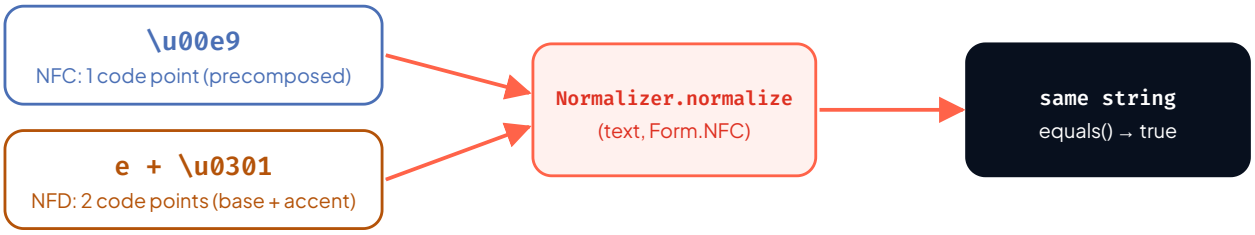

© TechTribers. For personal use only. Do not redistribute.

```
decompose, then drop the combining marks
System.out.println(plain);
System.out.println(plain.toLowerCase(Locale.ROOT).contains("brulee") + " " +
text.toLowerCase(Locale.ROOT).contains("brulee"));
Collator c = Collator.getInstance(Locale.ROOT); c.setStrength(Collator.PRIMARY);
System.out.println(c.equals("resume", "r\u00e9sum\u00e9") + " " + "\uFB01".length() + " " +
Normalizer.normalize("\uFB01", Normalizer.Form.NFKC));
```

▶ Output

```
1 2 false true true
Creme Brulee A la carte
true false
true 1 fi
```

Same visible letter, different code points: equals() is false until you normalise



Strip accents: normalize to NFD, then remove \p{M} (marks). Compare user text after normalising both sides.

💡 **Pro tip**

For "search ignoring accents and case" the robust recipe is NFD, remove `\p{M}`, lower-case with `Locale.ROOT`, or use a `Collator` with `PRIMARY` strength.

🗺️ **Q629** ST-050 • Intermediate

Which String methods depend on the default Locale or charset?

⚡ **Quick answer**

Locale-dependent: `toLowerCase()`, `toUpperCase()`, `String.format(String, ...)`, `formatted`, `printf`, `Scanner` / `NumberFormat` default instances, `MessageFormat`. Charset-dependent: `getBytes()` and `new String(byte[])` without a charset, `FileReader` / `FileWriter` (UTF-8 default only from Java 18), `PrintStream` console encoding. Always pass `Locale.ROOT` and an explicit charset for machine-readable data.

Note what does **not** change with the locale: `String.valueOf(double)`, `Double.toString` and `Double.parseDouble` always use `.` as the decimal separator, which is why they are the safe choice for data interchange, while `String.format("%.1f")` prints `1,5` under a German default locale.

A review checklist for internationalisation bugs: grep for `toLowerCase()`, `toUpperCase()`, `String.format(` and `getBytes()` with no locale/charset, `new FileReader(`, `new String(bytes)` and `Scanner(` without a locale.

© TechTribers. For personal use only. Do not redistribute.

```java
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
Locale original = Locale.getDefault();
try {
    Locale.setDefault(Locale.forLanguageTag("tr"));
    System.out.println(esc("TITLE".toLowerCase()) + " " + "TITLE".toLowerCase(Locale.ROOT));
    Locale.setDefault(Locale.GERMANY);
    System.out.println(String.format("%.1f", 1.5) + " " + String.format(Locale.ROOT, "%.1f", 1.5) +
" " + String.valueOf(1.5) + " " + Double.parseDouble("1.5"));
} finally { Locale.setDefault(original); }
System.out.println(Charset.defaultCharset() + " " + "\u00e9".getBytes().length + " " + new
String(new byte[]{(byte) 0xC3, (byte) 0xA9}).length());
```

▶ Output

```
t\u0131tle title
1,5 1.5 1.5 1.5
UTF-8 2 1
```

⚠️ Interview trap

Setting `Locale.setDefault` in a library is dangerous (it is JVM-wide and not thread-safe); tests that do it must restore the old value, as the `finally` block does.

🔡 **Q630** ST-051 • Advanced

How does case-insensitive comparison behave across Unicode (equalsIgnoreCase, toLowerCase, ß, ı, Σ)?

⚡ Quick answer

`equalsIgnoreCase` compares **character by character**: two characters match if they are equal, or their upper-case forms are equal, or their lower-case forms are equal. So it never expands `\u00df` to `SS`, and `"stra\u00dfe".equalsIgnoreCase("STRASSE")` is `false` even though `toUpperCase` of the first gives exactly `STRASSE`. Case rules are language-specific, so for serious text matching use `Collator`, `toLowerCase(Locale.ROOT)` after normalisation, or `UNICODE_CASE` regex flags.

Take-aways: `ß` and `SS`, dotted/dotless `i`, the Kelvin sign `\u212A` (which lower-cases to `k`) and the two Greek sigmas show that "same letter, different case" has no single definition. Without `UNICODE_CASE`, regex `CASE_INSENSITIVE` only folds **ASCII**. For keys and identifiers, restrict the alphabet and compare `toLowerCase(Locale.ROOT)`; for human-language search use a library such as ICU4J.

© TechTribers. For personal use only. Do not redistribute.

```java
System.out.println("stra\u00dfe".equalsIgnoreCase("STRASSE") + " " +
"stra\u00dfe".toUpperCase(Locale.ROOT).equals("STRASSE") + " " +
"STRASSE".equalsIgnoreCase("stra\u00dfe"));
System.out.println("\u0130".equalsIgnoreCase("i") + " " + "\u0131".equalsIgnoreCase("I") + " " +
"I".equalsIgnoreCase("i") + " " + "\u212A".equalsIgnoreCase("k"));
System.out.println(String.CASE_INSENSITIVE_ORDER.compare("a", "B") + " " +
"a".compareToIgnoreCase("B") + " " + "\u03c3".equalsIgnoreCase("\u03c2"));
System.out.println(Pattern.compile("stra\u00dfe", Pattern.CASE_INSENSITIVE |
Pattern.UNICODE_CASE).matcher("STRA\u00dfE").matches() + " " + Pattern.compile("k",
Pattern.CASE_INSENSITIVE).matcher("\u212A").matches() + " " + Pattern.compile("k",
Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE).matcher("\u212A").matches());
```

▶ Output

```
false true false
true true true true
-1 -1 true
true false true
```

⚠️ Interview trap

Security: comparing usernames or domains with only ASCII folding allows look-alike registrations (the Kelvin sign, Turkish i). Normalise and restrict characters first.

🔬 **Q631** ST-052 • Intermediate

What do the Character methods and Unicode regex classes check, and why isn't isLetter the same as [A-Za-z]?

⚡ Quick answer

`Character.isLetter`, `isDigit`, `isLetterOrDigit`, `isWhitespace`, `isUpperCase` etc. are **Unicode-aware**: `isLetter('\u00e9')` and `isLetter('\u4e2d')` are true, `isDigit('\u0664')` (Arabic-Indic four) is true. `[A-Za-z]` and `[0-9]` are **ASCII-only**; `\\d` is ASCII unless you set `UNICODE_CHARACTER_CLASS`. For "letter in any language" use `\\p{L}`; for a script use `\\p{IsHan}`; check ASCII explicitly with `c < 128`.

Choosing by intent: **validate an ASCII identifier or code** → explicit ranges or `c < 128` checks; **"is this a word character in the user's language"** → `isLetter` / `\\p{L}`; **parse digits** → `Character.digit(c, radix)` (which also accepts Unicode digits, so check ASCII first if you need to). `Character.isWhitespace` excludes the non-breaking space `\u00a0`, while `isSpaceChar` includes it, and `String.strip` / `isBlank` use `isWhitespace`.

© TechTribers. For personal use only. Do not redistribute.

```java
String s = "Caf\u00e9 42 \u4e2d\u6587 \u0664\u0662 _-";
long letters = s.chars().filter(Character::isLetter).count(), digits =
s.chars().filter(Character::isDigit).count(), ascii = s.chars().filter(c -> c < 128).count(), ws =
s.chars().filter(Character::isWhitespace).count();
System.out.println(letters + " " + digits + " " + ascii + " " + ws);
System.out.println(s.matches("[A-Za-z ]+") + " " + "Caf\u00e9".matches("\\p{L}+") + " " +
"\u4e2d\u6587".matches("\\p{IsHan}+") + " " + "42".matches("[0-9]+") + " " +
"\u0664\u0662".matches("[0-9]+") + " " + "\u0664\u0662".matches("\\d+") + " " +
Pattern.compile("\\d+", Pattern.UNICODE_CHARACTER_CLASS).matcher("\u0664\u0662").matches());
System.out.println(Character.getNumericValue('Z') + " " + Character.digit('Z', 36) + " " +
Character.isJavaIdentifierStart('$') + " " + Character.toChars(0x1F600).length + " " +
Character.getName('A') + " " + Character.isEmoji(0x1F600));
```

▶ Output

```
6 4 11 4
false true true true false false true
35 35 true 2 LATIN CAPITAL LETTER A true
```

⚠️ **Interview trap**

Validating a numeric field with `Character.isDigit` accepts Arabic-Indic, Devanagari and full-width digits, which then fail or parse unexpectedly downstream. Use `c >= '0' && c <= '9'` for ASCII-only input.

Q632 ST-053 • Advanced

How do you reverse or truncate text safely when it contains surrogate pairs and combining marks?

⚡ **Quick answer**

Reverse by **code points** (`StringBuilder.reverse()` already treats surrogate pairs as units, a manual `char[]` swap does not). Truncate at a **code point or grapheme boundary** with `offsetByCodePoints` or `BreakIterator` , never at an arbitrary `char` index. Combining marks and emoji sequences are several code points, so even a code-point-aware reverse puts accents on the wrong letter.

The last line shows the limit of `reverse()` : reversing `e\u0301e\u0301` yields `\u0301e\u0301e` , so the accent lands on the **wrong** (previous) letter. A grapheme-aware reverse iterates `BreakIterator` boundaries and builds the result from clusters. For almost all application code the right answer is not to reverse or slice user text by hand at all.

```java
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray())
b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
String s = "ab\uD83D\uDE00";
System.out.println(new StringBuilder(s).reverse().codePointAt(0) == 0x1F600);              //
StringBuilder.reverse keeps the pair together
char[] c = s.toCharArray();
```


© TechTribers. For personal use only. Do not redistribute.

```
for (int i = 0, j = c.length - 1; i < j; i++, j--) { char t = c[i]; c[i] = c[j]; c[j] = t; }
System.out.println(Character.isLowSurrogate(c[0]) + " " + (new String(c).codePointAt(0) ==
0x1F600));       // naive char swap breaks the pair
String text = "caf\u00e9 \uD83D\uDE00!";
String cut = text.substring(0, 6);
System.out.println(cut.length() + " " + Character.isHighSurrogate(cut.charAt(5)));
// a lone high surrogate: invalid text
int end = text.offsetByCodePoints(0, 6);
System.out.println(esc(text.substring(0, end)) + " " + end);
String combined = "e\u0301e\u0301";
BreakIterator bi = BreakIterator.getCharacterInstance(); bi.setText(combined);
System.out.println(combined.length() + " " + bi.next() + " " + esc(new
StringBuilder(combined).reverse().toString()));
```

▶ Output

```
true
true false
6 true
caf\u00e9 \ud83d\ude00 7
4 2 \u0301e\u0301e
```

💡 **Pro tip**

If a truncated string will be shown to users, truncate at a grapheme boundary and add an ellipsis. If it goes to a database column, truncate by the column's unit (bytes or code points) at a code point boundary.

Section 07

Regular Expressions in Java

The Pattern/Matcher API, groups and backreferences, quantifier types, lookarounds, flags, backtracking risk, common recipes, and what regex really costs.

Questions 633–640 · 8 questions



🧪 **Q633** ST-054 • Intermediate

What are the main parts of the Pattern and Matcher API (matches, find, lookingAt, groups, replace, results)?

⚡ **Quick answer**

`Pattern.compile(regex)` builds a reusable, thread-safe pattern; `pattern.matcher(text)` creates a **stateful, not thread-safe** `Matcher`. `matches()` requires the **whole** text to match, `lookingAt()` matches a **prefix**, `find()` scans for the next match anywhere. `group(n)`, `start()`, `end()` describe the match; `replaceAll(Function)` and `results()` (Java 9) give a functional style.

© TechTribers. For personal use only. Do not redistribute.

Calling `group()` before a successful `find()` / `matches()` throws `IllegalStateException: No match found`. Reuse: compile once (`static final Pattern`), but create a **new** `Matcher` **per use** (or `reset`) when threads are involved. `appendReplacement` / `appendTail` and `replaceAll(Function)` let you compute replacements from the match.

Java

```
Pattern p = Pattern.compile("(\\w+)@(\\w+)\\.com");
Matcher m = p.matcher("mail asha@tt.com or ravi@dev.com now");
while (m.find()) System.out.println(m.group() + " user=" + m.group(1) + " host=" + m.group(2) + " ["
+ m.start() + "," + m.end() + ")");
System.out.println(p.matcher("asha@tt.com").matches() + " " + p.matcher("x asha@tt.com").matches() +
" " + p.matcher("asha@tt.com!").lookingAt() + " " + p.matcher("x asha@tt.com").find());
System.out.println(p.matcher("a@b.com c@d.com").replaceAll(r -> r.group(2).toUpperCase()) + " " +
p.matcher("a@b.com c@d.com").results().map(MatchResult::group).toList());
System.out.println(Pattern.matches("\\d{3}", "123") + " " + "a1b2".replaceAll("[0-9]", "#") + " " +
p.pattern() + " " + p.matcher("zz").groupCount());
System.out.println(Pattern.compile(",").splitAsStream("a,b,c").toList() + " " +
Pattern.compile("\\d").asPredicate().test("abc1") + " " +
Pattern.compile("\\d").asMatchPredicate().test("abc1"));
```

▶ Output

```
asha@tt.com user=asha host=tt [5,16)
ravi@dev.com user=ravi host=dev [20,32)
true false true true
B D [a@b.com, c@d.com]
true a#b# (\w+)@(\w+)\.com 2
[a, b, c] true false
```

⚠️ **Interview trap**

`String.matches(regex)` and `Pattern.matches` always require a **full** match, so `"abc123".matches("\\d+")` is false. People expecting "contains" semantics should use `find()` .

Q634 ST-055 • Intermediate

How do capturing, non-capturing and named groups and backreferences work?

⚡ **Quick answer**

`( … )` captures and numbers groups from the left by opening parenthesis (group 0 is the whole match). `(?: … )` groups **without** capturing. `(?<name> … )` names a group, read with `group("name")` and referenced as `${name}` in replacements. A backreference `\\1` or `\\k<name>` matches the **same text** the group matched earlier, which expresses "repeated" patterns like doubled letters.

Notes: a group that did **not participate** in the match returns `null` (the alternation example prints `null,b`); in a quantified group (`(ab)+`) `group(1)` holds the **last** repetition; use non-capturing groups for grouping only (they are cheaper and don't shift numbers). `Matcher.namedGroups()` (Java 20) returns the name-to-index map.

© TechTribers. For personal use only. Do not redistribute.

```java
Matcher m = Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})").matcher("on 2026-10-15 and 2027-01-02");
while (m.find()) System.out.println(m.group("year") + "/" + m.group("month") + "/" + m.group("day") + " " + m.group(0));
System.out.println("2026-10-15".replaceAll("(?<y>\\d{4})-(?<m>\\d{2})-(?<d>\\d{2})", "${d}/${m}/${y}"));
System.out.println("hello wood".replaceAll("(o)\\1", "[$1$1]") + " " + "abcabc".matches("(abc)\\1") + " " + "xyzxy".matches("(xy)z\\1"));
Matcher nc = Pattern.compile("(?:ab)+(c)").matcher("ababc");
nc.matches();
System.out.println(nc.groupCount() + " " + nc.group(1));
System.out.println(Pattern.compile("(a)|(b)").matcher("b").results().map(r -> r.group(1) + "," + r.group(2)).toList());
System.out.println(Pattern.compile("(\\w)\\1").matcher("committee").results().map(MatchResult::group).toList());
```

▶ Output

```
2026/10/15 2026-10-15
2027/01/02 2027-01-02
15/10/2026
hello w[oo]d true true
1 c
[null,b]
[mm, tt, ee]
```

💡 **Pro tip**

Prefer **named groups** in anything long-lived: `group("year")` survives edits to the pattern, `group(3)` doesn't.

Q635 ST-056 • Advanced

What is the difference between greedy, reluctant and possessive quantifiers?

⚡ **Quick answer**

Greedy (`X` , `X+` , `X?` , `X{n,m}`) *take as much as possible and* ***backtrack*** *(give characters back) if the rest fails.* ***Reluctant/lazy*** (`X??` , `X+?` , `X??`) take as little as possible and expand on failure. **Possessive** (`X*+` , `X++` , `X?+`) take as much as possible and **never give back**, which is faster and prevents catastrophic backtracking but can cause a match to fail. `(?>...)` is an atomic group with the same effect.

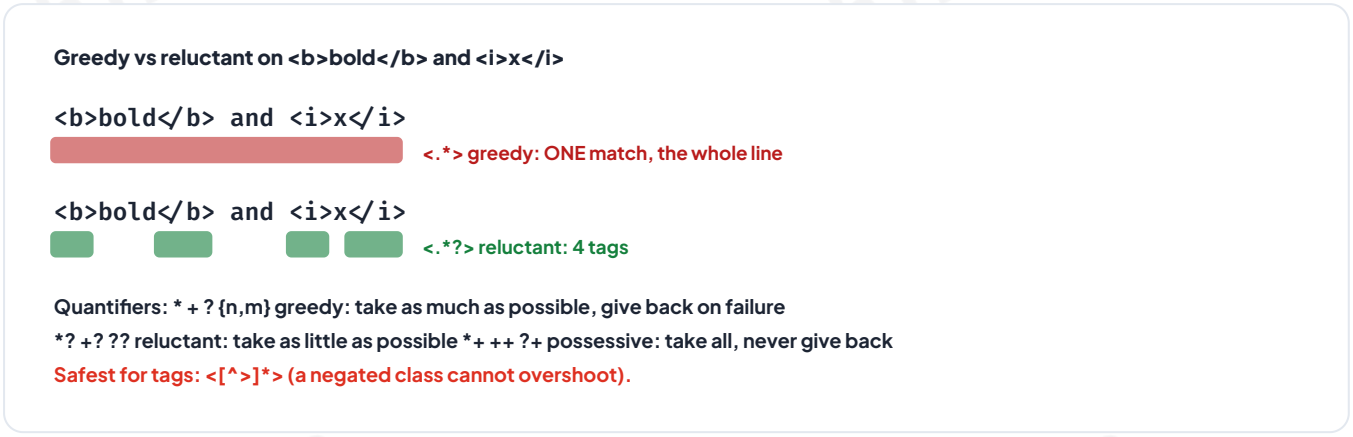
Why `a*+a` fails on `aaa` : the possessive `a*+` consumes all three `a` s and refuses to give one back, so the final `a` has nothing to match. That is exactly the property that makes possessive quantifiers safe against backtracking explosions. Practical rule for delimiters: prefer a **negated class** (`<[^>]>` , `"[^"]*"`) over `.*?` : it can't overshoot and is faster.

© TechTribers. For personal use only. Do not redistribute.

```java
String html = "<b>bold</b> and <i>italic</i>";
System.out.println(html.replaceAll("<.*>", "X") + " | " + html.replaceAll("<.*?>", "X") + " | " +
html.replaceAll("<[^>]*>", "X"));
Matcher g = Pattern.compile("a.*b").matcher("aXbYb"); g.find();
Matcher r = Pattern.compile("a.*?b").matcher("aXbYb"); r.find();
System.out.println(g.group() + " " + r.group());
System.out.println("aaa".matches("a*a") + " " + "aaa".matches("a*+a") + " " + "aaab".matches("a++b")
+ " " + "aaab".matches("(?>a+)b"));
System.out.println("xaaay".replaceAll("a{2}", "-") + " " + "xaaay".replaceAll("a{1,2}", "-") + " " +
"xaaay".replaceAll("a{1,2}?", "-") + " " + "xaaay".replaceAll("a{2,}", "-"));
```

▶ Output

```
X | XboldX and XitalicX | XboldX and XitalicX
aXbYb aXb
true false true true
x-ay x--y x---y x-y
```



⚠️ **Interview trap**

`<.>` *matches from the first* `<` *to the* ***last*** `>` *in the line. Greedy* `.` is the most common reason a regex "matches too much".

👀 **Q636** ST-057 • Advanced

How do lookahead, lookbehind and word boundaries work (and what are their traps)?

⚡ **Quick answer**

Lookarounds are **zero-width** assertions: `(?=x)` positive lookahead, `(?!x)` negative lookahead, `(?<=x)` positive lookbehind, `(?<!x)` negative lookbehind. They test the surrounding text without consuming it. `\\b` is a word boundary and `\\B` its opposite. Lookbehind in Java needs a **bounded** length (no unbounded `*` or `+`).

© TechTribers. For personal use only. Do not redistribute.

The `\\d+(?!px)` result (`?0px ?em`) is the trap: `\\d+` matches `100`, the lookahead fails because `px` follows, so the engine **backtracks** to `10` (followed by `0`, not `px`) and succeeds. To exclude whole numbers followed by `px` write `\\d+(?![\\d]|px)` or `\\b\\d+\\b(?!px)`.

Lookaheads at the start (`^(?=...)(?=...)`) are the standard way to express **independent password rules**, but check complexity rules with code or a policy library, not a growing regex. Splitting `camelCase` and inserting thousands separators are classic zero-width tricks.

Java

```
String pw = "Passw0rd!";
Pattern strong = Pattern.compile("^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[^\\w\\s]).{8,}$");        //
several independent conditions at one position
System.out.println(strong.matcher(pw).matches() + " " + strong.matcher("password").matches() + " " +
strong.matcher("Short1!").matches());
System.out.println("price: $100, cost: $25".replaceAll("(?<=\\$)\\d+", "N") + " " +
"foo.bar.baz".replaceAll("\\.(?=baz)", "!") + " " + "100px 200em".replaceAll("\\d+(?!px)", "?"));
System.out.println("cat concat category".replaceAll("\\bcat\\b", "dog") + " " + "cat
concat".replaceAll("cat", "dog"));
System.out.println(Arrays.toString("camelCaseString".split("(?<=[a-z])(?=[A-Z])")) + " " +
"1234567".replaceAll("(?<=\\d)(?=(\\d{3})+$)", ","));
```

▶ Output

```
true false false
price: $N, cost: $N foo.bar!baz ?0px ?em
dog concat category dog condog
[camel, Case, String] 1,234,567
```

💡 **Pro tip**

When a regex gets complicated, build it from named, tested pieces with `Pattern.COMMENTS` (the `x` flag) and comment each part.

🚩 **Q637** ST-058 • Intermediate

What do the Pattern flags do, and what are Pattern.quote and LITERAL for?

⚡ **Quick answer**

Flags: `CASE_INSENSITIVE` (ASCII only unless `UNICODE_CASE`), `MULTILINE` (`^` / `$` match at line boundaries), `DOTALL` (`.` matches newlines), `COMMENTS` (ignore whitespace and `#` comments in the pattern), `LITERAL` (treat the pattern as plain text), `UNICODE_CASE`, `UNICODE_CHARACTER_CLASS`. Inline forms: `(?i)`, `(?m)`, `(?s)`, `(?x)`. * *`Pattern.quote(text)`* * wraps text in `\\Q…\\E` so metacharacters match literally.

Use `Pattern.quote` (or `LITERAL`) whenever the pattern text comes from data (a search box, a configuration value, a file name): otherwise `.`, `*` or `(` in the input change the meaning or throw `PatternSyntaxException`. Flags combine with `|`. Matching dates, ids and keys case-insensitively should use `CASE_INSENSITIVE | UNICODE_CASE` only if non-ASCII data is expected, since the Unicode folding is slower.

© TechTribers. For personal use only. Do not redistribute.

```java
System.out.println(Pattern.compile("hello", Pattern.CASE_INSENSITIVE).matcher("HeLLo").matches() + "
" + "HeLLo".matches("(?i)hello") + " " + Pattern.compile("^b",
Pattern.MULTILINE).matcher("a\nb").find() + " " + Pattern.compile("^b").matcher("a\nb").find());
System.out.println(Pattern.compile("a.b", Pattern.DOTALL).matcher("a\nb").matches() + " " +
Pattern.compile("a.b").matcher("a\nb").matches() + " " + Pattern.compile("a b # comment",
Pattern.COMMENTS).matcher("ab").matches());
System.out.println(Pattern.compile("\\p{Lu}+", Pattern.UNICODE_CASE |
Pattern.CASE_INSENSITIVE).matcher("\u00c9a").matches() + " " + Pattern.compile("1+1",
Pattern.LITERAL).matcher("1+1").matches() + " " + Pattern.quote("1+1") + " " +
Pattern.compile(Pattern.quote("1+1")).matcher("1+1").matches());
Pattern shared = Pattern.compile("\\d+");                                    // thread-safe:
compile once, share
System.out.println(shared.matcher("a12b345").results().map(MatchResult::group).toList() + " " +
shared.flags() + " " + Pattern.compile("x", Pattern.MULTILINE | Pattern.DOTALL).flags());
```

▶ Output

```
true true true false
true false true
true true \Q1+1\E true
[12, 345] 0 40
```

⚠️ **Interview trap**

`MULTILINE` changes `^` and `$` only; it does **not** make `.` match newlines (that is `DOTALL`). People often need both when parsing multi-line text.

💣 **Q638** ST-059 • Advanced

What is catastrophic backtracking (ReDoS), and how do you defend against it?

⚡ **Quick answer**

A backtracking regex engine can take **exponential time** on certain patterns and inputs (nested or overlapping quantifiers such as `(.*a){11}` or `(a|aa)+$`) when the match fails. A user-supplied string can then freeze a thread: a **ReDoS** denial of service. Defences: avoid nested/overlapping quantifiers, use **possessive quantifiers or atomic groups**, cap input length before matching, anchor and constrain patterns, use a timeout-capable or non-backtracking engine (RE2/J), and test patterns against adversarial inputs.

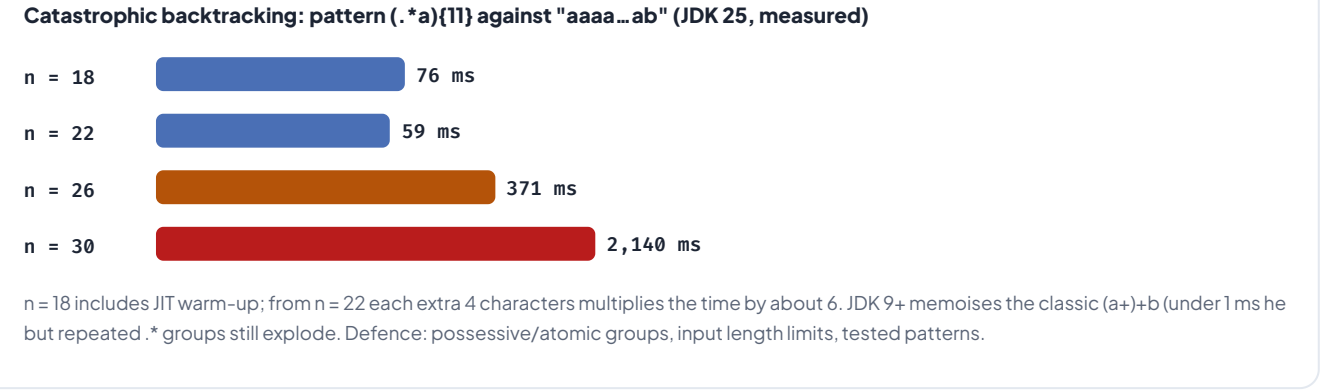
Honest scope. The famous examples `(a+)+b`, `^(a|aa)+$` and `^(\\w+\\s?)*$` did **not** blow up on JDK 25 in my tests (each finished in about a millisecond, because the JDK's `Pattern` caches failed positions for many loops), so don't assume every textbook ReDoS pattern still hurts on current JDKs. `(.*a){11}` still did, and the same engine exists in older JDKs where the classics are catastrophic. Treat the problem as real, but **measure your own patterns**.

Defences in practice: validate length first (`if (input.length() > 200) reject`), prefer specific classes over `.`, *make quantified groups unambiguous, use* `+` / `(?>...)` where backtracking can't help, run untrusted regex

© TechTribers. For personal use only. Do not redistribute.

matching with a time budget (a `CharSequence` wrapper that checks a deadline in `charAt` is a known trick), and never let users supply the regex itself.

```java
// Pattern: (.*a){11}   Input: "a" repeated n times followed by "b" (the match must fail)
Pattern risky = Pattern.compile("(.*a){11}");
for (int n : new int[]{18, 22, 26, 30}) {
    String input = "a".repeat(n) + "b";
    long t = System.nanoTime();
    risky.matcher(input).matches();
    System.out.println("n=" + n + ": " + (System.nanoTime() - t) / 1_000_000 + " ms");
}
// Measured on JDK 25 (one run):
//   n=18: 76 ms (includes JIT warm-up);  n=22: 59 ms;  n=26: 371 ms;  n=30: 2140 ms     <- from n=22, about 6x per 4 extra characters
// Classic textbook patterns such as (a+)+b finished in < 1 ms: JDK 9+ memoises many of them.
```



n = 18 includes JIT warm-up; from n = 22 each extra 4 characters multiplies the time by about 6. JDK 9+ memoises the classic (a+)+b (under 1 ms he but repeated .* groups still explode. Defence: possessive/atomic groups, input length limits, tested patterns.

⚠️ **Interview trap**

A regex that is fast on matching inputs can be slow on **non-matching** ones. Always include long, almost-matching inputs in tests.

Q639 ST-060 • Intermediate

What are reliable regex recipes (email, IPv4, date, phone), and what are their limits?

⚡ **Quick answer**

Regex is good for **shape** checks (format, character classes, rough ranges), not for full validation. A simple email regex accepts something that looks like an address, an IPv4 pattern can enforce 0–255 octets, a date regex can restrict month and day ranges but not that `2026-02-31` doesn't exist. Validate *semantics* with real parsers (`LocalDate` , `InetAddress` , `jakarta.mail`) and confirm email by sending a message.

© TechTribers. For personal use only. Do not redistribute.

The `2026-02-31` line is the point: the regex says **true** (right shape) while `LocalDate.parse` rejects the impossible day. The same split applies to email (`a@b` is rejected here but valid on an intranet; `"quoted"@example.com` is valid but rejected), IPv6 (use `InetAddress` /Guava), URLs (`java.net.URI`) and phone numbers (use libphonenumber).

Java

```
Map<String, Pattern> recipes = new LinkedHashMap<>();
recipes.put("email (simple)", Pattern.compile("^[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+$"));
recipes.put("IPv4", Pattern.compile("^((25[0-5]|2[0-4]\\d|1?\\d?\\d)(\\.|$)){4}$"));
recipes.put("date yyyy-mm-dd", Pattern.compile("^\\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\\d|3[01])$"));
recipes.put("Indian mobile", Pattern.compile("^(\\+91[- ]?)?[6-9]\\d{9}$"));
recipes.put("hex colour", Pattern.compile("^#([0-9a-fA-F]{3}){1,2}$"));
String[][] tests = {{"email (simple)", "asha.k+tt@dev.example.com"}, {"email (simple)", "a@b"},
{"IPv4", "192.168.1.255"}, {"IPv4", "256.1.1.1"},
        {"date yyyy-mm-dd", "2026-02-31"}, {"date yyyy-mm-dd", "2026-13-01"}, {"Indian mobile", "+91
9876543210"}, {"Indian mobile", "5876543210"}, {"hex colour", "#FF00aa"}, {"hex colour", "#12"}};
for (String[] t : tests) System.out.println(t[0] + " " + t[1] + " -> " +
recipes.get(t[0]).matcher(t[1]).matches());
try { LocalDate.parse("2026-02-31"); } catch (DateTimeParseException e) {
System.out.println("LocalDate rejects it: " + e.getMessage()); }
```

▶ Output

```
email (simple) asha.k+tt@dev.example.com -> true
email (simple) a@b -> false
IPv4 192.168.1.255 -> true
IPv4 256.1.1.1 -> false
date yyyy-mm-dd 2026-02-31 -> true
date yyyy-mm-dd 2026-13-01 -> false
Indian mobile +91 9876543210 -> true
Indian mobile 5876543210 -> false
hex colour #FF00aa -> true
hex colour #12 -> false
LocalDate rejects it: Text '2026-02-31' could not be parsed: Invalid date 'FEBRUARY 31'
```

💡 **Pro tip**

Write regex as the **cheap first filter**, then validate with a real parser or a service. It keeps the pattern readable and the validation correct.

© TechTribers. For personal use only. Do not redistribute.

Q640 ST-061

• Intermediate

What does regex cost, and why do String.matches, replaceAll and split compile a Pattern on every call?

⚡ **Quick answer**

`String.matches`, `replaceAll`, `replaceFirst` and `split` (for patterns longer than a single non-special character) call `Pattern.compile` **every time**, then match. Compiling is far more expensive than matching a short string, so in loops hoist the pattern into a `static final Pattern`. Measured: 200,000 digit checks took about **165 ms** with `String.matches`, **29 ms** with a precompiled `Pattern`, 58 ms with `chars().allMatch` and 22 ms with a plain loop.

Measured (200,000 checks of the 5-character string `12345`, JDK 25, three runs):

Approach	Time
`String.matches("\\d+")`	165–184 ms
`chars().allMatch(...)`	58–95 ms
Precompiled `Pattern`	29–34 ms
Plain `charAt` **loop**	22–39 ms

Guidance: precompile when the regex is used repeatedly; use a plain loop or `indexOf` for trivial tests (digits only, prefix, one delimiter); reserve regex for genuinely structured text. `String.split(String)` has a **fast path** for single-character delimiters that are not regex metacharacters (and two-character escaped ones), which is why `split(",")` is cheap but `split("\\s+")` compiles a pattern each time.

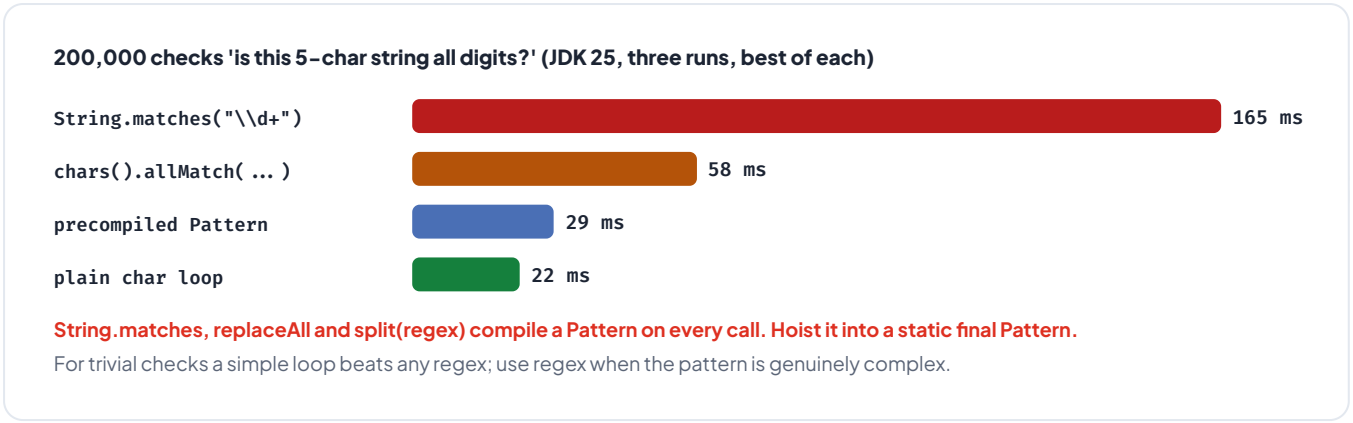
Java

```
static final Pattern DIGITS = Pattern.compile("\\d+");
static boolean isDigitsLoop(String s) { if (s.isEmpty()) return false; for (int i = 0; i <
s.length(); i++) { char c = s.charAt(i); if (c < '0' || c > '9') return false; } return true; }
String[] samples = {"12345", "12a45", "", "007"};
for (String s : samples) System.out.println("[" + s + "] " + s.matches("\\d+") + " " +
DIGITS.matcher(s).matches() + " " + isDigitsLoop(s) + " " + (!s.isEmpty() && s.chars().allMatch(c ->
c >= '0' && c <= '9')));
System.out.println("a,b;c".split("[,;]").length + " " + Pattern.compile("
[,;]").split("a,b;c").length + " " + "a,b".split(",").length);        // a single plain char: split
has a fast path, no Pattern
```

▶ **Output**

```
[12345] true true true true
[12a45] false false false false
[] false false false false
[007] true true true true
3 3 2
```


© TechTribers. For personal use only. Do not redistribute.



String.matches, replaceAll and split(regex) compile a Pattern on every call. Hoist it into a static final Pattern.

For trivial checks a simple loop beats any regex; use regex when the pattern is genuinely complex.

💡 **Pro tip**

A `static final Pattern` costs nothing, is thread-safe, and documents the format. It is the single cheapest regex optimisation.

Section 08

Best Practices & Pitfalls

A checklist of what to do (and not do) with strings: null handling, security, internationalisation, logging, and performance.

Questions 641–648 · 8 questions



📋 **Q641** ST-062 • Basic

What are the key best practices when working with Strings in Java?

⚡ **Quick answer**

Compare with `equals` (constant first or `Objects.equals`), never `==`; build in loops with `StringBuilder`; pass `Locale.ROOT` and an explicit charset for machine text; compile regexes once; prefer `isBlank` / `strip` over `trim`; never log or store secrets as strings; use `String.join` / `Collectors.joining` instead of manual delimiter logic; and keep domain values in enums, not free-form strings.

Do	Don't
`"lit".equals(s)`, `Objects.equals(a, b)`	`s == "lit"`, `s.equals("lit")` when `s` may be null
`StringBuilder` for loops	`s += x` in a loop
`toLowerCase(Locale.ROOT)`	`toLowerCase()` on identifiers
`getBytes(UTF_8)`	`getBytes()`

© TechTribers. For personal use only. Do not redistribute.

`static final Pattern`	`s.matches(...)` in a hot loop
`isBlank()`, `strip()`	`trim().isEmpty()` (misses Unicode spaces)
`split(sep, -1)` for CSV-like data	`split(sep)` that drops trailing fields
Enums / constants for fixed values	Free-form strings as flags

Java

```
String input = null;
System.out.println("expected".equals(input) + " " + Objects.requireNonNullElse(input, "dflt") + " "
+ Objects.toString(input, "-") + " " + String.valueOf(input) + " " + ("" + input));
List<String> parts = List.of("a", "b", "c");
System.out.println(String.join(",", parts) + " " +
parts.stream().collect(Collectors.joining(",")).equals("a,b,c"));
StringBuilder sb = new StringBuilder(32);
for (int i = 0; i < 5; i++) sb.append(i).append(',');
if (!sb.isEmpty()) sb.setLength(sb.length() - 1);                    // remove the
trailing separator
System.out.println(sb);
System.out.println("  ".isBlank() + " " + "".isEmpty() + " " + "\u2003 ".isBlank() + " " + " x
".strip().equals("x") + " " + "\u2003x".trim().length() + " " + "\u2003x".strip().length());
System.out.println("a-b".split("-", -1).length + " " + "ABC".toLowerCase(Locale.ROOT) + " " + new
String("x".getBytes(StandardCharsets.UTF_8), StandardCharsets.UTF_8));
```

▶ Output

```
false dflt - null null
a,b,c true
0,1,2,3,4
true true true true 2 1
2 abc x
```

💡 Pro tip

`trim()` removes only characters up to U+0020; `strip()` removes **Unicode whitespace** such as the em space `\u2003` (the last line shows 2 vs 1).

Q642 ST-063 • Intermediate

What are the most common String pitfalls, and how do you spot them in code review?

⚡ Quick answer

Ignoring the **return value** of an immutable method (`s.toUpperCase();`), `==` for content, regex surprises (`replaceAll(".", ...)` , `split(".")`), `compareTo` on numeric strings, `char` arithmetic (`'a' + 1` is an `int`), `+` precedence (`1 + 2 + "a"`), `substring` off-by-one, and assuming `length()` equals the number of characters.

© TechTribers. For personal use only. Do not redistribute.

Review checklist: (1) every call to a `String` method whose result isn't used; (2) `==` / `!=` between strings; (3) `.equals` on a possibly-null receiver; (4) `+=` in loops; (5) regex methods with unescaped metacharacters or user-controlled patterns; (6) `split` without `-1` for fixed-width records; (7) `toLowerCase()` / `getBytes()` / `format` without locale/charset; (8) strings used as secrets, ids with meaning, or enum substitutes.

The `compareTo` line: `"10".compareTo("9")` is negative (`'1' < '9'`), so numeric strings sort wrong lexicographically (`"10"` before `"9"`).

Java

```
String s = "Hello";
s.toUpperCase();                                                    // result ignored: strings
are immutable
System.out.println(s);
s = s.toUpperCase();
System.out.println(s);
System.out.println("abc".substring(1, 2) + " " + "a,b,c".replaceAll(".", "x") + " " +
"a.b".split("\\.").length + " " + "a.b".split(".").length);
System.out.println("10".compareTo("9") + " " + (Integer.parseInt("10") - Integer.parseInt("9")) + "
" + "apple".compareTo("Banana"));
char c = 'a'; c += 1;
System.out.println(c + " " + (c + 1) + " " + (char) (c + 1) + " " + ("" + c + 1) + " " + (int) 'A');
System.out.println("a" + 'b' + 'c' + " " + ('a' + 'b' + "c") + " " + 'a' + 1 + " " + (char) ('a' +
1));
System.out.println(new StringBuilder().append('a' + 1).append((char) ('a' + 1)));
```

▶ Output

```
Hello
HELLO
b xxxxx 2 0
-8 1 31
b 99 c b1 65
abc 195c a1 b
98b
```

⚠ Interview trap

`'a' + 'b' + "c"` is `195c` , not `abc` : the first `+` is integer addition. Put a string first (`"" + 'a' + 'b'`) or use `String.valueOf` / `StringBuilder` .

© TechTribers. For personal use only. Do not redistribute.

Q643 ST-064

• Intermediate

How should you handle null and empty strings?

⚡ Quick answer

Decide the contract: either the API **never returns null** (return `""`, an empty list or `Optional`) or it documents when it does. Use `Objects.requireNonNull` to fail fast on required arguments, `Objects.requireNonNullElse` / `Objects.toString(o, dflt)` for defaults, `String.valueOf(o)` to print safely, and `isBlank()` for "empty or whitespace". Distinguish *null* (unknown), *empty* (known, no text) and *blank* (only whitespace).

Conventions that scale: parameters that must have text → validate at the boundary (`requireNonNull` + `isBlank` check); return values → never null for strings and collections; `Optional<String>` only as a return type (not a field or parameter). `"" + null` and `+ null` produce the text `null`, which hides bugs (a literal `null` in an email or SQL), while `concat(null)` throws.

Java

```
String name = null;
System.out.println(Objects.requireNonNullElse(name, "anonymous") + " " + Objects.toString(name, "")
+ "| " + String.valueOf(name) + " " + Optional.ofNullable(name).map(String::strip).filter(s ->
!s.isEmpty()).orElse("none"));
for (String t : new String[]{null, "", "  ", "x"}) System.out.println((t == null ? "null" : "[" + t
+ "]") + " -> " + (t == null || t.isBlank()) + " " +
Optional.ofNullable(t).filter(Predicate.not(String::isBlank)).isPresent());
try { Objects.requireNonNull(name, "name must not be null"); } catch (NullPointerException e) {
System.out.println(e.getMessage()); }
static String nullToEmpty(String s) { return s == null ? "" : s; }
static String emptyToNull(String s) { return s == null || s.isEmpty() ? null : s; }
System.out.println("[" + nullToEmpty(null) + "] " + emptyToNull("") + " " + emptyToNull("x") + " " +
(name + "!") + " " + "x".concat(String.valueOf(name)));
try { "x".concat(null); } catch (NullPointerException e) { System.out.println("concat(null) throws;
+ null does not"); }
```

▶ Output

```
anonymous | null none
null -> true false
[] -> true false
[  ] -> true false
[x] -> false true
name must not be null
[] null x null! xnull
concat(null) throws; + null does not
```

⚠ Interview trap

`Optional.of(x)` throws on null (use `ofNullable`), and `Objects.requireNonNullElse(a, b)` throws if both are null.

© TechTribers. For personal use only. Do not redistribute.

🏷️ **Q644** ST-065 • Intermediate

Why avoid "stringly-typed" code, and what should replace it?

⚡ **Quick answer**

Using strings for values with a fixed meaning (status, type, role, currency) loses compile-time checks: typos compile, switch statements can't be exhaustive, and refactoring can't find usages. Replace them with **enums** (finite sets), **records/value classes** (structured data like `Email`, `OrderId`), `java.time` types for dates, `BigDecimal` / `Money` for amounts, and `URI` / `Path` / `UUID` for what they are.

The bad version silently returns `unknown` for the typo `"ACTIV"` and for the legitimate-but-forgotten `"CLOSED"`. The enum version is **exhaustive** (adding a constant makes the compiler flag every `switch`), and the `Email` record centralises validation and normalisation once. In APIs keep strings at the **boundary** (JSON, DB columns) and convert immediately to typed values, so the domain code never handles raw text.

Java

```
enum Status { ACTIVE, SUSPENDED, CLOSED }
record Email(String value) {
    Email { if (!value.contains("@")) throw new IllegalArgumentException("not an email: " + value);
value = value.toLowerCase(Locale.ROOT); }
}
static String describeBad(String status) { switch (status) { case "ACTIVE": return "ok"; case
"SUSPENDED": return "paused"; default: return "unknown"; } }
static String describe(Status s) { return switch (s) { case ACTIVE -> "ok"; case SUSPENDED ->
"paused"; case CLOSED -> "gone"; }; }
System.out.println(describeBad("ACTIV") + " " + describeBad("CLOSED") + " " +
describe(Status.CLOSED));
System.out.println(Status.valueOf("ACTIVE") + " " + Status.ACTIVE.name().equals("ACTIVE") + " " +
new Email("Asha@TT.com") + " " + new Email("Asha@TT.com").equals(new Email("asha@tt.com")));
try { Status.valueOf("ACTIV"); } catch (IllegalArgumentException e) {
System.out.println(e.getMessage().replace(Status.class.getCanonicalName(), "Status")); }
try { new Email("nope"); } catch (IllegalArgumentException e) { System.out.println(e.getMessage());
}
```

▶ **Output**

```
unknown unknown gone
ACTIVE true Email[value=asha@tt.com] true
No enum constant Status.ACTIV
not an email: nope
```

💡 **Pro tip**

Enum names are stable identifiers: persist `name()` (not `ordinal()`) and parse with `valueOf` inside a try/catch or a lookup map at the boundary.

© TechTribers. For personal use only. Do not redistribute.

Q645 ST-066

• Advanced

What are the string-related security risks (SQL, log, format-string injection, path traversal)?

⚡ Quick answer

Building commands by **concatenating untrusted strings** lets input become code: **SQL injection** (use `PreparedStatement` bind parameters), **log injection/forging** (strip CR/LF from logged input), **format-string injection** (never pass user text as the *format* of `String.format` / `printf` / `MessageFormat`), **path traversal** (`../` in file names: `normalize()` and check `startsWith(base)`), plus HTML/JS (escape for the output context) and regex injection (`Pattern.quote`).

Principles: **parameterise** (SQL, LDAP, XPath), **encode for the sink** (HTML, URL, JSON, shell), **validate against an allow-list** for structured input, **normalise before checking** (paths, Unicode, case), and keep secrets out of strings and logs. Use `Path.normalize()` and `startsWith` (or `toRealPath`) for file access; never build shell commands from strings (use `ProcessBuilder` with an argument list).

Java

```
String user = "x' OR '1'='1";
String bad = "SELECT * FROM users WHERE name = '" + user + "'";            // concatenation: the
input becomes SQL
System.out.println(bad);
System.out.println("SELECT * FROM users WHERE name = ?   -- the same input bound as a parameter is "
+ user.length() + " characters of DATA");
String log = "login failed for asha\n2026-10-15 INFO admin logged in";
System.out.println(log.replaceAll("[\\r\\n]", "_"));                        // neutralise forged
log lines
try { String.format("Hello " + "%s %s", "only one"); } catch (IllegalFormatException e) {
System.out.println("format string injection: " + e.getClass().getSimpleName()); }
Path base = Path.of("/srv/files").toAbsolutePath().normalize();
Path requested = base.resolve("../../etc/passwd").normalize();
System.out.println(requested + " inside base? " + requested.startsWith(base));
System.out.println(Pattern.quote("1+1") + " " + "<b>".replace("<", "&lt;").replace(">", "&gt;"));
```

▶ Output

```
SELECT * FROM users WHERE name = 'x' OR '1'='1'
SELECT * FROM users WHERE name = ?   -- the same input bound as a parameter is 12 characters of DATA
login failed for asha_2026-10-15 INFO admin logged in
format string injection: MissingFormatArgumentException
/etc/passwd inside base? false
\Q1+1\E &lt;b&gt;
```

⚠ Interview trap

Allow-lists beat block-lists: removing `../` from input once leaves `..../` which becomes `../` again after the replacement. Normalise and then check.

© TechTribers. For personal use only. Do not redistribute.

Q646 ST-067 • Intermediate

How do you externalise and format messages for different languages (ResourceBundle, MessageFormat, ChoiceFormat)?

⚡ Quick answer

Keep user-visible text out of the code: `ResourceBundle` loads `messages_xx.properties` for a locale, `MessageFormat` fills `{0}` placeholders with locale-aware number/date formatting (apostrophes must be doubled `''`), and `ChoiceFormat` expresses simple plurals. Never build sentences by concatenation, since word order differs between languages. For full plural and gender rules use ICU4J's `MessageFormat` or the framework's i18n support.

Two classic surprises shown in the last line: `MessageFormat` formats a **number** argument with grouping (`1,234,567`) but leaves a **string** unchanged, and a single `'` starts a quoted section (so `It's {0}` loses the placeholder; write `It''s {0}`). In applications use `ResourceBundle.getBundle("messages", locale)` with files per language and keep a **default** bundle as fallback.

Java

```
System.out.println(MessageFormat.format("{0} has {1} items costing {2}", "Cart", 3, 1234.5));
System.out.println(MessageFormat.format("It''s {0}", "ok") + " " + MessageFormat.format("
{0,number,#.##} {1,number,percent}", 3.14159, 0.256));
MessageFormat plural = new MessageFormat("{0,choice,0#no files|1#one file|1<{0} files}");
for (int n : new int[]{0, 1, 5}) System.out.println(plural.format(new Object[]{n}));
ResourceBundle rb = new ListResourceBundle() { protected Object[][] getContents() { return new
Object[][]{{"greeting", "Hello, {0}!"}}; } };
System.out.println(MessageFormat.format(rb.getString("greeting"), "Asha"));
System.out.println(MessageFormat.format("{0}", 1234567) + " " + MessageFormat.format("{0}",
"1234567") + " " + String.format("%s", 1234567));
```

▶ Output

```
Cart has 3 items costing 1,234.5
It's ok 3.14 26%
no files
one file
5 files
Hello, Asha!
1,234,567 1234567 1234567
```

💡 Pro tip

Give translators whole sentences with named or numbered placeholders, never fragments to concatenate; the same sentence needs different word order in Hindi, German or Japanese.

© TechTribers. For personal use only. Do not redistribute.

Q647 ST-068

• Intermediate

Why use parameterised logging instead of string concatenation?

⚡ Quick answer

`log.debug("value=" + obj)` builds the message (calling `toString()` and allocating strings) **before** the logger checks whether debug is enabled, so disabled levels still cost CPU and garbage. Parameterised logging (`log.debug("value={}", obj)` in SLF4J) defers formatting until the level is enabled, and for expensive arguments you can pass a **supplier** or guard with `isDebugEnabled()` .

The counter shows the cost: the concatenation called `toString()` once even though nothing was printed; the guarded and lazy forms called it zero times. Loggers (SLF4J/Logback, Log4j 2) format `{}` placeholders only when the level is enabled, which is what makes `log.debug("user {} logged in from {}", id, ip)` cheap when debug is off. Also use `{}` for **exceptions** as the last argument so the stack trace is logged.

Java

```
static final AtomicInteger calls = new AtomicInteger();
class Expensive { @Override public String toString() { calls.incrementAndGet(); return "expensive";
} }
Expensive e = new Expensive();
boolean debug = false;
String eager = "value=" + e;                                    // always builds the
string, even if nobody logs it
System.out.println(calls.get());
if (debug) System.out.println("value=" + e);                    // guarded: not executed
System.out.println(calls.get());
Supplier<String> lazy = () -> "value=" + e;                     // lazy: built only when
asked
System.out.println(calls.get() + " " + (debug ? lazy.get() : "skipped") + " " + calls.get());
System.out.println(MessageFormat.format("value={0}", e) + " " + calls.get());
```

▶ Output

```
1
1
1 skipped 1
value=expensive 2
```

⚠️ Interview trap

`log.debug("value=" + expensive())` evaluates `expensive()` eagerly regardless of level. Pass a lambda (`() -> expensive()`) with APIs that accept a `Supplier` , or guard it.

© TechTribers. For personal use only. Do not redistribute.

Q648 ST-069

• Intermediate

What are the practical String performance tips?

⚡ Quick answer

Avoid repeated concatenation in loops, pre-size builders, hoist compiled patterns, prefer `indexOf` / `charAt` loops over regex for trivial parsing, avoid needless copies (`substring` , `toCharArray` , `toString()` of a builder in a loop), parse with `Pattern` / `CharSequence` views, keep text Latin-1 where possible, and don't intern or deduplicate unless a heap dump proves duplicates. Measure with JMH; micro-optimising without data usually changes nothing.

Ranked by usual payoff: (1) don't build quadratic strings; (2) stream large text instead of holding it; (3) precompile regexes and avoid regex for simple splits; (4) avoid `String.format` in hot paths; (5) reuse `StringBuilder` s *within a single thread/method* (not shared); (6) reduce object count (parse into numbers/enums early). Things that rarely matter: `final` on string variables, `StringBuffer` vs `StringBuilder` , and manual interning.

Since Java 9, JIT intrinsics make `indexOf` , `equals` , `compareTo` , `hashCode` and `contains` very fast (vectorised on x86/ARM), so prefer the built-in methods over hand-written loops for those operations.

Java

```
String csv = "id,name,qty\n1,pen,5\n2,ink,12\n";
List<String[]> viaSplit = csv.lines().skip(1).map(l -> l.split(",")).toList();
// simple, allocates arrays
List<String[]> viaIndex = new ArrayList<>();
for (String line : csv.lines().skip(1).toList()) {
// no regex, no array of the right size needed
    int a = line.indexOf(','), b = line.indexOf(',', a + 1);
    viaIndex.add(new String[]{line.substring(0, a), line.substring(a + 1, b), line.substring(b +
1)});
}
System.out.println(viaSplit.size() + " " + viaIndex.size() + " " + Arrays.equals(viaSplit.get(1),
viaIndex.get(1)) + " " + Arrays.toString(viaIndex.get(0)));
StringBuilder sb = new StringBuilder(csv.length());
csv.lines().forEach(l -> sb.append(l.toUpperCase(Locale.ROOT)).append('|'));
System.out.println(sb + " " + (sb.capacity() >= csv.length()));
System.out.println("a-b-c".replace("-", "") + " " + "a-b-c".chars().filter(ch -> ch != '-').count()
+ " " + String.valueOf(new char[]{'x'}).concat("y") + " " + "abc".contains("b") + " " +
"abc".indexOf('b'));
```

▶ Output

```
2 2 true [1, pen, 5]
ID,NAME,QTY|1,PEN,5|2,INK,12| true
abc 3 xy true 1
```

💡 Pro tip

When someone reports "string handling is slow", record a JFR profile first: most of the time the hot spot is regex compilation, `String.format` , or a concatenation loop.

© TechTribers. For personal use only. Do not redistribute.

Section 09

String Coding Questions

31 string problems asked in interviews: reverse, palindrome, anagram, counting, cleaning, ciphers, big numbers, parsing, stack tricks and backtracking, each with tested code and edge cases.

Questions 649–679 · 31 questions



Q649 ST-070 • Basic

Reverse a string (several ways, including surrogate-safe).

⚡ Quick answer

Swap characters from both ends of a `char[]` (O(n), in place), use `new StringBuilder(s).reverse()` (concise and surrogate-pair aware), recurse (O(n²) copying, shown for completeness) or build from `codePoints()` to keep emoji intact. Always test the empty string, one character and a palindrome.

Complexity: loop and builder are O(n) time and O(n) space (the char array). The recursive version copies on every level (O(n²) time, O(n) stack) and fails with `StackOverflowError` for long inputs. A pure in-place reverse isn't possible for `String` (immutable). Interview follow-ups: reverse **words** (earlier part), reverse only vowels, reverse with grapheme clusters.

Java

```
static String reverseLoop(String s) { char[] c = s.toCharArray(); for (int i = 0, j = c.length - 1;
i < j; i++, j--) { char t = c[i]; c[i] = c[j]; c[j] = t; } return new String(c); }
static String reverseBuilder(String s) { return new StringBuilder(s).reverse().toString(); }
static String reverseRec(String s) { return s.length() <= 1 ? s : reverseRec(s.substring(1)) +
s.charAt(0); }
static String reverseStream(String s) { return s.chars().mapToObj(c -> String.valueOf((char)
c)).reduce("", (a, b) -> b + a); }
static String reverseCodePoints(String s) { int[] cps = s.codePoints().toArray(); StringBuilder sb =
new StringBuilder(); for (int i = cps.length - 1; i >= 0; i--) sb.appendCodePoint(cps[i]); return
sb.toString(); }
for (String f : List.of("hello", "", "a", "racecar")) System.out.println("[" + reverseLoop(f) + "]
[" + reverseBuilder(f) + "] [" + reverseRec(f) + "] [" + reverseStream(f) + "] [" +
reverseCodePoints(f) + "]");
String emoji = "ab\uD83D\uDE00";
System.out.println((reverseLoop(emoji).codePointAt(0) == 0x1F600 ? "pair kept" : "pair broken") + "
" + (reverseBuilder(emoji).codePointAt(0) == 0x1F600 ? "pair kept" : "pair broken") + " " +
(reverseCodePoints(emoji).codePointAt(0) == 0x1F600 ? "pair kept" : "pair broken"));
```

▶ Output

```
[olleh] [olleh] [olleh] [olleh] [olleh]
[] [] [] [] []
[a] [a] [a] [a] [a]
```


© TechTribers. For personal use only. Do not redistribute.

```
[racecar] [racecar] [racecar] [racecar] [racecar]
pair broken pair kept pair kept
```

> 💡 **Pro tip**
> Say "StringBuilder.reverse()" first (it's the production answer), then show the two-pointer loop to prove you can do it by hand.

Q650 ST-071 • Basic

Check whether a string is a palindrome (exact, and ignoring case and punctuation).

> ⚡ **Quick answer**
> Use two pointers from both ends: O(n) time, O(1) extra space. For the loose version ("A man, a plan, a canal: Panama") skip non-alphanumeric characters with `Character.isLetterOrDigit` and compare `Character.toLowerCase` values. Empty and one-character strings are palindromes.

The reverse-and-compare one-liner is O(n) extra space; the two-pointer version needs none. For Unicode text compare **code points** (not `char` s) and consider normalisation (`e\u0301` vs `\u00e9`). A related question: *valid palindrome II* (can you delete at most one character?), solved by two pointers with one retry on mismatch.

Java

```
static boolean isPal(String s) { int i = 0, j = s.length() - 1; while (i < j) if (s.charAt(i++) !=
s.charAt(j--)) return false; return true; }
static boolean isPalLoose(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i++)) != Character.toLowerCase(s.charAt(j--))) return
false;
    }
    return true;
}
System.out.println(isPal("racecar") + " " + isPal("ab") + " " + isPal("") + " " + isPal("Aba"));
System.out.println(isPalLoose("A man, a plan, a canal: Panama") + " " + isPalLoose("race a car") + "
" + isPalLoose(".,") + " " + isPalLoose("0P"));
System.out.println(new StringBuilder("level").reverse().toString().equals("level") + " " +
"madam".contentEquals(new StringBuilder("madam").reverse()));
```

▶ **Output**

```
true false true false
true false true false
true true
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
`isPal("Aba")` is false (case-sensitive). State whether the problem wants case/punctuation ignored before you code.

Q651 ST-072 • Basic

Check whether two strings are anagrams (sorting, counting, Unicode-safe).

⚡ **Quick answer**
Two strings are anagrams if they contain the same characters with the same counts. Options: **sort both** and compare (O(n log n)), **count into an `int[26]` (O(n), for lowercase letters), or a `HashMap<Integer, Integer>` of code points** for general text. Check lengths first; normalise case and strip spaces if the problem says so.

Counting is the best general answer (O(n) time, O(1) space for a fixed alphabet). The map version handles any Unicode text. `"aab"/"abb"` shows why a **set** comparison is wrong: the counts differ. (Grouping many anagrams together, and finding anagram substrings, were covered in the coding rounds of Parts 5–6.)

Java

```
static boolean anagramSort(String a, String b) { char[] x = a.toCharArray(), y = b.toCharArray();
Arrays.sort(x); Arrays.sort(y); return Arrays.equals(x, y); }
static boolean anagramCount(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] cnt = new int[26];
    for (int i = 0; i < a.length(); i++) { cnt[a.charAt(i) - 'a']++; cnt[b.charAt(i) - 'a']--; }
    for (int c : cnt) if (c != 0) return false;
    return true;
}
static boolean anagramMap(String a, String b) {
    Map<Integer, Integer> m = new HashMap<>();
    a.codePoints().forEach(c -> m.merge(c, 1, Integer::sum));
    b.codePoints().forEach(c -> m.merge(c, -1, Integer::sum));
    return m.values().stream().allMatch(v -> v == 0);
}
for (String[] p : new String[][]{{"listen", "silent"}, {"abc", "abd"}, {"aab", "abb"}, {"", ""}})
System.out.println(p[0] + "/" + p[1] + " " + anagramSort(p[0], p[1]) + " " + anagramCount(p[0],
p[1]) + " " + anagramMap(p[0], p[1]));
System.out.println(anagramSort("Dormitory".toLowerCase().replace(" ", ""), "dirty
room".toLowerCase().replace(" ", "")));
```

▶ **Output**

```
listen/silent true true true
abc/abd false false false
aab/abb false false false
/ true true true
true
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Ask about the alphabet (26 lowercase? ASCII? Unicode?) and about case/spaces; it decides between `int[26]`, `int[128]` and a map.

Q652 ST-073 • Basic

Count vowels, consonants, digits and spaces in a string.

⚡ **Quick answer**

Classify each character once: letters are vowels if they appear in `"aeiou"` (case-insensitive), other letters are consonants, then digits, whitespace and the rest. A single pass is O(n). Use `Character.isLetter` etc. for Unicode-aware classification, or ASCII ranges if the input is known to be ASCII.

Counting vowels as `"aeiou".indexOf(c) >= 0` is the compact idiom. The regex variant allocates and is slower. Decide how to treat `y`, accented vowels (`\u00e9`) and non-letters, and say so. A loop with a `switch` on lower-cased characters is the fastest.

Java

```
String s = "Hello, World! 123";
long vowels = s.toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();
long consonants = s.chars().filter(Character::isLetter).count() - vowels;
long digits = s.chars().filter(Character::isDigit).count(), spaces =
s.chars().filter(Character::isWhitespace).count();
long others = s.length() - vowels - consonants - digits - spaces;
System.out.println(vowels + " " + consonants + " " + digits + " " + spaces + " " + others);
int v = 0, c = 0;
for (char ch : s.toCharArray()) if (Character.isLetter(ch)) { if ("aeiouAEIOU".indexOf(ch) >= 0)
v++; else c++; }
System.out.println(v + " " + c);
System.out.println(s.replaceAll("[^aeiouAEIOU]", "").length() + " " + s.replaceAll("[aeiouAEIOU]",
""));
```

▶ **Output**

```
3 7 3 2 2
3 7
3 Hll, Wrld! 123
```

💡 **Pro tip**

Lower-case once (`toLowerCase(Locale.ROOT)`) instead of listing upper and lower case vowels; but then count with the lower-cased string to avoid double conversion.

© TechTribers. For personal use only. Do not redistribute.

Q653 ST-074 • Basic

Remove duplicate characters from a string, keeping the first occurrence.

⚡ Quick answer

Scan left to right with a `Set<Character>` (or a `boolean[128]` for ASCII); append a character only if `set.add(c)` returns true. O(n) time. A stream version with `chars().distinct()` also preserves encounter order. Beware case: `aA` has two distinct characters unless you normalise.

`HashSet<Character>` boxes each char; the `boolean[128]` table is faster and allocation-free but only valid for ASCII (it throws `ArrayIndexOutOfBoundsException` for larger code units; use `boolean[65536]` or a set for general text). To remove **all** characters that repeat (keep only unique ones), count first, then filter. Related: duplicates in a list (Part 12).

Java

```
static String dedupe(String s) { StringBuilder sb = new StringBuilder(); Set<Character> seen = new
HashSet<>(); for (char c : s.toCharArray()) if (seen.add(c)) sb.append(c); return sb.toString(); }
static String dedupeStream(String s) { return s.chars().distinct().collect(StringBuilder::new,
StringBuilder::appendCodePoint, StringBuilder::append).toString(); }
static String dedupeBits(String s) { boolean[] seen = new boolean[128]; StringBuilder sb = new
StringBuilder(); for (char c : s.toCharArray()) if (!seen[c]) { seen[c] = true; sb.append(c); }
return sb.toString(); }
System.out.println(dedupe("programming") + " " + dedupeStream("programming") + " " +
dedupeBits("programming") + " [" + dedupe("") + "] " + dedupe("aAaA"));
System.out.println(new LinkedHashSet<>
(Arrays.asList("a,b,a,c".split(","))).stream().collect(Collectors.joining(",")));
```

▶ Output

```
progamin progamin progamin [] aA
a,b,c
```

⚠️ Interview trap

`new HashSet<>(Arrays.asList(chars))` doesn't preserve order, so use `LinkedHashSet` if the first-occurrence order matters.

Q654 ST-075 • Basic

Check whether a string contains only digits, only letters or only alphanumerics (ASCII vs Unicode).

⚡ Quick answer

Decide the alphabet first. **ASCII digits**: every char between `'0'` and `'9'`. **Any-language digits/letters**: `Character.isDigit` / `isLetter` (Unicode-aware). A regex (`[0-9]+` , `\\p{L}+`) is short but slower. Don't forget the **empty string**: `allMatch` on an empty stream is `true` , so add an explicit `!s.isEmpty()` check if empty must be rejected.

© TechTribers. For personal use only. Do not redistribute.

Notice `\u0664\u0662` (Arabic-Indic digits): `Character.isDigit` is true, the ASCII check is false. Validating a field that you later pass to `Integer.parseInt` should use the **ASCII** check or be ready for `parseInt` to accept those digits. For identifiers use `Character.isJavaIdentifierStart/Part`.

Java

```
static boolean allDigits(String s) { return !s.isEmpty() && s.chars().allMatch(c -> c >= '0' && c <= '9'); }
static boolean allLetters(String s) { return !s.isEmpty() && s.chars().allMatch(Character::isLetter); }
static boolean alnumAscii(String s) { return s.matches("[A-Za-z0-9]+"); }
static String esc(String s) { StringBuilder b = new StringBuilder(); for (char c : s.toCharArray()) b.append(c < 128 ? String.valueOf(c) : String.format("\\u%04x", (int) c)); return b.toString(); }
for (String t : List.of("12345", "12a45", "", "\u0664\u0662", "abc", "ab1"))
    System.out.println("[" + esc(t) + "] digits=" + allDigits(t) + " letters=" + allLetters(t) + " alnum=" + alnumAscii(t) + " unicodeDigits=" + (!t.isEmpty() && t.chars().allMatch(Character::isDigit)));
```

▶ Output

```
[12345] digits=true letters=false alnum=true unicodeDigits=true
[12a45] digits=false letters=false alnum=true unicodeDigits=false
[] digits=false letters=false alnum=false unicodeDigits=false
[\u0664\u0662] digits=false letters=false alnum=false unicodeDigits=true
[abc] digits=false letters=true alnum=true unicodeDigits=false
[ab1] digits=false letters=false alnum=true unicodeDigits=false
```

> 💡 **Pro tip**
> Say the alphabet out loud before coding: it prevents the most common bug in validation questions.

Q655 ST-076 • Basic

Convert a sentence to Title Case (capitalise each word).

> ⚡ **Quick answer**
> Walk the characters with an `upNext` flag: after whitespace, upper-case the next letter, otherwise lower-case it. This handles multiple spaces and preserves separators. Splitting on `" "` and capitalising each word is shorter but collapses or drops spacing unless you keep the separators. Use `Character.toUpperCase` (Unicode-aware) rather than `toUpperCase()` on substrings in a locale-sensitive context.

Real title case also keeps small words lower-case (`of`, `the`), handles hyphens and apostrophes, and is language-specific; for UI text use a library or locale rules. The stream version fails with `StringIndexOutOfBoundsException` on an empty word (consecutive spaces) because `w.charAt(0)` is invalid: the character walk avoids that.

© TechTribers. For personal use only. Do not redistribute.

```java
static String titleCase(String s) {
    StringBuilder sb = new StringBuilder(); boolean up = true;
    for (char c : s.toCharArray()) {
        if (Character.isWhitespace(c)) { up = true; sb.append(c); }
        else if (up) { sb.append(Character.toUpperCase(c)); up = false; }
        else sb.append(Character.toLowerCase(c));
    }
    return sb.toString();
}
static String capitalize(String s) { return s.isEmpty() ? s : Character.toUpperCase(s.charAt(0)) +
s.substring(1); }
System.out.println(titleCase("hello wORLD of java") + "|" + titleCase("  multiple   spaces ") + "|"
+ capitalize("java") + capitalize("") + "|" + capitalize("\u00e9cole").length());
System.out.println(Arrays.stream("the quick brown fox".split(" ")).map(w ->
Character.toUpperCase(w.charAt(0)) + w.substring(1)).collect(Collectors.joining(" ")));
```

▶ Output

```
Hello World Of Java|  Multiple   Spaces |Java|5
The Quick Brown Fox
```

⚠️ Interview trap

`Character.toUpperCase(c) + s.substring(1)` has type *`String`* here only because `substring` returns a String; `Character.toUpperCase(a) + Character.toUpperCase(b)` would add two chars as integers.

🐍 **Q656** ST-077 • Intermediate

Convert between camelCase and snake_case.

⚡ Quick answer

camel → snake: insert `_` before each upper-case letter that follows a lower-case letter or digit, then lower-case (`replaceAll("([a-z0-9])([A-Z])", "$1_$2")`); acronyms (`HTTPServer`) need a second rule for an upper-case run followed by an upper+lower pair. snake → camel: walk the characters and upper-case the one after each underscore.

The first rule alone turns `HTTPServer` into `httpserver` (no lower→upper transition); the two-step version yields `http_server` . Decide the policy for leading underscores, repeated underscores and digits, and round-trip tests (`snakeToCamel(camelToSnake(x))`) reveal the ambiguities. Libraries (Jackson's `PropertyNamingStrategies` , Guava `CaseFormat`) already implement these.

© TechTribers. For personal use only. Do not redistribute.

```java
static String camelToSnake(String s) { return s.replaceAll("([a-z0-9])([A-Z])",
"$1_$2").toLowerCase(Locale.ROOT); }
static String snakeToCamel(String s) { StringBuilder sb = new StringBuilder(); boolean up = false;
for (char c : s.toCharArray()) { if (c == '_') up = true; else { sb.append(up ?
Character.toUpperCase(c) : c); up = false; } } return sb.toString(); }
for (String s : List.of("userFirstName", "HTTPServer", "parseXMLFile", "a")) System.out.println(s +
" -> " + camelToSnake(s));
for (String s : List.of("user_first_name", "_private", "a__b", "x")) System.out.println(s + " -> " +
snakeToCamel(s));
System.out.println("HTTPServer".replaceAll("([A-Z]+)([A-Z][a-z])", "$1_$2").replaceAll("([a-z0-9])
([A-Z])", "$1_$2").toLowerCase(Locale.ROOT));
```

▶ Output

```
userFirstName -> user_first_name
HTTPServer -> httpserver
parseXMLFile -> parse_xmlfile
a -> a
user_first_name -> userFirstName
_private -> Private
a__b -> aB
x -> x
http_server
```

💡 **Pro tip**

Use `Locale.ROOT` for the final `toLowerCase`, or `I` turns into a dotless `\u0131` under a Turkish default locale.

Q657 ST-078 • Basic

Remove all whitespace, or collapse multiple spaces into one.

⚡ **Quick answer**

`replaceAll("\\s+", " ")` collapses runs, `replaceAll("\\s", "")` removes all, `strip()` / `trim()` remove the ends, and `String.join(" ", s.trim().split("\\s+"))` rebuilds the string. A single pass with a `prevSpace` flag is O(n) and allocation-light. Remember that `\\s` matches ASCII whitespace only by default, so use `Character.isWhitespace` (or `(?U)`) for Unicode spaces.

The last line shows the Unicode trap: `\s` does **not** match the em space `\u2003` unless `(?U)` (UNICODE_CHARACTER_CLASS) is set, while `Character.isWhitespace` does. `trim()` also ignores it (use `strip()`). The single-pass loop is the answer when the interviewer forbids regex or asks for O(n) with one allocation.

```java
String s = "  hello \t  world \n again  ";
System.out.println("[" + s.trim().replaceAll("\\s+", " ") + "]");
System.out.println("[" + s.replaceAll("\\s", "") + "] [" + s.strip() + "] [" + String.join(" ",
```


© TechTribers. For personal use only. Do not redistribute.

```
s.trim().split("\\s+")) + "]");
StringBuilder sb = new StringBuilder(); boolean prevSpace = true;
for (char c : s.toCharArray()) {
    if (Character.isWhitespace(c)) { if (!prevSpace) sb.append(' '); prevSpace = true; }
    else { sb.append(c); prevSpace = false; }
}
if (sb.length() > 0 && sb.charAt(sb.length() - 1) == ' ') sb.setLength(sb.length() - 1);
System.out.println("[" + sb + "]");
System.out.println(s.length() + " " + s.chars().filter(Character::isWhitespace).count() + " " +
"a\u2003b".replaceAll("\\s", "").length() + " " + "a\u2003b".replaceAll("(?U)\\s", "").length());
```

▶ Output

```
[hello world again]
[helloworldagain] [hello          world
 again] [hello world again]
[hello world again]
26 11 3 2
```

💡 **Pro tip**

Non-breaking space `\u00a0` is **not** whitespace for `isWhitespace`, `strip` or `\s`; remove it explicitly (`replace('\u00a0', ' ')`) when cleaning web text.

🔐 **Q658** ST-079 • Basic

Implement a Caesar cipher (encrypt and decrypt).

⚡ **Quick answer**

Shift each letter by `k` positions within its case's 26-letter alphabet, leave other characters unchanged: `(c - base + k) % 26 + base` with `k` normalised to `0..25` (`((k % 26) + 26) % 26`) so negative shifts and shifts above 26 work. Decrypt with `-k`. Brute-force breaking tries all 25 shifts.

Normalising the shift is the usual trap: in Java `-1 % 26` is `-1` (the remainder keeps the dividend's sign), which would give a negative character offset. The cipher is trivially breakable (26 keys, frequency analysis), so it is only an exercise; real encryption uses AES via `javax.crypto`. Variants: ROT13 (`shift = 13`, its own inverse), Vigen\u00e8re (a different shift per key letter).

Java

```
static String caesar(String s, int shift) {
    StringBuilder sb = new StringBuilder(); int k = ((shift % 26) + 26) % 26;
    for (char c : s.toCharArray()) {
        if (c >= 'a' && c <= 'z') sb.append((char) ('a' + (c - 'a' + k) % 26));
        else if (c >= 'A' && c <= 'Z') sb.append((char) ('A' + (c - 'A' + k) % 26));
        else sb.append(c);
    }
    return sb.toString();
}
String enc = caesar("Hello, World!", 3);
System.out.println(enc + " " + caesar(enc, -3) + " " + caesar("xyz", 3) + " " + caesar("abc", -1) +
```


© TechTribers. For personal use only. Do not redistribute.

```
" " + caesar("abc", 29) + " " + caesar("ABC", 26));
for (int k = 1; k <= 3; k++) System.out.println(k + ": " + caesar("Khoor", -k));
```

▶ Output

```
Khoor, Zruog! Hello, World! abc zab def ABC
1: Jgnnq
2: Ifmmp
3: Hello
```

⚠️ Interview trap

`(c - 'a' - k) % 26` with a negative result produces a negative index. Add 26 first, or normalise `k` as above.

Q659 ST-080 • Basic

Decode a run-length encoded string ("a3b2c1" becomes "aaabbc").

⚡ Quick answer

Scan the input: read a character, then read the following digits as a (possibly multi-digit) count, and append the character that many times. Treat a missing count as 1. O(output length). Encoding is the reverse: count each run. (The *encode* direction was covered in the string-compression question of Part 5.)

Edge cases: multi-digit counts (`x12`), a count of 0, a missing count, and the ambiguity when the **data itself contains digits** (`"1"` repeated three times encodes as `13` , which decodes as thirteen). Real RLE formats use a separator or escape. The regex version is clear but only for well-formed input.

Java

```
static String decode(String s) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < s.length(); ) {
        char ch = s.charAt(i++); int n = 0;
        while (i < s.length() && Character.isDigit(s.charAt(i))) n = n * 10 + (s.charAt(i++) - '0');
        sb.append(String.valueOf(ch).repeat(Math.max(n, 1)));
    }
    return sb.toString();
}
static String encode(String s) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < s.length(); ) { int j = i; while (j < s.length() && s.charAt(j) ==
s.charAt(i)) j++; sb.append(s.charAt(i)).append(j - i); i = j; }
    return sb.toString();
}
System.out.println(decode("a3b2c1") + " " + decode("x12") + " " + decode("") + "| " + decode("ab") +
" " + encode("aaabbc") + " " + decode(encode("wwwwaaadexxxxxx")).equals("wwwwaaadexxxxxx"));
Matcher m = Pattern.compile("(\\D)(\\d+)").matcher("a3b2");
StringBuilder out = new StringBuilder();
while (m.find()) out.append(m.group(1).repeat(Integer.parseInt(m.group(2))));
System.out.println(out);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
aaabbc xxxxxxxxxxx | ab a3b2c1 true
aaabb
```

💡 Pro tip

Ask: "can the original text contain digits?" If yes, switch to a count-then-char format or add an escape.

🔗 Q660 ST-081 • Intermediate

Check whether two strings are isomorphic.

⚡ Quick answer

Two strings are isomorphic if the characters of the first can be replaced to get the second, with a **consistent one-to-one mapping** (no two characters map to the same target and one character never maps to two). Keep two maps (`s → t` and `t → s`) and verify both while scanning. O(n) time, O(alphabet) space.

Why two maps: `"ab"` / `"aa"` maps `a→a` and `b→a`, which a single forward map accepts but the reverse map rejects. A compact alternative compares the **first-occurrence index patterns** of both strings (`egg` → 0,1,1 and `add` → 0,1,1). Related: *word pattern* (same idea with words instead of characters).

Java

```
static boolean isomorphic(String s, String t) {
    if (s.length() != t.length()) return false;
    Map<Character, Character> st = new HashMap<>(), ts = new HashMap<>();
    for (int i = 0; i < s.length(); i++) {
        char a = s.charAt(i), b = t.charAt(i);
        if (st.computeIfAbsent(a, k -> b) != b || ts.computeIfAbsent(b, k -> a) != a) return false;
    }
    return true;
}
for (String[] p : new String[][]{{"egg", "add"}, {"foo", "bar"}, {"paper", "title"}, {"ab", "aa"},
{"", ""}}) System.out.println(p[0] + "/" + p[1] + " " + isomorphic(p[0], p[1]));
```

▶ Output

```
egg/add true
foo/bar false
paper/title true
ab/aa false
/ true
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Comparing `computeIfAbsent(...) != b` is a `Character` vs `char` comparison and unboxes (fine here), but comparing two `Character` objects with `!=` compares references and fails above the cache range.

➕ Q661 ST-082 • Intermediate

Add two non-negative numbers given as strings (without BigInteger).

⚡ **Quick answer**

Add digit by digit from the right with a **carry**, appending `sum % 10` to a `StringBuilder` and finally reversing it. Loop while either string has digits left **or** a carry remains (so `"999" + "1"` produces `"1000"`). O(max(n, m)) time. Works for numbers of any length.

Checking against `BigInteger` is the way to test such code. Pitfalls: leading zeros in the input (`"007" + "3"` gives `10`: fine), empty strings (treat as 0 or reject), negative numbers (need sign handling and subtraction), and forgetting the final carry. The same skeleton solves *add two numbers as linked lists* (Part 6) and *add binary strings* (use base 2).

Java

```
static String add(String a, String b) {
    StringBuilder sb = new StringBuilder();
    int i = a.length() - 1, j = b.length() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry > 0) {
        int sum = carry + (i >= 0 ? a.charAt(i--) - '0' : 0) + (j >= 0 ? b.charAt(j--) - '0' : 0);
        sb.append(sum % 10);
        carry = sum / 10;
    }
    return sb.reverse().toString();
}
System.out.println(add("999", "1") + " " + add("123", "456") + " " + add("0", "0") + " " +
add("9999999999999999999", "1") + " " + add("5", "95"));
System.out.println(add("12345678901234567890123456789O",
"98765432109876543210987654321O").equals(new BigInteger("12345678901234567890123456789O").add(new
BigInteger("98765432109876543210987654321O")).toString()));
```

▶ **Output**

```
1000 579 0 100000000000000000000 100
true
```

💡 **Pro tip**

Write the loop condition as `i >= 0 || j >= 0 || carry > 0` first. It's the part interviewers watch for.

© TechTribers. For personal use only. Do not redistribute.

Q662 ST-083

• Advanced

Multiply two non-negative numbers given as strings.

⚡ Quick answer

Use the schoolbook method: digit `i` of `a` times digit `j` of `b` contributes to positions `i + j` and `i + j + 1` of a result array of length `n + m`. Accumulate `mul + res[i + j + 1]`, keep `mul % 10` there and add `mul / 10` to `res[i + j]`, then strip leading zeros. O(n·m) time. Special-case `"0"`.

Why the result array has `n + m` digits: the product of an n-digit and an m-digit number has at most n + m digits. `res[i + j]` may temporarily exceed 9; it is normalised when the loop reaches that position as `i + j' + 1`. Faster algorithms exist (Karatsuba, FFT) and are what `BigInteger.multiply` uses for large operands.

Java

```
static String multiply(String a, String b) {
    if (a.equals("0") || b.equals("0")) return "0";
    int[] res = new int[a.length() + b.length()];
    for (int i = a.length() - 1; i >= 0; i--)
        for (int j = b.length() - 1; j >= 0; j--) {
            int mul = (a.charAt(i) - '0') * (b.charAt(j) - '0') + res[i + j + 1];
            res[i + j + 1] = mul % 10;
            res[i + j] += mul / 10;
        }
    StringBuilder sb = new StringBuilder();
    for (int d : res) if (!(sb.isEmpty() && d == 0)) sb.append(d);
    return sb.toString();
}
System.out.println(multiply("123", "456") + " " + multiply("999", "999") + " " + multiply("0",
"12345") + " " + multiply("99999999999", "99999999999") + " " + multiply("2", "3"));
System.out.println(multiply("123456789", "987654321").equals(new
BigInteger("123456789").multiply(new BigInteger("987654321")).toString()));
```

▶ Output

```
56088 998001 0 9999999999800000000001 6
true
```

⚠️ Interview trap

Initialising `res` to the wrong length, or trimming zeros with `Integer.parseInt` (which overflows), are the usual mistakes. Build the string from the array and skip leading zeros.

© TechTribers. For personal use only. Do not redistribute.

Q663 ST-084

• Intermediate

Validate whether a string is a valid integer or decimal number without using parse methods.

⚡ **Quick answer**

For an integer: optional sign, then at least one digit, then only digits. For a general number: optional sign; digits with an optional fractional part (`5.` , `.5` , `3.14`); optional exponent `e|E` with an optional sign and digits. A **regex** expresses the grammar compactly; a hand-written **state machine** is the interview-favourite for performance and clarity of rules. Decide the policy on whitespace, empty input and forms like `.` or `1e` .

Compare with the JDK: `Integer.parseInt(" 1")` fails but `Double.parseDouble(" 1 ")` trims; `parseInt` accepts Unicode digits; `"1e10"` is a valid double but not an int. This regex uses `\\d` (ASCII). Hex, `Infinity` / `NaN` , locale decimal commas and thousands separators are separate grammars; use `NumberFormat` / `BigDecimal` for those.

Java

```
static boolean isInteger(String s) {
    int i = 0, n = s.length();
    if (n == 0) return false;
    if (s.charAt(0) == '+' || s.charAt(0) == '-') i++;
    if (i == n) return false;
    for (; i < n; i++) if (s.charAt(i) < '0' || s.charAt(i) > '9') return false;
    return true;
}
static final Pattern NUMBER = Pattern.compile("[+-]?(\\d+(\\.\\d*)?|\\.\\d+)([eE][+-]?\\d+)?");
for (String t : List.of("42", "-7", "+", "4a", "", "3.14", ".5", "5.", "1e10", "1e", "--1", "0x1F",
" 1")) System.out.println("[" + t + "] int=" + isInteger(t) + " number=" +
NUMBER.matcher(t).matches());
```

▶ **Output**

```
[42] int=true number=true
[-7] int=true number=true
[+] int=false number=false
[4a] int=false number=false
[] int=false number=false
[3.14] int=false number=true
[.5] int=false number=true
[5.] int=false number=true
[1e10] int=false number=true
[1e] int=false number=false
[--1] int=false number=false
[0x1F] int=false number=false
[ 1] int=false number=false
```

💡 **Pro tip**

Decide first whether you need *validation* or *parsing*. If you will parse anyway, just try `parseInt` in a try/catch (or a regex pre-check for speed).

© TechTribers. For personal use only. Do not redistribute.

Q664 ST-085

• Intermediate

Find the longest substring with at most K distinct characters (sliding window).

⚡ Quick answer

Keep a window `[left, right]` and a count map of the characters inside it. Extend `right` each step; while the window has **more than K distinct** characters, shrink from `left`, removing characters whose count drops to 0. Track the best length. O(n) time (each index enters and leaves the window once), O(K) space.

Why `merge(l, -1, sum) == 0` then `remove`: the map size is the number of distinct characters in the window, so a character must disappear from the map when its count reaches zero. K = 1 gives the longest run of one repeated character; with K = 2 this is LeetCode 159. The same template solves *longest substring without repeating characters* (Part 5) and *minimum window substring* (Part 6): expand right, shrink left while the window violates a constraint.

Java

```
static int longestKDistinct(String s, int k) {
    Map<Character, Integer> count = new HashMap<>();
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        count.merge(s.charAt(right), 1, Integer::sum);
        while (count.size() > k) {                                  // too many distinct
characters: shrink from the left
            char l = s.charAt(left++);
            if (count.merge(l, -1, Integer::sum) == 0) count.remove(l);
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}
static String longestKDistinctText(String s, int k) {
    Map<Character, Integer> count = new HashMap<>();
    int bestStart = 0, bestLen = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        count.merge(s.charAt(right), 1, Integer::sum);
        while (count.size() > k) { char l = s.charAt(left++); if (count.merge(l, -1, Integer::sum)
== 0) count.remove(l); }
        if (right - left + 1 > bestLen) { bestLen = right - left + 1; bestStart = left; }
    }
    return s.substring(bestStart, bestStart + bestLen);
}
System.out.println(longestKDistinct("eceba", 2) + " " + longestKDistinct("aa", 1) + " " +
longestKDistinct("abc", 0) + " " + longestKDistinct("", 3) + " " + longestKDistinct("araaci", 2) + "
" + longestKDistinct("araaci", 1) + " " + longestKDistinct("abcabcbb", 3));
System.out.println(longestKDistinctText("eceba", 2) + " " + longestKDistinctText("araaci", 2) + " ["
+ longestKDistinctText("abc", 0) + "]");
```

▶ Output

```
3 2 0 0 4 2 8
ece araa []
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Say the invariant: "the window always has at most K distinct characters after the while loop." Then each step either grows or shifts the window, never loses the best.

ID **Q665** ST-086 • Basic

Check whether all characters in a string are unique.

> ⚡ **Quick answer**
> Use a `HashSet` (O(n) time, O(n) space), compare `chars().distinct().count()` with the length, or for a known small alphabet a **bitmask** (O(n) time, O(1) space): one bit per letter. If the length exceeds the alphabet size (e.g. > 26 for lowercase letters), the answer is immediately false by the pigeonhole principle.

The bitmask version assumes lowercase `a-z` (it misbehaves for other characters: shifting by a large or negative amount wraps around in Java). For ASCII use a `boolean[128]` or two `long`s as a 128-bit mask. The "without extra data structure" variant sorts the characters first (O(n log n)) or compares each pair (O(n²)).

Java

```
static boolean unique(String s) { return s.chars().distinct().count() == s.length(); }
static boolean uniqueBits(String s) { int seen = 0; for (char c : s.toCharArray()) { int bit = 1 <<
(c - 'a'); if ((seen & bit) != 0) return false; seen |= bit; } return true; }
static boolean uniqueSet(String s) { Set<Character> set = new HashSet<>(); for (char c :
s.toCharArray()) if (!set.add(c)) return false; return true; }
for (String t : List.of("abcdef", "hello", "", "abcdefghijklmnopqrstuvwxyz")) System.out.println("["
+ t + "] " + unique(t) + " " + uniqueBits(t) + " " + uniqueSet(t));
System.out.println(unique("a".repeat(2)) + " " + unique("a".repeat(27)));
```

▶ **Output**

```
[abcdef] true true true
[hello] false false false
[] true true true
[abcdefghijklmnopqrstuvwxyz] true true true
false false
```

> 💡 **Pro tip**
> Mention the pigeonhole shortcut: `s.length() > 26` can only be unique if the alphabet is larger, a small detail that signals experience.

© TechTribers. For personal use only. Do not redistribute.

Q666 ST-087

• Advanced

Find the longest common substring of two strings.

⚡ Quick answer

Dynamic programming: `dp[i][j]` is the length of the common **suffix** of `a[0..i)` and `b[0..j)`. If the characters match, `dp[i][j] = dp[i-1][j-1] + 1`, otherwise 0; track the maximum and its end index. O(n·m) time and space (O(min) with a rolling row). A substring must be **contiguous**, unlike the longest common *subsequence* (Part 5).

The difference from LCS is the reset to 0 on a mismatch: that is what enforces contiguity. When several substrings tie, this version returns the first one found in `a`. Faster approaches: a **suffix automaton** or suffix array (O(n + m)), and binary search on the length with rolling hashes (O(n log n)).

Java

```
static String lcsubstr(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    int best = 0, end = 0;
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                if (dp[i][j] > best) { best = dp[i][j]; end = i; }
            }
    return a.substring(end - best, end);
}
System.out.println(lcsubstr("abcdxyz", "xyzabcd") + " " + lcsubstr("zxabcdezy", "yzabcdezx") + " ["
+ lcsubstr("abc", "xyz") + "] " + lcsubstr("hello", "yellow"));
```

▶ Output

```
abcd abcdez [] ello
```

💡 Pro tip

Compress the table to two rows when memory matters: `dp[i]` only depends on the previous row.

Q667 ST-088

• Basic

Check whether one string is a subsequence of another.

⚡ Quick answer

Use two pointers: advance through the longer string `t` and move the pointer in `s` only when the characters match; `s` is a subsequence if its pointer reaches the end. O(|t|) time, O(1) space. An `indexOf` loop (search each next character after the previous position) is equally valid and is the basis of the follow-up for **many queries** (precompute positions per character and binary-search).

© TechTribers. For personal use only. Do not redistribute.

Follow-up (LeetCode 392): millions of subsequence checks against the same `t`: build `Map<Character, List<Integer>>` of positions once, then for each character binary-search the first position greater than the previous one: O(|s| log |t|) per query. A subsequence keeps **order** but not adjacency; a substring needs adjacency.

Java

```
static boolean isSubsequence(String s, String t) { int i = 0; for (int j = 0; j < t.length() && i < s.length(); j++) if (s.charAt(i) == t.charAt(j)) i++; return i == s.length(); }
System.out.println(isSubsequence("ace", "abcde") + " " + isSubsequence("aec", "abcde") + " " + isSubsequence("", "x") + " " + isSubsequence("x", "") + " " + isSubsequence("abc", "abc"));
static boolean viaIndexOf(String s, String t) { int pos = 0; for (char c : s.toCharArray()) { pos = t.indexOf(c, pos); if (pos < 0) return false; pos++; } return true; }
System.out.println(viaIndexOf("ace", "abcde") + " " + viaIndexOf("aec", "abcde"));
```

▶ Output

```
true false true false true
true false
```

⚠️ **Interview trap**

Greedy matching is correct for subsequence checks (take the earliest match); it would not be for related problems like counting the number of distinct subsequences.

Q668 ST-089 • Basic

Count the words in a sentence (spaces, punctuation and tabs).

⚡ **Quick answer**

Define "word" first: a maximal run of non-whitespace characters. Walk the string with an `inWord` flag (O(n), no allocation), or `trim().split("\\s+")` (watch the empty-string case, which yields an array with **one** empty element), or use a `\\b\\w+\\b` regex to count letter/digit words ignoring punctuation. `StringTokenizer` also counts whitespace-separated tokens.

The last line shows why the definition matters: whitespace-separated tokens give 5 for `it's a well-known 3.5 fact`, while `\\w+` words give 8 (it, s, a, well, known, 3, 5, fact). `"".split("\\s+").length` is 1, not 0, so the split-based count is wrong for empty input unless guarded. For word *frequencies* see the word-count question in Part 13.

© TechTribers. For personal use only. Do not redistribute.

```java
static int countWords(String s) { int n = 0; boolean in = false; for (char c : s.toCharArray()) { if
(Character.isWhitespace(c)) in = false; else if (!in) { in = true; n++; } } return n; }
String text = "  Hello,   world! \n This is\tJava.  ";
System.out.println(countWords(text) + " " + text.trim().split("\\s+").length + " " +
"".split("\\s+").length + " " + "  ".trim().isEmpty() + " " + countWords("") + " " +
countWords("one"));
System.out.println(Arrays.stream(text.split("\\W+")).filter(w -> !w.isEmpty()).count() + " " +
Pattern.compile("\\b\\w+\\b").matcher(text).results().count() + " " + new
StringTokenizer(text).countTokens());
System.out.println(Pattern.compile("\\b\\w+\\b").matcher("it's a well-known 3.5
fact").results().count() + " " + countWords("it's a well-known 3.5 fact"));
```

▶ Output

```
5 5 1 true 0 1
5 5 5
8 5
```

💡 **Pro tip**

Ask: "what counts as a word?" Then pick the matching implementation instead of the most familiar one.

🔁 **Q669** ST-090 • Advanced

Count palindromic substrings in a string.

⚡ **Quick answer**

Expand around each centre: for every one of the `2n - 1` centres (each character, and each gap between two characters) expand left and right while the characters match, counting each success. O(n²) time, O(1) space. A brute-force check of all O(n²) substrings costs O(n³). Manacher's algorithm does it in O(n).

Substrings are counted by **position**, so `"aaa"` has 6 palindromic substrings (`a` ×3, `aa` ×2, `aaa`). The brute-force list is shown for verification. The longest palindromic substring (Part 5) uses the same expand-around-centre idea but tracks the best bounds. DP (`dp[i][j]` true if `s[i]==s[j]` and the inside is a palindrome) is O(n²) time and space.

```java
static int countPalSubstrings(String s) {
    int count = 0;
    for (int c = 0; c < 2 * s.length() - 1; c++) {
        int l = c / 2, r = l + c % 2;                                   // even c: single-
character centre; odd c: the gap after it
        while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { count++; l--; r++; }
    }
    return count;
}
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(countPalSubstrings("abc") + " " + countPalSubstrings("aaa") + " " +
countPalSubstrings("abba") + " " + countPalSubstrings("") + " " + countPalSubstrings("racecar"));
List<String> subs = new ArrayList<>();
String s = "aaba";
for (int i = 0; i < s.length(); i++) for (int j = i + 1; j <= s.length(); j++) { String t =
s.substring(i, j); if (new StringBuilder(t).reverse().toString().equals(t)) subs.add(t); }
System.out.println(subs);
```

▶ Output

```
3 6 6 0 10
[a, aa, a, aba, b, a]
```

💡 Pro tip

`2n - 1` centres is the detail to say out loud: odd and even length palindromes are the two cases people forget.

---

abc **Q670** ST-091 • Basic

Reverse only the vowels in a string.

⚡ Quick answer

Two pointers: move `i` forward to the next vowel and `j` backward to the previous vowel, swap them, and continue until the pointers cross. O(n) time, O(n) for the char array. Consonants and punctuation stay in place. Include upper-case vowels.

This is LeetCode 345. The inner `while` loops must keep checking `i < j`, otherwise the pointers can pass each other on strings without vowels. When `i == j` the swap is a harmless self-swap. A variant reverses **words' letters only** (keeping punctuation) with the same two-pointer idea using `Character.isLetter`.

Java

```
static String reverseVowels(String s) {
    char[] c = s.toCharArray(); int i = 0, j = c.length - 1;
    while (i < j) {
        while (i < j && "aeiouAEIOU".indexOf(c[i]) < 0) i++;
        while (i < j && "aeiouAEIOU".indexOf(c[j]) < 0) j--;
        char t = c[i]; c[i++] = c[j]; c[j--] = t;
    }
    return new String(c);
}
System.out.println(reverseVowels("hello") + " " + reverseVowels("leetcode") + " " +
reverseVowels("xyz") + " " + reverseVowels("aA") + " [" + reverseVowels("") + "]");
```

▶ Output

```
holle leotcede xyz Aa []
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Use `"aeiouAEIOU".indexOf(ch) >= 0` (or a `boolean[128]`) for the vowel test; it's shorter and faster than a `Set<Character>`.

🧮 Q671 ST-092 • Intermediate

Remove adjacent duplicates from a string using a stack (and the k-duplicates variant).

> ⚡ **Quick answer**
> Use a **stack** (a `StringBuilder` works as one): for each character, if it equals the top, pop it, otherwise push it. The result has no two equal neighbours. For the **k-duplicates** version keep `(char, count)` pairs and pop when the count reaches `k`. O(n) time and space. This is the string form of "remove matching pairs" problems.

Why a stack: removing a pair can create **new** adjacent equal characters (`abba` → `aa` → empty), and the stack naturally exposes the new neighbour after each pop. A repeated `replace("aa", "")` loop is O(n²). Related: valid parentheses (Part 5), `reduce` of `ab` / `ba` pairs, and the "remove all adjacent duplicates in a string II" problem above.

Java

```
static String removeAdjacent(String s) {
    StringBuilder st = new StringBuilder();
    for (char c : s.toCharArray()) { if (!st.isEmpty() && st.charAt(st.length() - 1) == c)
st.setLength(st.length() - 1); else st.append(c); }
    return st.toString();
}
static String removeK(String s, int k) {
    Deque<int[]> st = new ArrayDeque<>();                                    // {character, run
length}
    for (char c : s.toCharArray()) {
        if (!st.isEmpty() && st.peek()[0] == c) { if (++st.peek()[1] == k) st.pop(); }
        else st.push(new int[]{c, 1});
    }
    StringBuilder sb = new StringBuilder();
    Iterator<int[]> it = st.descendingIterator();
    while (it.hasNext()) { int[] e = it.next(); sb.append(String.valueOf((char) e[0]).repeat(e[1]));
}
    return sb.toString();
}
System.out.println(removeAdjacent("abbaca") + " [" + removeAdjacent("aabb") + "] " +
removeAdjacent("azxxzy") + " " + removeK("deeedbbcccbdaa", 3) + " " +
removeK("pbbcggttciiippooaais", 2));
```

▶ **Output**

```
ca [] ay aa ps
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Using `String` concatenation as the stack (`result = result.substring(0, n - 1)`) makes it quadratic. A `StringBuilder` with `setLength` is O(1) per pop.

🌳 **Q672** ST-093 • Intermediate

Generate all combinations of well-formed parentheses for n pairs.

⚡ **Quick answer**

Backtracking: build the string one character at a time, adding `(` while fewer than `n` opens are used and `)` while closes are fewer than opens. Each valid result has length `2n`. The number of results is the Catalan number `C(n)` (5 for n = 3, 14 for n = 4), so the work is O(4^n / √n).

The pruning rules (`open < n` , `close < open`) mean every path generates only valid prefixes: no results are generated and thrown away. The count sequence 1, 2, 5, 14, 42, 132, 429, 1430 is the Catalan numbers. Replacing `cur + "("` with a shared `StringBuilder` and `setLength` after the recursive call avoids copying. This is the template for most "generate all valid ..." problems (subsets, permutations, N-Queens in Part 6).

Java

```
static List<String> generate(int n) { List<String> out = new ArrayList<>(); build("", 0, 0, n, out);
return out; }
static void build(String cur, int open, int close, int n, List<String> out) {
    if (cur.length() == 2 * n) { out.add(cur); return; }
    if (open < n) build(cur + "(", open + 1, close, n, out);                // can still open
    if (close < open) build(cur + ")", open, close + 1, n, out);            // can close only an
open pair
}
System.out.println(generate(3) + " " + generate(1) + " " + generate(0).size() + " " +
generate(4).size());
System.out.println(IntStream.rangeClosed(1, 8).mapToObj(i ->
String.valueOf(generate(i).size())).collect(Collectors.joining(",")));
```

▶ **Output**

```
[((())), (()()), (())(), ()(()), ()()()] [()] 1 14
1,2,5,14,42,132,429,1430
```

💡 **Pro tip**

State the invariant: "a prefix is valid iff closes never exceed opens, and opens never exceed n". That is the whole solution.

© TechTribers. For personal use only. Do not redistribute.

Q673 ST-094

• Intermediate

Convert a number to an Excel column title (1 → A, 27 → AA) and back.

⚡ Quick answer

It is a **bijective base-26** system (no zero digit). To convert number → title: repeatedly do `n--`, take `n % 26` as the letter, then `n /= 26`; reverse the result. Title → number: `n = n * 26 + (letter - 'A' + 1)` for each character. The `n--` is what handles the missing zero.

Why `26 → Z` and `27 → AA`: there's no digit for zero, so a plain base-26 conversion would map 26 to `A0`. Decrementing first shifts the range `1..26` to `0..25`. Compare with ordinary radix conversion, where `Integer.toString(n, radix)` and `Integer.parseInt(s, radix)` already do the work for bases up to 36.

Java

```
static String title(int n) { StringBuilder sb = new StringBuilder(); while (n > 0) { n--;
sb.append((char) ('A' + n % 26)); n /= 26; } return sb.reverse().toString(); }
static int number(String s) { int n = 0; for (char c : s.toCharArray()) n = n * 26 + (c - 'A' + 1);
return n; }
System.out.println(title(1) + " " + title(26) + " " + title(27) + " " + title(52) + " " + title(701)
+ " " + title(702) + " " + title(703) + " " + number("AB") + " " + number("ZY") + " " +
number(title(123456)));
System.out.println(Integer.toString(35, 36) + " " + Integer.toString(255, 2) + " " +
Long.toString(-255, 16) + " " + Integer.toHexString(-1) + " " + Integer.parseInt("zz", 36));
```

▶ Output

```
A Z AA AZ ZY ZZ AAA 28 701 123456
z 11111111 -ff ffffffff 1295
```

⚠ Interview trap

Off-by-one is the usual bug: `title(26)` must be `Z`, `title(52)` `AZ`, `title(702)` `ZZ`. Test those three boundaries.

Q674 ST-095

• Intermediate

Simplify a Unix file path ("/a/./b/../../c/" becomes "/c").

⚡ Quick answer

Split on `/` and process each segment with a **stack**: ignore empty and `.`, pop on `..` (if not empty), push everything else. Join with `/` and prefix `/`. O(n). An empty stack gives `/`. Compare with `Path.normalize()`, which does the same but keeps leading `..` for relative paths.

Edge cases: `..` at the root stays at root (the `!st.isEmpty()` check); `...` is a normal name (only `.` and `..` are special); repeated slashes disappear in `split`. For security checks use `Path.normalize()` and `startsWith(base)` (see the security question). `ArrayDeque` as a stack iterates **top first**, hence `descendingIterator()` to print bottom to top.

© TechTribers. For personal use only. Do not redistribute.

```java
static String simplify(String path) {
    Deque<String> st = new ArrayDeque<>();
    for (String p : path.split("/")) {
        if (p.isEmpty() || p.equals(".")) continue;
        if (p.equals("..")) { if (!st.isEmpty()) st.pop(); }
        else st.push(p);
    }
    if (st.isEmpty()) return "/";
    StringBuilder sb = new StringBuilder();
    Iterator<String> it = st.descendingIterator();
    while (it.hasNext()) sb.append('/').append(it.next());
    return sb.toString();
}
for (String p : List.of("/home/", "/../", "/home//foo/", "/a/./b/../../c/", "/a/../../b/../c//.//",
"/...")) System.out.println(p + " -> " + simplify(p));
System.out.println(Path.of("/a/./b/../../c/").normalize());
```

▶ Output

```
/home/ -> /home
/../ -> /
/home//foo/ -> /home/foo
/a/./b/../../c/ -> /c
/a/../../b/../c//.// -> /c
/... -> /...
/c
```

> 💡 **Pro tip**
> `new ArrayDeque<>()` with `push` / `pop` is the modern stack; `Stack` is legacy (Part 12).

Q675 ST-096 • Basic

Check whether a sentence is a pangram (contains every letter a–z).

> ⚡ **Quick answer**
> Track which of the 26 letters appeared: a `boolean[26]` with a counter (O(n)), a bitmask with 26 bits, or `chars().filter(Character::isLetter).distinct().count() == 26` after lower-casing. Early-exit when the counter reaches 26. Decide about non-ASCII letters: the classic definition counts only `a-z`.

The bitmask form uses one `int`: set bit `c - 'a'`, and the sentence is a pangram when `mask == (1 << 26) - 1` (all 26 low bits set). `toLowerCase(Locale.ROOT)` is again important (Turkish `I`). The pangram also exists for other alphabets; the test generalises by the size of the alphabet.

```java
static boolean pangram(String s) {
    boolean[] seen = new boolean[26]; int n = 0;
    for (char c : s.toLowerCase(Locale.ROOT).toCharArray()) if (c >= 'a' && c <= 'z' && !seen[c -
```


© TechTribers. For personal use only. Do not redistribute.

```
'a']) { seen[c - 'a'] = true; n++; }
    return n == 26;
}
static boolean pangramStream(String s) { return
s.toLowerCase(Locale.ROOT).chars().filter(Character::isLetter).filter(c -> c <
128).distinct().count() == 26; }
String q = "The quick brown fox jumps over the lazy dog";
System.out.println(pangram(q) + " " + pangramStream(q) + " " + pangram("Hello World") + " " +
pangram("") + " " + pangram("abcdefghijklmnopqrstuvwxy"));
System.out.println(q.toLowerCase().chars().filter(Character::isLetter).distinct().sorted().mapToObj(
c -> String.valueOf((char) c)).collect(Collectors.joining()));
int mask = 0; for (char c : "abc".toCharArray()) mask |= 1 << (c - 'a');
System.out.println(Integer.toBinaryString(mask) + " " + Integer.bitCount(mask) + " " + (mask == (1
<< 26) - 1));
```

▶ Output

```
true true false false false
abcdefghijklmnopqrstuvwxyz
111 3 false
```

💡 **Pro tip**

Mention the bitmask: O(1) space and a single `int` comparison at the end.

Q676 ST-097 • Intermediate

Generate the count-and-say (look-and-say) sequence.

⚡ **Quick answer**

Start with `"1"`; each next term describes the previous term by **counting runs**: `1` → `11` (one 1) → `21` (two 1s) → `1211` → `111221`. For each term scan runs of equal characters and append `count + character`. Iterative O(total length) building with a `StringBuilder`. The terms grow by about 30% each step (Conway's constant).

Lengths: 1, 2, 2, 4, 6, 6, 8, 10, 14, 20, ... The ratio of consecutive lengths approaches Conway's constant (about 1.3036), measured here: n = 20 has 302 characters, n = 30 has 4,462, n = 40 has 63,138 and n = 50 has 894,810. The run-scanning inner loop is the same as in run-length encoding (earlier in this section); only the output format differs.

Java

```java
static String countAndSay(int n) {
    String s = "1";
    for (int i = 1; i < n; i++) {
        StringBuilder sb = new StringBuilder();
        for (int j = 0; j < s.length(); ) {
            int k = j;
            while (k < s.length() && s.charAt(k) == s.charAt(j)) k++;
            sb.append(k - j).append(s.charAt(j));
            j = k;
```


© TechTribers. For personal use only. Do not redistribute.

```
        }
        s = sb.toString();
    }
    return s;
}
for (int i = 1; i <= 6; i++) System.out.print(countAndSay(i) + " ");
System.out.println();
System.out.println(countAndSay(10).length() + " " + countAndSay(20).length());
```

▶ Output

```
1 11 21 1211 111221 312211
20 302
```

⚠️ **Interview trap**

Counting runs with `s.split("")` or regex backreferences works but allocates heavily. Two pointers with a `StringBuilder` is the expected answer.

Q677 ST-098 • Advanced

Parse a CSV line with quoted fields and escaped quotes.

⚡ **Quick answer**

A state machine over the characters with an `inQuotes` flag: outside quotes a comma ends the field and a quote starts a quoted section; **inside** quotes a comma is data, and two consecutive quotes (`""`) mean one literal quote. Never use `split(",")` for real CSV: it breaks on quoted commas and drops trailing empty fields. For production use a library (Apache Commons CSV, OpenCSV, Jackson CSV).

The parser keeps the trailing empty field in `a,,c,` (4 fields), while `"a,b,,".split(",")` returns **2**. Real CSV also has fields with **embedded newlines** (a record spans lines, so you can't parse line by line), different delimiters and BOMs (RFC 4180 is the reference). Interview extensions: handle `\r\n`, trim spaces around fields, report malformed quotes.

Java

```
static final char QUOTE = (char) 34;                                    // the double-quote character
static List<String> parseCsv(String line) {
    List<String> out = new ArrayList<>(); StringBuilder cur = new StringBuilder(); boolean quoted =
false;
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (quoted) {
            if (c == QUOTE) { if (i + 1 < line.length() && line.charAt(i + 1) == QUOTE) {
cur.append(QUOTE); i++; } else quoted = false; }
            else cur.append(c);
        } else if (c == QUOTE) quoted = true;
        else if (c == ',') { out.add(cur.toString()); cur.setLength(0); }
        else cur.append(c);
    }
    out.add(cur.toString());
```


© TechTribers. For personal use only. Do not redistribute.

```
    return out;
}
System.out.println(parseCsv("a,b,c") + " " + parseCsv("name,\"Smith, John\",42") + " " +
parseCsv("\"say \"\"hi\"\"\",x") + " " + parseCsv("a,,c,") + " " + parseCsv("").size() + " " +
"a,b,,".split(",").length);
```

▶ Output

```
[a, b, c] [name, Smith, John, 42] [say "hi", x] [a, , c, ] 1 2
```

> 💡 **Pro tip**
> Describe the two states (in/out of quotes) and the one special case (`""` inside quotes) before writing code.

🪓 Q678 ST-099 • Intermediate

Implement your own split() for a single-character delimiter (keeping empty fields).

> ⚡ **Quick answer**
> Scan with `indexOf(sep, start)` (or a manual loop): each time the separator is found, add `substring(start, idx)` and move `start` past the separator; after the loop add the **remainder**. This keeps leading, middle and trailing empty strings (like `split(sep, -1)`) and avoids regex entirely. O(n).

The loop runs to `i <= s.length()` so the **last** segment is added by the same branch as the others. `mySplit("", ',')` returns one empty string, matching `"".split(",", -1)`. The `indexOf` version supports multi-character separators and overlapping rules (`start = idx + sep.length()`), which the JDK's single-char fast path also uses internally.

Java

```
static List<String> mySplit(String s, char sep) {
    List<String> out = new ArrayList<>(); int start = 0;
    for (int i = 0; i <= s.length(); i++)
        if (i == s.length() || s.charAt(i) == sep) { out.add(s.substring(start, i)); start = i + 1;
}
    return out;
}
static List<String> splitIndexOf(String s, String sep) {
    List<String> out = new ArrayList<>(); int start = 0, idx;
    while ((idx = s.indexOf(sep, start)) >= 0) { out.add(s.substring(start, idx)); start = idx +
sep.length(); }
    out.add(s.substring(start));
    return out;
}
System.out.println(mySplit("a,b,,c,,", ',') + " " + mySplit(",", ',') + " " + mySplit("abc", ',') +
" " + Arrays.asList("a,b,,c,,".split(",", -1)).equals(mySplit("a,b,,c,,", ',')));
System.out.println(splitIndexOf("a--b----c", "--") + " " + splitIndexOf("abc", "--") + " " +
mySplit("", ',').size());
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
[a, b, , c, , ] [, ] [abc] true
[a, b, , c] [abc] 1
```

⚠️ **Interview trap**

Forgetting the final segment (adding only inside the `if (charAt == sep)` branch) drops the last field: the most common bug in this question.

✏️ **Q679** ST-100 • Intermediate

Check whether two strings are at most one edit apart (insert, delete or replace one character).

⚡ **Quick answer**

If the lengths differ by more than 1 the answer is no. Otherwise walk both strings with two pointers: on the first mismatch, if the lengths are equal skip a character in **both** (replace), else skip it only in the **longer** one (insert/delete); a second mismatch fails. O(n) time, O(1) space, much cheaper than computing the full edit distance (Part 6).

Identical strings return true here (zero edits is "at most one"); a stricter variant requiring *exactly* one edit returns false for them. `ab` / `ba` is **two** replacements, so false. This is the classic CtCI problem 1.5; the follow-up is the general edit distance (Levenshtein) in Part 6, where the DP table replaces the single-mismatch trick.

Java

```
static boolean oneEditAway(String a, String b) {
    if (Math.abs(a.length() - b.length()) > 1) return false;
    String s = a.length() <= b.length() ? a : b, l = a.length() <= b.length() ? b : a;      // s =
shorter, l = longer
    int i = 0, j = 0; boolean edited = false;
    while (i < s.length() && j < l.length()) {
        if (s.charAt(i) != l.charAt(j)) {
            if (edited) return false;
            edited = true;
            if (s.length() == l.length()) i++;                                    // replace:
advance both
        } else i++;
        j++;                                                                     //
insert/delete: advance only the longer one
    }
    return true;
}
for (String[] p : new String[][]{{"pale", "ple"}, {"pales", "pale"}, {"pale", "bale"}, {"pale",
"bake"}, {"abc", "abc"}, {"", "a"}, {"ab", "ba"}})
    System.out.println(p[0] + "/" + p[1] + " " + oneEditAway(p[0], p[1]));
```

▶ **Output**

```
pale/ple true
pales/pale true
pale/bale true
```


© TechTribers. For personal use only. Do not redistribute.

```
pale/bake false
abc/abc true
/a true
ab/ba false
```

> 💡 **Pro tip**
> Normalise the problem by always treating the shorter string as `s`. It removes half of the case analysis.

© TechTribers. For personal use only. Do not redistribute.

Series progress

679 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 15 starts at Q680.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

TechTribers TechTribers

© TechTribers. For personal use only. Do not redistribute.